





UNIVERSITE DE FIANARANTSOA

ECOLE DE MANAGEMENT ET D'INNOVATION TECHNOLOGIQUE

Mention : INFORMATIQUE

Parcours : Développement d'Application Internet et Intranet (DAII)

MEMOIRE DE FIN DE CYCLE

Pour l'obtention du Diplôme de Licence en Informatique

CONCEPTION ET REALISATION D'UNE PLATEFORME DE SUIVI DE PROJETS ET PARTENARIATS DE L'UNIVERSITE DE FIANARANTSOA

Présenté par : TSIVAHINY Yastellin Barbis

Encadreur : Dr RAKOTOARISON Tsiorinantenaina

Evaluateur : Mr MAMINIAINA Indrafo Henri

Année universitaire : 2025-2026







UNIVERSITE DE FIANARANTSOA

ECOLE DE MANAGEMENT ET D'INNOVATION TECHNOLOGIQUE

Mention : INFORMATIQUE

Parcours : Développement d'Application Internet et Intranet (DAII)

MEMOIRE DE FIN DE CYCLE

Pour l'obtention du Diplôme de Licence en Informatique

CONCEPTION ET REALISATION D'UNE PLATEFORME DE SUIVI DE PROJETS ET PARTENARIATS DE L'UNIVERSITE DE FIANARANTSOA

Présenté par : TSIVAHINY Yastellin Barbis

Encadreur : Dr RAKOTOARISON Tsiorinantenaina

Evaluateur : Mr MAMINIAINA Indrafo Henri

Année universitaire : 2025-2026

TSIVAHINY Yastellin Barbis

Développeur Web

Téléphone : 034 27 825 92 • **Email :** tsivahinyyastellinbarbis@gmail.com

Adresse : Soanierana, Fianarantsoa

LinkedIn : linkedin.com/in/yastellin-barbis-tsivahiny • **GitHub :** github.com/Yastellin

PROFIL

Étudiant en troisième année de Licence en Développement d'Applications Internet et Intranet (DAII) à l'EMIT (Université de Fianarantsoa). Sérieux, rigoureux et passionné par les nouvelles technologies, je possède une solide formation académique et des compétences pratiques dans la conception et le développement d'applications web modernes.

FORMATION

2023 – 2026 : Licence 3 en Développement d'Applications Internet et Intranet (DAII)

- Écoles de Management et d'Innovation Technologique (EMIT) – Université de Fianarantsoa

2022 : Baccalauréat Série Scientifique

EXPÉRIENCES PROFESSIONNELLES

2025 : Stagiaire en informatique — TAKALOU SARL, Fort-Dauphin

- Configuration de réseaux locaux (LAN).
- Participation à la maintenance informatique.
- Interventions techniques auprès des entreprises clientes de TAKALOU.

PROJETS ACADÉMIQUES

Gym Manager — Gestion de salle de sport associative | Technologies : React, Node.js, PostgreSQL

- Développement d'une plateforme web de gestion des membres et des abonnements.
- Gestion des adhérents, des abonnements, des paiements, du planning et des statistiques avec authentification sécurisée.

MediRdv — Gestion de rendez-vous médicaux | Technologies : Spring Boot, Vue.js, PostgreSQL

- Développement d'une plateforme web avec authentification sécurisée (Spring Security).
- Conception des interfaces utilisateur et gestion des rendez-vous en ligne.

EMIT Planning — Gestion d'emploi du temps | Technologies : ASP.NET Core MVC, MySQL

- Développement d'un système de planification avec détection automatique des conflits.
- Interface interactive avec FullCalendar, statistiques (Chart.js) et export PDF.

COMPÉTENCES TECHNIQUES

- **Langages :** JavaScript (ES6+), HTML5/CSS3, Java, PHP, Python
- **Frameworks & Bibliothèques :** Node.js, React, Vue.js, Spring Boot, Bootstrap 5, Tailwind CSS
- **Bases de données :** MySQL, PostgreSQL, Firebase
- **Outils & Environnements :** Git, GitHub, Postman, VS Code

INFORMATIONS COMPLÉMENTAIRES

LANGUES

Malagasy : Langue maternelle

Français : Courant

Anglais : Intermédiaire

QUALITÉS

- Curieux & Rigoureux
- Esprit d'analyse
- Autonome
- Travail d'équipe

CENTRES D'INTÉRÊT

- Veille technologique
- Culture & Loisirs créatifs

AVANT-PROPOS

Ce rapport est rédigé dans le cadre du mémoire de fin de cycle pour l'obtention du diplôme de Licence en informatique, parcours Développement d'Application Internet et Intranet (DAII), à l'École de Management et d'Innovation Technologique de l'Université de Fianarantsoa.

Il rend compte du stage effectué au sein d'ITDC MADA, du 17 août au 31 octobre 2026, autour du projet de plateforme de gestion et de suivi des projets et partenariats, commandité par l'équipe Projet et Partenariat de la Présidence.

Le document présente le cadre du projet, son analyse et sa conception, puis les éléments de mise en œuvre et d'évaluation. Les informations qui nécessitent une validation complémentaire sont maintenues distinctes des faits établis dans les sources consultées.

REMERCIEMENTS

La réalisation de ce rapport de stage a nécessité l'accompagnement, la disponibilité et le soutien de plusieurs personnes. Leurs conseils, leurs encouragements et leurs différentes contributions ont constitué une aide précieuse tout au long de mon stage et de la rédaction de ce présent rapport.

De ce fait, je tiens à exprimer mes sincères remerciements à toutes les personnes qui ont contribué, de près ou de loin, au bon déroulement de mon stage ainsi qu'à la réalisation de ce rapport. J'adresse particulièrement mes remerciements :

- À **Monsieur le Président de l'Université de Fianarantsoa**, Professeur **HAJALALAINA Aimé Richard**, pour son engagement en faveur de la formation universitaire ;
- À **Madame la Docteure FANJANIAINA Marie Lucia**, Directrice de l'Équipe Projets et Partenariats de la Présidence de l'Université de Fianarantsoa, ainsi qu'à l'ensemble des membres de son équipe, pour la confiance qu'ils m'ont accordée en me permettant de concevoir une solution adaptée à leurs besoins, ainsi que pour leur disponibilité et leur collaboration tout au long de ce projet ;
- Au **Professeur RAKOTONIRAINY Hasina Lalaina**, Directeur de l'École de Management et d'Innovation Technologique (EMIT), pour m'avoir accueilli au sein de cet établissement et m'avoir permis de poursuivre mes études ;
- Au **Docteur TAREHY Brice Evrard**, Responsable de la Mention Développement d'Applications Intranet et Internet (DAII), pour son accompagnement tout au long de mon parcours universitaire ;
- Au **Docteur HERY ANTENAINA Paul Franck**, Directeur Gérant de **ITDC MADA**, pour m'avoir accueilli au sein de son entreprise et m'avoir donné l'opportunité d'effectuer mon stage dans un environnement professionnel ;
- Au **Docteur RAKOTOARISON Tsiorinantenaina**, mon encadreur professionnel, pour sa disponibilité, ses conseils, son accompagnement et le partage de ses connaissances tout au long de mon stage ;
- À l'ensemble de l'équipe informatique de **ITDC MADA**, ainsi qu'à mes collègues, pour leur accueil, leur collaboration, leur esprit d'équipe et leur soutien durant cette période de stage.

Par ailleurs, j'adresse mes profonds remerciements à ma famille pour son soutien moral et financier, ses encouragements et sa confiance tout au long de mes études. Leur présence et leurs sacrifices ont constitué un soutien précieux dans la réalisation de mon parcours universitaire.

Enfin, je remercie toutes les personnes qui ont contribué, directement ou indirectement, à la réussite de mon stage et à l'élaboration de ce rapport. Qu'elles trouvent ici l'expression de ma profonde gratitude et de mes sincères remerciements.

Résumé

Ce mémoire présente la conception et la réalisation d'une plateforme de gestion et de suivi des projets et partenariats de l'Université de Fianarantsoa. Le projet a été commandité par l'équipe Projet et Partenariat de la Présidence et le stage s'est déroulé au sein de l'entreprise d'accueil ITDC MADA, du 17 août au 31 octobre 2026. La problématique porte sur l'organisation des informations et la différenciation des accès selon les responsabilités des utilisateurs. La plateforme distingue les profils Présidence, Établissement et Décideur, ainsi qu'un espace public sans authentification. Le backend utilise NestJS et TypeScript ; la persistance repose sur TypeORM et PostgreSQL 16. Le frontend est développé avec React et Vite, et les services sont définis dans Docker Compose. Les fonctions concernent notamment les projets, partenariats, conventions, documents, publications, notifications, tableaux de bord et exports. L'évaluation distingue les fonctions présentes dans le code des résultats de tests effectivement documentés. Les activités et le chronogramme ne sont pas présentés comme des fonctions opérationnelles.

Mots clés : projet universitaire ; partenariat ; plateforme web ; NestJS ; TypeORM ; PostgreSQL ; contrôle des accès.

Abstract

This dissertation presents the design and implementation of a project and partnership management platform for the University of Fianarantsoa. The project was commissioned by the Projects and Partnerships team of the Presidency, and the internship took place at ITDC MADA, the host company, from 17 August to 31 October 2026. The problem addressed concerns the organization of information and the allocation of access rights according to user responsibilities. The application distinguishes the Presidency, Institution and Decision-Maker roles, together with a public area that does not require authentication. The backend uses NestJS and TypeScript; persistence relies on TypeORM and PostgreSQL 16. The frontend is built with React and Vite, while Docker Compose defines the development services. The application covers projects, partnerships, agreements, documents, publications, notifications, dashboards and exports. The evaluation distinguishes features present in the code from test results that have actually been documented. Activities and timeline management are not presented as operational features.

Keywords: university project; partnership; web platform; NestJS; TypeORM; PostgreSQL; access control.

LISTE DES FIGURES

Figure 1.1 : Organigramme de l'EMIT 11
Figure 2.1 : Méthode de travail 14
Figure 2.2 : Organigramme ITDC MADA 15
Figure 3.1 : Diagramme de Gantt de réalisation du projet 26
Figure 4.1 : Diagramme de flux de l'organisation actuelle 29
Figure 5.1 : Diagramme de cas d'utilisation global 48
Figure 5.2 : Séquence système de la soumission d'un projet 51
Figure 5.3 : Modèle de domaine des principaux concepts de la plateforme 52
Figure 6.1 : Séquence de conception de la soumission d'un projet 55
Figure 6.2 : Diagramme de classes du traitement de la soumission d'un projet 56
Figure 6.3 : Diagramme de paquetage de l'application 57
Figure 7.1 : Espace de travail de Draw.io 60
Figure 7.2 : Exécution des migrations de la base de données 61
Figure 7.3 : Fichier docker-compose.yml 62
Figure 7.4 : Authentification JWT — login 63
Figure 7.5 : Contrôleur des projets — création 64
Figure 7.6 : Service des projets — création 65
Figure 7.7 : Service des projets — soumission 66
Figure 7.8 : Contrôleur des projets — publication 67
Figure 7.9 : Service des projets — publication 67
Figure 7.10 : Service des conventions — création 68
Figure 7.11 : Service des conventions — mise à jour 69
Figure 7.12 : Notifications d'expiration des conventions 70
Figure 7.13 : Service des tableaux de bord 71
Figure 8.1 : Page de connexion de la plateforme 73
Figure 8.2 : Tableau de bord de la Présidence 74
Figure 8.3 : Gestion des établissements 74
Figure 8.4 : Gestion des partenaires 75
Figure 8.5 : Gestion des partenariats 76

Figure 8.6 : Création d'un projet par un établissement 76
Figure 8.7 : Soumission d'un projet par un établissement 77
Figure 8.8a : Publication d'un projet par la Présidence 78
Figure 8.8b : Demande de modification d'un projet 78
Figure 8.9 : Gestion des conventions par statut 79
Figure 8.10 : Téléversement et consultation des documents 80
Figure 8.11 : Notification liée à une soumission d'un nouveau projet 81
Figure 8.12 : Consultation des tableaux de bord 81
Figure 8.13 : Espace public de la plateforme 82

LISTE DES TABLEAUX

Tableau 3.1 : Acteurs et besoins fonctionnels de la plateforme 19
Tableau 3.2 : Synthèse des besoins fonctionnels et des exigences retenues 20
Tableau 3.3 : Moyens humains mobilisés pour la réalisation du projet 22
Tableau 3.4 : Moyens matériels mobilisés 22
Tableau 3.5 : Principaux moyens logiciels utilisés 23
Tableau 3.6 : Planification initiale du projet 25
Tableau 4.1 : Moyens humains et matériels mobilisés 30
Tableau 4.2 : Technologies principales du backend et du frontend 32
Tableau 5.1 : Dictionnaire des données de la plateforme 36
Tableau 5.2 : Règles de gestion proposées pour la plateforme 44
Tableau 5.3 : Description textuelle du cas d'utilisation « S'authentifier » 49
Tableau 5.4 : Description textuelle du cas d'utilisation « Créer et soumettre un projet » 50
Tableau 9.1 : Comparaison entre les pratiques antérieures et les fonctionnalités de la plateforme 83

LISTE DES ABREVIATIONS

AES — Administration économique et sociale.

API — Interface de programmation d'application.

CIGSI — Conception, intégration et gestion des systèmes d'information.

CORS — Cross-Origin Resource Sharing (partage des ressources entre origines).

CSV — Comma-Separated Values (valeurs séparées par des virgules).

CSS — Cascading Style Sheets (feuilles de style en cascade).

CUFP — Centre Universitaire de Formation Professionnalisant (forme utilisée dans le rapport).

DAII — Développement d'Application Internet et Intranet.

DOC/DOCX — Formats de documents Microsoft Word.

DTO — Data Transfer Object (objet de transfert de données).

EMIT — École de Management et d'Innovation Technologique.

HTTP — Hypertext Transfer Protocol (protocole de transfert hypertexte).

ICM — Information, Communication et Multimédia.

ITDC MADA — Nom de l'entreprise d'accueil; le sigle n'est pas développé dans les sources examinées.

JPEG — Joint Photographic Experts Group (format d'image JPEG).

JSON/JSONB — JavaScript Object Notation; JSONB est la représentation binaire correspondante utilisée par PostgreSQL.

JWT — JSON Web Token.

LMD — Licence-Master-Doctorat.

M1 — Première année de master.

MD — Management décisionnel (parcours de formation cité dans le rapport).

MEDA — Management d'entreprises et développement des affaires.

PDF — Portable Document Format.

PNG — Portable Network Graphics (format d'image).

REST — Representational State Transfer (style d'architecture d'API).

RG — Règle de gestion; préfixe des identifiants de règles dans le rapport.

S7 — Septième semestre.

SQL — Structured Query Language (langage de requête structuré).

UML — Unified Modeling Language (langage de modélisation unifié).

URL — Uniform Resource Locator (adresse d'une ressource).

UUID — Universally Unique Identifier (identifiant universellement unique).

XLSX — Format de classeur Microsoft Excel Open XML.

ZIP — Format d'archive ZIP.

GLOSSAIRE

API — Interface HTTP utilisée par le frontend pour accéder aux fonctions du backend.

Authentification — Vérification de l'identité d'un utilisateur avant l'accès aux fonctions protégées.

Backend — Partie serveur qui applique les règles métier, contrôle les accès et échange avec la base de données.

Base de données relationnelle — Dépôt de données organisé en tables liées; le projet utilise PostgreSQL.

Conteneur — Environnement isolé utilisé pour exécuter un service; les services de développement sont décrits avec Docker Compose.

DTO — Objet qui transporte les données d'une requête et dont les champs sont validés par le backend.

Endpoint — Point d'accès HTTP associé à une opération de l'API.

Frontend — Interface web développée avec React et Vite dans le projet.

JWT — Jeton signé utilisé dans le mécanisme d'authentification décrit par le projet.

ORM — Outil qui relie les objets du code aux données relationnelles; TypeORM assure cette fonction dans le projet.

Persistance — Conservation des données au-delà de l'exécution d'une requête ou d'un processus.

Rôle — Profil d'accès qui détermine les actions disponibles pour un utilisateur.

TypeORM — Outil de persistance utilisé par le backend pour accéder à PostgreSQL.

UML — Langage de modélisation utilisé pour représenter notamment les cas d'utilisation, séquences et classes.

UUID — Identifiant universellement unique utilisé dans le modèle de données présenté.

SOMMAIRE (3 NIVEAUX)

AVANT-PROPOS 6

REMERCIEMENTS I

Résumé III

Abstract III

LISTE DES FIGURES IV

LISTE DES TABLEAUX VI

LISTE DES ABREVIATIONS VII

GLOSSAIRE VIII

SOMMAIRE (3 NIVEAUX) X

INTRODUCTION 1

PARTIE I : PRESENTATION GENERALE 3

CHAPITRE I : PRÉSENTATION DE L'ÉCOLE de Management et d'Innovation Technologique 4

1.1 Historique : 4

1.2 Missions 5

1.3 Formations existantes 5

1.4 Organigramme de l'EMIT 10

CHAPITRE II : PRÉSENTATION D'ITDC MADA 12

2.1 Historique 12

2.2 Services 12

2.3 Formations 13

2.4 Mission 13

2.6 Méthode du travail 14

CHAPITRE III : PRÉSENTATION DU PROJET 16

3.1. Formulation du projet 16

3.2. Justification de l'importance du sujet 17

3.3. Objectifs et besoins des utilisateurs 17
3.4. Moyens nécessaires à la réalisation du projet 20
3.5. Chronogramme de réalisation du projet 23
3.6. Résultat attendu 25
PARTIE II : ANALYSE ET CONCEPTION 27
CHAPITRE IV : ANALYSE PRÉALABLE 28
4.1. Analyse de l'existant 28
4.2. Choix de la solution 30
4.3. Choix de la méthode de conception et de gestion du projet 30
4.4. Choix de l'outil de conception 31
4.5. Choix du langage de programmation 31
4.6. Choix des frameworks et des technologies d'interface 31
4.7. Choix de l'environnement de développement 32
4.8. Choix du système de gestion de base de données 33
CHAPITRE V : CONCEPTION DU PROJET 34
5.1. Présentation de la méthode de conception 34
5.2. Dictionnaire des données 34
5.3. Règles de gestion 42
5.4. Diagramme de cas d'utilisation 45
5.5. Description textuelle des cas d'utilisation 47
5.6. Diagramme de séquence système 49
5.7. Modèle de domaine 49
CHAPITRE VI : CONCEPTION DÉTAILLÉE 51
6.1. Architecture de l'application 51
6.2. Diagramme de séquence de conception 52
6.3. Diagramme de classe 54
6.4. Diagramme de paquetage 55

PARTIE III : MISE EN ŒUVRE ET RÉSULTATS 56
CHAPITRE VII : MISE EN ŒUVRE 57
7.1. Architecture du système 57
7.2. Configuration des outils de conception 57
7.3. Génération de la base de données 58
7.4. Configuration de l'environnement de développement 59
7.5. Extraits de code des fonctionnalités principales 61
7.6. Tests réalisés 69
CHAPITRE VIII : PRÉSENTATION DE L'APPLICATION 71
8.1. Authentification 71
8.2. Tableau de bord de la Présidence 71
8.3. Gestion des établissements 72
8.4. Gestion des partenaires 73
8.5. Gestion des partenariats 73
8.6. Création et suivi d'un projet 74
8.7. Soumission d'un projet 75
8.8. Publication et demande de modification d'un projet 75
8.9. Gestion des conventions 77
8.10. Téléversement et consultation des documents 77
8.11. Notifications 78
8.12. Consultation des tableaux de bord 79
8.13. Espace public 79
CHAPITRE IX : ÉVALUATION ET SUGGESTIONS 81
9.1. Évaluation de l'application 81
9.2. Contributions académiques et professionnelles 82
9.3. Limitations de l'étude et pistes de recherche futures 82
CONCLUSION 84

BIBLIOGRAPHIE XII
Sources primaires du projet ..... **Erreur ! Signet non défini.**
Ouvrages et normes de référence XII
WEBOGRAPHIE XII
Dépôt GitHub du projet ..... **Erreur ! Signet non défini.**
GLOSSAIRE XIII
ANNEXES XIV
Annexe A — Sources techniques de la version examinée XIV
TABLE DES MATIÈRES XV

INTRODUCTION

L'Université de Fianarantsoa entretient des relations avec plusieurs établissements et partenaires à travers des projets, des partenariats et des conventions. Le suivi de ces relations produit des informations administratives, institutionnelles et documentaires portant notamment sur les partenaires, les établissements, les projets, les documents, les conventions et leurs échéances. Ces données sont utilisées par la Présidence et les établissements et nécessitent une organisation cohérente ainsi qu'un accès adapté aux responsabilités de chaque acteur. C'est dans ce contexte que s'inscrit le projet de « Plateforme de gestion et de suivi des projets et partenariats de l'Université de Fianarantsoa ». J'ai effectué mon stage au sein de l'entreprise ITDC MADA, du 17 août 2026 au 31 octobre 2026. L'équipe Projet et Partenariat de la Présidence a commandité le projet. Ma contribution a principalement porté sur la conception et la mise en œuvre du backend. L'objectif du projet a consisté à regrouper les informations utiles au suivi dans une application web et à fournir aux utilisateurs un accès correspondant à leurs rôles.

Avant la réalisation de la plateforme, le suivi reposait sur des systèmes internes, des fichiers Excel ou plusieurs supports ne formant pas une solution centralisée. Cette organisation pouvait disperser les informations, compliquer les recherches et rendre difficile le rapprochement entre un projet, son établissement, ses partenaires, ses documents et ses conventions. Le suivi des dates de fin des conventions constituait également un besoin important. La problématique a donc consisté à concevoir une solution centralisant les données, structurant les opérations et différenciant les accès. La plateforme prend en compte quatre acteurs : la Présidence, l'Établissement, le Décideur et le Visiteur. Les trois premiers disposent de comptes authentifiés associés aux rôles `PRESIDENCE`, `ETABLISSEMENT` et `DECIDEUR`, tandis que le Visiteur accède à l'espace public sans compte. La question étudiée porte ainsi sur la conception d'une application web cohérente, maintenable et suffisamment sécurisée pour accompagner le suivi des projets, des partenaires, des partenariats, des conventions et des documents, tout en organisant les échanges entre l'interface, l'API backend et la base de données.

L’hypothèse retenue est qu’une application web organisée par modules métier, avec des couches internes et une séparation entre l’interface, l’API et la base de données, améliore la structuration du suivi. La solution utilise React/Vite pour le frontend, NestJS et TypeScript pour le backend, TypeORM pour l’accès aux données et PostgreSQL 16 pour leur conservation. L’API REST est préfixée par `/api`, l’authentification repose sur JWT et les données sont contrôlées par des DTO et le `ValidationPipe`. Les rôles et le périmètre de l’établissement limitent l’accès aux ressources concernées. La plateforme gère les projets, les partenaires, les partenariats, les conventions par statut, les documents, les notifications d’expiration et les tableaux de bord. Les activités et le chronogramme ne font pas partie du périmètre fonctionnel retenu. Docker Compose exécute les trois services de l’application, tandis que les migrations TypeORM gèrent la structure de la base. Les tests effectivement exécutés, leurs commandes et leurs résultats sont distingués des outils et scénarios de test prévus ; ils sont rapportés à la section 7.6 uniquement lorsqu’ils sont attestés. Cette organisation met en évidence la contribution du backend dans la validation des données, l’application des règles d’accès, les traitements métier et la communication avec la base.

Ce mémoire se compose de trois parties. La première présente le cadre général, l’établissement, le service d’accueil et le projet réalisé. La deuxième porte sur l’analyse et la conception : étude de l’existant, choix techniques, règles de gestion, cas d’utilisation, modèle de domaine et conception détaillée. La troisième décrit la mise en œuvre, la configuration de l’environnement, la base de données, les fonctionnalités backend, la présentation de l’application et son évaluation. Le chapitre 8 suit un parcours de démonstration centré sur les comptes Présidence et Établissement, puis présente l’espace public. Le chapitre 9 évalue les apports de la solution et ses limites. Le mémoire se termine par une conclusion générale, la bibliographie, la webographie, le glossaire et les annexes. L’ensemble met en évidence la contribution du développement backend à la centralisation du suivi des projets et des partenariats de l’Université de Fianarantsoa.

PARTIE I : PRESENTATION GENERALE

CHAPITRE I : PRÉSENTATION DE L'EMIT

L'Ecole de Management et d'Innovation Technologique (EMIT) est une grande école publique au sein de l'Université de Fianarantsoa, qui a basculé totalement vers le système L.M.D. L'EMIT nous a reconnu comme étudiants.

1.1 Historique :

L'EMIT est une école universitaire publique pluridisciplinaire, rattachée à l'Université de Fianarantsoa. La grande maturité au niveau de l'enseignement et la compétence des étudiants sortants de l'établissement ont permis aux dirigeants sous l'approbation du Ministère de mettre en place la conversion du Centre en Ecole au sein de l'Université de Fianarantsoa par le Décret N°2016-1394 du 15 Novembre 2016. L'EMIT prépare d'une part le diplôme de Master en deux mentions selon trois parcours et d'autre part le diplôme de Licence en trois mentions selon cinq parcours.

Auparavant, elle était connue sous le nom du Centre Universitaire de Formation Professionnalisant (CUFP), créée par le Décret N°2005-205 du 26 Avril 2005 et dispensait le diplôme de Licence professionnelle en Administration ainsi qu'en Informatique. Mais avant cela, elle était connue également sous le nom du Centre de Formation Continue (CFC), créé par l'Arrêté Rectoral N°99-23/UF/R du 10 Mars 1999 qui formait de diplôme de Technicien Supérieur.

L'EMIT a été sélectionnée « Meilleur Etablissement » pendant le Salon de la Recherche organisé par l'Organisation Internationale du Travail les 5 et 6 Juillet 2017. Depuis l'année universitaire 2013-2014, l'école est basculée totalement vers le système Licence, Master et Doctorat (LMD). Toutes les offres de formation dispensée à l'EMIT sont habilitées par le Ministère de l'Enseignement Supérieur et de la Recherche Scientifique. L'école propose neuf parcours répartis en trois mentions, Présentes ci-dessous :

Cycle	Mention Management	Mention Informatique	Mention Information, Communication et Multimédia
Licence	Administration Economique et Sociale (AES)	Développement d'Application Internet/Intranet (DAII)	Information et communication (IC)
		Conception, Intégration et Gestion des Systèmes d'Information (CIGSI)	Communication Multimédia (CM)
Master	Management Décisionnel (MD)	Ingénierie Géo spatiale et Technologies de l'Information (IGTI)	**Communication Multimédia (CM)**
	Management d'Entreprises et Développement des Affaires (MEDA)	**Modélisation et Ingénierie Informatique (M2I)**	**Information et Communication (IC)**
		Système d'Information, Géomatique et Décision (SIGD)	**Communication Numérique et Management de Projet (CNMP)**
		Sciences des Données et Intelligence Artificielle (SDIA)	

Source : Ecole de Management et d'Innovation Technologique.

1.2 Missions

L'école a pour mission, d'abord de dispenser des formations initiales et continues en informatique, en administration et en relations publiques et multimédia. Ensuite, elle offre des services connexes à l'informatique. Puis, elle forme des techniciens opérationnels immédiatement au sein des entreprises. Enfin, elle assure le perfectionnement professionnel des étudiants, des demandeurs d'emploi, des employés et des cadres d'entreprises.

1.3 Formations existantes

L'école présente actuellement deux cycles : Licence et Master. Chaque cycle possède plusieurs parcours assurés par un chef de mention.

- **Cycle Licence**

L'école possède trois (03) mentions pour cinq (05) parcours en cycle Licence. Les étudiants toutes mentions confondues doivent en faire, un stage de réalisation en entreprise avec un rapport de stage soutenu pour les étudiants en deuxième année de Licence (L2) et un stage de fin d'études suivi de la soutenance d'un mémoire pour les étudiants en troisième année de Licence (L3). La formation dure trois années universitaire et à la fin de la formation, les étudiants obtiennent des diplômes de Licence.

- **Mention « Management », Parcours « Administration Economique et Sociale (AES) »**

La condition d'accès en première année de Licence se fait par voie de concours pour les titulaires d'un Baccalauréat général de toutes séries (A, C, D, S, L, OSE) ou d'un Baccalauréat technique G1 et G2. A l'issue de la formation, les étudiants ont les compétences de :

- Assister le Directeur Général, le Directeur des Ressources Humaines et le Directeur
- Administratif et Financier ;
- Gérer les Ressources Humaines ;
- Gérer une entreprise ou un projet.

- **Mention « Informatique », Parcours « Développement d'Application Internet Intranet (DA2I) » et Parcours « Conception, Intégration et Gestion des Systèmes Information (CIGSI) »**

La condition d'accès en première année de Licence se fait par voie de concours pour les élèves titulaires du diplôme de Baccalauréat Général Scientifique (Série C, série D et série S), Baccalauréat Technique Professionnelle et Baccalauréat Technique Technologique (Filière Industrielle, Maintenance Automobile, Ouvrage Bois, Ouvrage Métallique, Génie Civile). A l'issue de la formation, les étudiants sont compétents en :

- Administration des bases de données ;
- Administration des réseaux et systèmes informatiques ;
- Développement d'application client/serveur

- **Mention « Information, Communication et Multimédia (ICM) », Parcours « Information et Communication (IC) » et « Communication Multimédia (CM) »**

La condition d'accès en première année de Licence se fait par voie de concours pour les titulaires d'un Baccalauréat général de toutes séries (A, C, D, S, L, OSE) ou d'un

Baccalauréat technique G1 et G2. A l'issue de la formation, les étudiants ont les compétences de :

- Rédiger un article dans un journal ;
- Occuper un poste d'un technicien de presse ;
- Travailler dans la revue de presse.

- **Cycle Master**

L'Ecole de Management et d'Innovation Technologique (EMIT) possède deux (03) mentions pour trois (09) parcours en Master Professionnel ainsi qu'en Master Recherche. La durée de la formation est de quatre semestres c'est-à-dire deux années universitaires.

- **Mention Management, Parcours Management Décisionnel (MD)**

La condition d'accès en première année de Master (M1) en S7 se fait par sélection de dossier après l'obtention du diplôme de Licence en Administration Economique et Sociale, en Gestion ou en Economie.

Le master en management décisionnel propose une formation approfondie et pluridisciplinaire, façonnant les étudiants dans les sphères de l'administration privée et publique. En mettant l'accent sur la stratégie décisionnelle, sur l'analyse de l'environnement d'intervention, la gestion des risques, les diplômés sont armés pour aborder des défis complexes. L'expertise des étudiants est fortement sollicitée dans divers secteurs, allant de la gestion d'entreprise à l'administration publique, où ils jouent un rôle crucial dans la prise de décision. En combinant une compréhension approfondie des disciplines et des compétences aiguisées en matière de décision, ce master offre une contribution significative à la réussite des organisations dans un environnement dynamique.

- **Management d'Entreprises et Développement des Affaires (MEDA)**

La condition d'accès en première année de Master (M1) en S7 se fait par sélection de dossier après l'obtention du diplôme de Licence en Administration Economique et Sociale, en Gestion ou en Economie.

Le master en management d'entreprises et développement des affaires prépare les étudiants à exceller dans la création et la constitution d'entreprises. Les diplômés acquièrent des compétences clés pour élaborer des plans d'affaires solides, identifier les opportunités de marché et gérer efficacement les ressources. Les étudiants sont bien positionnés pour fonder leur propre entreprise en tant qu'entrepreneurs prospères ou occuper des rôles clés dans des start-ups. Les débouchés incluent la création d'entreprises innovantes, la gestion de projets entrepreneuriaux et la consultation stratégique pour les nouvelles initiatives commerciales. Ce master offre ainsi une voie dynamique vers l'entreprenariat et le succès dans le domaine des affaires.

- **Mention « Informatique », Parcours « Système d'Information, Géomatique et Décision (SIGD) » et Parcours « Modélisation et Ingénierie Informatique (M2I) »**

La condition d'accès en première année de Master (M1) en S7 se fait par sélection de dossier après l'obtention du diplôme de Licence en Informatique et en Mathématique et Informatique pour les Sciences Sociales (MISS). Le Recrutement en S9 se fait par validation des crédits acquis.

Le parcours « Système d'Information, Géomatique et Décision » a pour objectif de donner un panorama des recherches actuelles et émergeantes en terme de système d'aide à la décision. En effet, les systèmes informatiques et la géomatique sont en plein essor, par les grilles de calcul et les multiples appareils mobiles intégrant des systèmes informatiques de plus en plus performants et complexes. Ces systèmes informatiques intégrant un parallélisme massif ou /et une mobilité des composants représentent un défi pour le génie logiciel qui doit fournir de nouvelles méthodes et des outils de production de logiciel pour la description de l'architecture de ces systèmes complexes et pour leur validation et/ou certification. De plus, l'effervescence des techniques en géomatique qui fournissent des données spatiales et temporelles dans différents domaines représente des moyens efficaces pour prendre les bonnes décisions.

- **Mention « Informatique », Parcours « Ingénierie Géo spatiale et Technologies de l'Information (IGTI) » et Parcours « Sciences des Données et Intelligence Artificielle (SDIA) »**

La condition d'accès en première année de Master (M1) en S7 se fait par sélection de dossier après l'obtention du diplôme de Licence en Informatique et en Mathématique et Informatique pour les Sciences Sociales (MISS). Le Recrutement en S9 se fait par validation des crédits acquis.

Ce parcours vise à former des experts en informatique de haut niveau, spécialisés en intelligence artificielle et science des données. Les étudiants développeront des compétences clés telles que la conception et le développement de logiciels pour l'analyse automatique de grandes masses de données, la programmation du Big Data et de l'IA, la modélisation et l'automatisation du raisonnement sur les données, la conception d'architectures d'entreprise, l'application des méthodes statistiques, la maîtrise des techniques de gestion d'entreprise, l'utilisation de technologies pour l'exploitation de données réelles.

Ce parcours met l'accent sur le développement de solutions en géomatique en utilisant une expertise avancée dans la conception de systèmes d'information géographique, la télédétection et l'analyse spatiale. Les étudiants qui le suivent sont aptes à proposer des solutions informatiques adaptées aux besoins des utilisateurs en géomatique.

- **Mention « Information, Communication et Multimédia », Parcours « Communication Multimédia (CM) »**

La condition d'accès en première année de Master (M1) en S7 se fait par sélection de dossier après l'obtention du diplôme de Licence en **Communication Multimédia (CM).** Le Recrutement en S9 se fait par validation des crédits acquis.

Le secteur de la communication multimédia est un secteur extrêmement varié et concurrentiel. Cette formation s'appuie par exemple, sur la recherche et traitement d'information avant la mise en publication (dans un site web, radio, télévision).

Le double diplôme entre le parcours Cultures et Métiers du Web (CMW) de l'Université Gustave Eiffel et le parcours Communication Multimédias (CM) de l'Université de Fianarantsoa propose l'obtention simultanée d'un diplôme de master en CMW et d'un diplôme de master en CM, avec deux suppléments de diplôme

- **Mention « Information, Communication et Multimédia », Parcours « Communication Numérique et Management de Projet(CNMP) »**

La condition d'accès en première année de Master (M1) en S7 se fait par sélection de dossier après l'obtention du diplôme de Licence en **Communication Multimédia (CM).** Le Recrutement en S9 se fait par validation des crédits acquis.

La formation en Communication Numérique et Management de Projet de l'EMIT vise à former des professionnels polyvalents, compétents dans la gestion de projets numériques, le leadership d'équipes, et la compréhension des enjeux des médias numériques, en collaboration avec l'Université Bordeaux Montaigne en France.

- **Mention « Information, Communication et Multimédia », Parcours « Information et Communication »**

La condition d'accès en première année de Master (M1) en S7 se fait par sélection de dossier après l'obtention du diplôme de Licence en **Information et Communication.** Le Recrutement en S9 se fait par validation des crédits acquis.

Le parcours Information et Communication vise à former des professionnels capables de comprendre et d'analyser les dynamiques de production, de diffusion et de réception de l'information dans divers contextes sociaux, culturels et technologiques. Il met l'accent sur le développement de compétences en communication écrite et orale, en gestion de l'information, ainsi qu'en maîtrise des outils numériques et médiatiques. Les étudiants apprennent à concevoir et mettre en œuvre des stratégies de communication adaptées aux besoins des organisations, des médias ou des institutions.

1.4 Organigramme de l'EMIT

La structure hiérarchique au sein de l'EMIT se compose d'un conseil (scientifique ou d'établissement), une direction, un collège des enseignants, des chefs de mentions et des services présents au sein de l'école. Cette structure est représentée de manière générale sur la figure 1.1.

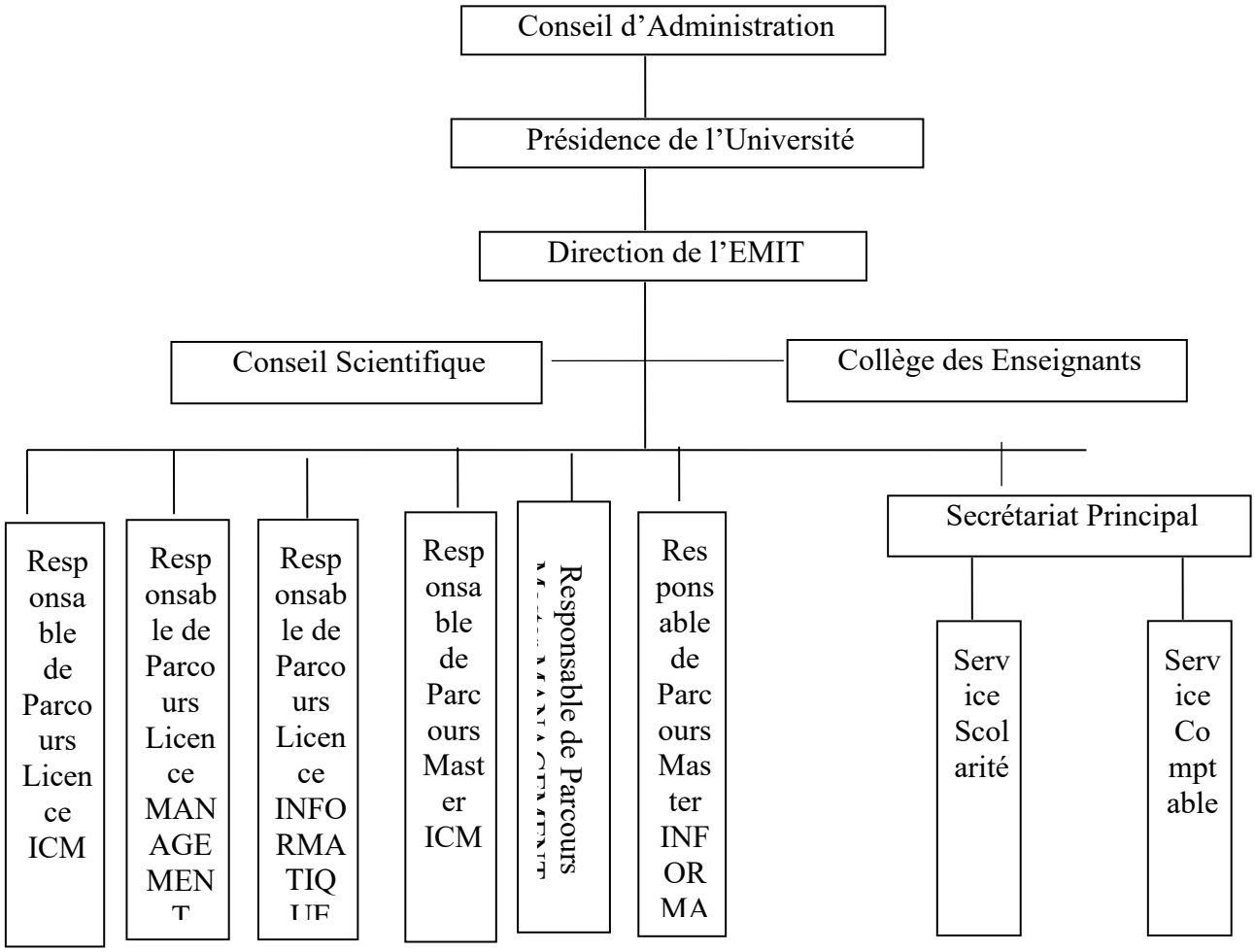


Figure 1.1 : Organigramme de l'EMIT

Source : Ecole de Management et d'Innovation Technologique

CHAPITRE II : PRÉSENTATION D'ITDC MADA

2.1 Historique

La société a été créé afin d'attirer l'attention des étudiants sortant des écoles informatiques. Les objectifs de l'ITDC MADA s'articulent autour de plusieurs axes complémentaires qui reflètent sa mission et son engagement. Elle vise à accompagner numériquement les entreprises en leur proposant des solutions digitales adaptées, telles que des applications mobiles, web ou de bureau, afin d'améliorer leur performance et leur compétitivité. Elle vise aussi à promouvoir un développement durable et local en mobilisant les outils de la géomatique dans ses actions environnementales, urbaines, rurales et économiques. Par ailleurs, l'ITDC MADA accorde une grande importance à la collecte, au traitement et à l'analyse de données, qu'elles soient spatiales ou tabulaires, pour renforcer la prise de décision dans les projets de développement. Enfin, elle mise sur la synergie d'une équipe pluridisciplinaire composée de spécialistes issus de différents domaines, afin de garantir la réussite et la pertinence des actions menées.

2.2 Services

ITDC MADA propose une large gamme de services allant du :

2.2.1 Développement d'application web

ITDC MADA accompagne les entreprises dans la création de site web pour renforcer leur visibilité et valoriser leurs activités. La société conçoit également des applications web personnalisée afin d'optimiser les processus internes et faciliter la gestion de tâches professionnelles.

2.2.2 Développement d'applications de bureau

ITDC MADA propose un service de développement d'applications de bureau, conçues pour répondre aux besoins internes des entreprises. Ces applications facilitent la gestion quotidienne et automatisent les tâches pour améliorer l'efficacité opérationnelle.

2.2.3 Applications mobiles

L'équipe d'ITDC MADA dispose de l'expertise nécessaire pour concevoir et développer des applications mobiles, qu'elles soient natives ou hybrides. Des solutions sur mesure sont mises en place afin de répondre aux besoins spécifiques des clients, tout en garantissant une expérience utilisateur fluide et optimale

2.2.4 Géomatique

Dans le domaine de SIG, ITDC MADA est la première référence de la géomatique à Madagascar. Leur équipe de développeurs est à la hauteur d'effectuer une analyse thématique à partir des données spatiales, puis de l'intégrer aux cartes et aux données sur un site internet ou intranet avec webmapping.

2.3 Formations

Apprendre, évoluer, entreprendre. ITDC MADA offre des formations concrètes pour acquérir des compétences utiles et durables comme :

La création de site web

Développement d'application mobile

Initiation au marketing digital

Rédaction web

Initiation à la géomatique, le traitement de données SIG

Maîtrise des outils bureautiques et initiation à l'informatique

2.4 Mission

A travers ses services, ITDC MADA a pour mission de promouvoir le développement numérique et technologique en proposent des solutions innovantes développement web, mobile et en géomatique pour répondre aux besoins de nos clients.

2.5 Client de l'ITDC MADA

MADA accueille une grande diversité de clients, sans distinction de statut ou de taille. Elle collabore aussi bien avec des organismes gouvernementaux que non gouvernementaux, ainsi qu'avec des entreprises de toutes dimensions, qu'il s'agisse de petites et moyennes entreprises (PME), de grandes sociétés nationales ou d'entités étrangères.

Cette ouverture témoigne de sa capacité d'adaptation et de sa volonté de répondre à des besoins variés dans différents secteurs d'activités.

Grâce à une équipe constituée d'ingénieurs qualifiés et expérimentés, ITDC MADA est toujours prête à assurer la mise en œuvre de projets technologiques et numériques, en apportant des solutions concrètes, innovantes et adaptées aux exigences de ses partenaires.

Par ailleurs, l'entreprise ne se limite pas uniquement à la prestation de services techniques. Elle propose également des formations enrichissantes destinées au personnel administratif des entreprises ou des institutions. Ces sessions de formation ont pour objectif de renforcer les compétences des participants dans l'utilisation des outils informatiques et bureautiques, contribuant ainsi à une amélioration significative de leur productivité et de leur autonomie dans les tâches quotidiennes.

2.6 Méthode du travail

Chez ITDC MADA, la gestion des projets repose sur la méthode agile, qui permet d'assurer le bon déroulement de chaque mission en tenant compte du budget, des délais et surtout des besoins des clients. Cette approche s'inscrit dans les méthodes agiles, qui privilégient une progression itérative et l'adaptation aux besoins constatés [Sommerville, 2021].

Cette approche vise à garantir la qualité, à maîtriser les coûts et à respecter les délais (QCD) tout en s'adaptant en permanence aux attentes des utilisateurs.

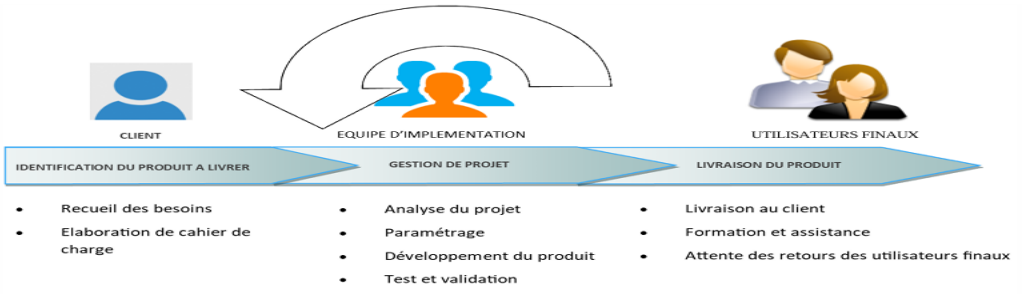


Figure 2.1 : Méthode de travail

2.7. Organigramme d'ITDC MADA

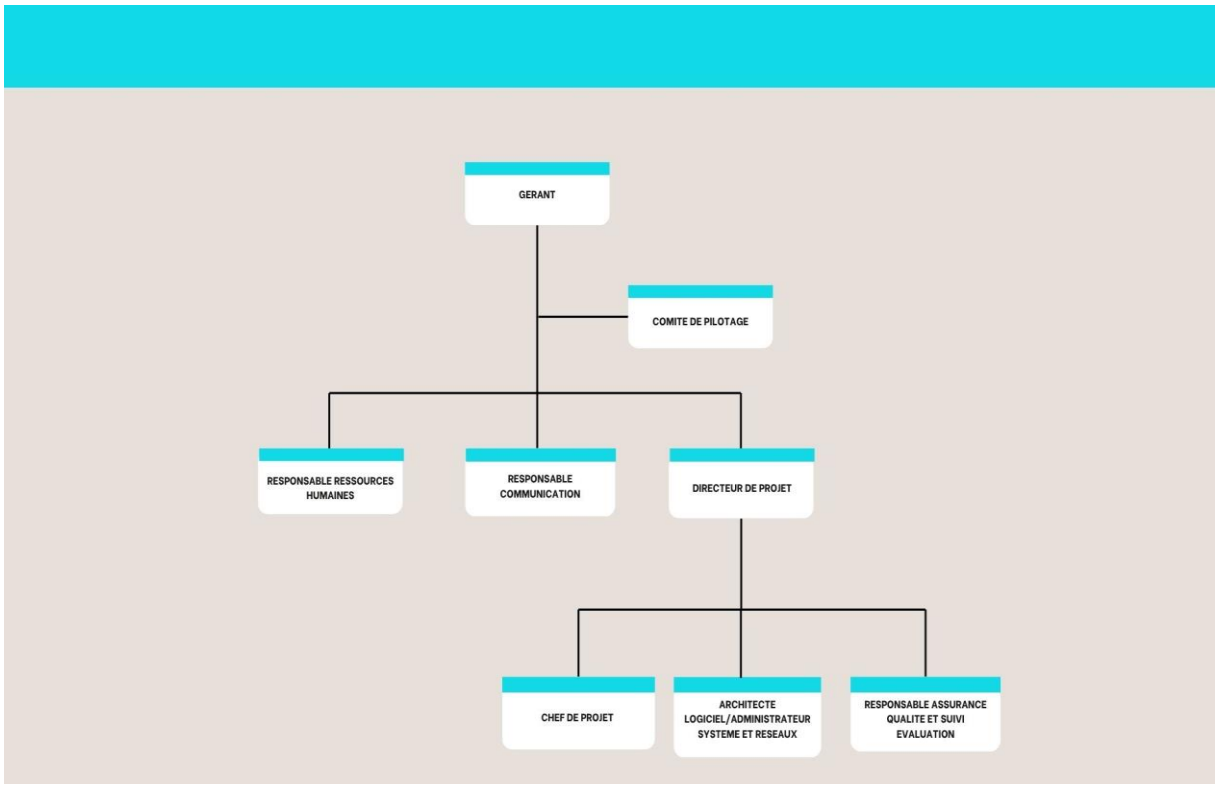


Figure 2.2 : Organigramme ITDC MADA

CHAPITRE III : PRÉSENTATION DU PROJET

3.1. Formulation du projet

3.1.1. Intitulé du projet

Le projet s'intitule « Plateforme de gestion et de suivi des projets et partenariats de l'Université de Fianarantsoa ».

Il s'agit de concevoir une application web qui centralise et organise les informations relatives aux partenaires, aux projets, aux partenariats, aux cotutelles, aux conventions et aux documents associés. La plateforme prévoit des espaces d'accès distincts selon le profil des utilisateurs et les responsabilités des entités concernées.

3.1.2. Contexte du projet

Le suivi des projets et des partenariats universitaires implique de gérer des informations de natures très différentes : partenaires, établissements, projets, partenariats, cotutelles, conventions, documents, ainsi que des contenus destinés à la consultation publique.

La plateforme répond à ce contexte en offrant un espace commun de gestion et de consultation.

3.1.3. Problématique du projet

Pour que le suivi fonctionne, les informations doivent pouvoir être organisées, retrouvées et consultées par les personnes autorisées. La diversité des données et des profils pose donc un double enjeu : centraliser, et répartir les accès.

La problématique du projet est formulée comme suit :

Comment concevoir une plateforme web permettant de centraliser les informations relatives aux projets et aux partenariats de l'Université de Fianarantsoa, tout en organisant leur gestion selon les responsabilités des utilisateurs et en facilitant la consultation des données ?

3.1.4. Présentation de la solution envisagée

La solution retenue est une plateforme web qui réunit les informations sur les partenaires, les projets et les partenariats. Elle comprend un espace pour la Présidence, un pour les établissements, un pour le Décideur, ainsi qu'un espace public accessible sans compte.

Les données consultables et les opérations possibles dépendent du profil de l'utilisateur et du domaine concerné. Le détail des fonctionnalités est présenté dans les sections suivantes.

3.2. Justification de l'importance du sujet

3.2.1. Importance administrative et institutionnelle

Suivre des projets et des partenariats, c'est conserver et consulter des informations qui touchent plusieurs entités et plusieurs domaines. Les réunir dans une organisation commune facilite leur recherche et leur consultation par les utilisateurs autorisés.

La centralisation est donc avant tout un enjeu de gestion de l'information. Elle doit toutefois respecter les responsabilités de la Présidence, des établissements et des autres profils, et maintenir la distinction entre données internes et informations rendues publiques.

3.2.2. Situation antérieure

Les pratiques de suivi antérieures au projet, les outils employés et les difficultés rencontrées sont [À COMPLÉTER : présenter les constats issus d'entretiens, de documents internes ou d'autres informations recueillies pendant le stage].

3.2.3. Pertinence de la solution proposée

La plateforme répond à un besoin de structuration : elle réunit plusieurs catégories de données dans un même système, prévoit des accès différenciés selon les profils et permet de consulter les informations autorisées.

Les choix techniques et de conception sont exposés dans les chapitres suivants.

3.3. Objectifs et besoins des utilisateurs

3.3.1. Objectif général

L'objectif général du projet est de concevoir une plateforme web qui centralise et gère les informations relatives aux projets et aux partenariats de l'Université de Fianarantsoa, selon les responsabilités et les droits d'accès de chaque utilisateur.

3.3.2. Objectifs spécifiques

Le projet vise à :

1. organiser les informations relatives aux partenaires ;
2. structurer les données relatives aux projets et aux partenariats ;
3. gérer les informations concernant les cotutelles ;
4. faciliter la consultation des informations par les utilisateurs autorisés ;
5. permettre la gestion des données d'établissement dans les limites du périmètre autorisé ;

6. permettre à la Présidence de gérer les données relevant de ses droits ;
7. permettre à la Présidence de publier les données soumises qui sont destinées à être rendues publiques ;
8. organiser l'accès aux conventions et aux documents selon les responsabilités prévues ;
9. faciliter la consultation des notifications destinées aux utilisateurs ;
10. proposer un espace public donnant accès aux informations rendues publiques ;
11. permettre la transmission de messages au moyen du formulaire public de contact.

3.3.3. Acteurs de la plateforme

La plateforme distingue trois profils authentifiés (Présidence, Établissement et Décideur) et un accès public sans compte, le Visiteur. Ce dernier consulte l'espace public et n'est pas un profil authentifié.

Les besoins associés à ces profils sont présentés dans le tableau 3.1.

Tableau 3.1 : Acteurs et besoins fonctionnels de la plateforme

Acteur	Besoin principal	Fonctionnalités associées	Limites d'accès connues
Présidence	Gérer les données relevant de ses droits et publier les données soumises destinées à la publication	Gestion des informations autorisées, publication et consultation des notifications destinées à la Présidence	Les droits varient selon le domaine fonctionnel
Établissement	Gérer les données relevant de son périmètre et effectuer les opérations qui lui sont autorisées	Consultation et gestion des informations accessibles pour son établissement	Pas d'accès aux conventions, aux documents, aux publications ni aux synthèses
Décideur	Consulter les informations et indicateurs autorisés	Consultation en lecture seule des données métier accessibles	Pas de modification des données métier
Visiteur	Consulter les informations rendues publiques et contacter l'Université	Espace public et formulaire de contact	Accès sans compte, limité aux informations publiques

Source : informations communiquées pour le projet.

Ce tableau résume les besoins généraux de chaque profil et les limites d'accès connues. Les fonctionnalités détaillées sont à rapprocher des règles validées pour chaque domaine.

a) La Présidence

La Présidence gère les données qui relèvent de ses droits et peut intervenir dans la publication des données soumises destinées à être rendues publiques ; cette publication ne forme pas une étape de validation distincte.

Elle consulte aussi les notifications qui lui sont destinées et en gère l'état de lecture. Ses autres droits de gestion varient selon le domaine fonctionnel.

b) L'Établissement

L'Établissement intervient sur les données et les opérations de son périmètre qui lui sont ouvertes ; ses droits varient selon le domaine.

Il n'a pas accès aux conventions, aux documents, aux publications ni aux synthèses.

c) Le Décideur

Le Décideur consulte les informations et les indicateurs qui lui sont accessibles, en lecture seule pour les données métier.

d) Le Visiteur

Le Visiteur accède à l'espace public sans authentification. Il peut consulter les informations rendues publiques et envoyer un message par le formulaire public de contact.

3.3.4. Besoins fonctionnels et exigences non fonctionnelles

Les besoins fonctionnels décrivent les opérations attendues de la plateforme ; les exigences non fonctionnelles, les qualités recherchées dans son fonctionnement et son organisation. Ces dernières sont des objectifs et non des performances mesurées.

Les principaux besoins et exigences sont présentés dans le tableau 3.2.

Tableau 3.2 : Synthèse des besoins fonctionnels et des exigences retenues

Catégorie	Besoin ou exigence	Description
Fonctionnel	Gestion des partenaires	Organiser les informations relatives aux partenaires selon les droits disponibles
Fonctionnel	Gestion des projets	Gérer les informations relatives aux projets dans les limites des fonctionnalités prévues
Fonctionnel	Gestion des partenariats	Organiser les informations relatives aux partenariats et à leurs domaines
Fonctionnel	Gestion des cotutelles	Gérer les informations relatives aux cotutelles

Fonctionnel	Gestion des conventions	Gérer les informations relatives aux conventions selon les droits applicables
Fonctionnel	Gestion des documents	Organiser l'accès aux documents selon les responsabilités prévues
Fonctionnel	Soumission et publication	Permettre la soumission des données et la publication par la Présidence des données destinées à être rendues publiques
Fonctionnel	Notifications	Permettre la consultation des notifications destinées aux utilisateurs
Fonctionnel	Espace public et contact	Consulter les informations publiques et transmettre un message
Fonctionnel	Exports et historiques détaillés	Exports projets/conventions en PDF ou XLSX (Présidence et Décideur) ; module avancé PDF, XLSX ou CSV en ZIP, aperçu et historique paginé (Présidence). Les versions de documents disposent d'une route de consultation réservée à la Présidence et au Décideur.
Non fonctionnel	Sécurité	Organiser les accès en fonction des profils et des autorisations
Non fonctionnel	Cohérence des données	Maintenir une organisation cohérente des informations gérées
Non fonctionnel	Maintenabilité	Favoriser une organisation compréhensible et maintenable
Non fonctionnel	Ergonomie	Faciliter l'utilisation des fonctions accessibles à chaque profil

Source : informations communiquées pour le projet

Ce tableau récapitule le périmètre fonctionnel et les qualités retenues pour la conception. Les exigences non fonctionnelles y sont formulées comme des objectifs ; aucune mesure de performance n'est avancée à ce stade.

3.4. Moyens nécessaires à la réalisation du projet

3.4.1. Moyens humains

Deux développeurs ont assuré la réalisation technique. J'ai surtout contribué au développement du backend, tandis que l'autre développeur a travaillé principalement sur le frontend.

L'équipe Projet et Partenariat de la Présidence a été le commanditaire et l'interlocuteur fonctionnel du projet.

Les moyens humains sont présentés dans le tableau 3.3.

Tableau 3.3 : Moyens humains mobilisés pour la réalisation du projet

Désignation	**Rôle ou contribution**
Étudiant stagiaire	Contribution principale au développement du backend
Autre développeur	Contribution principale au développement du frontend
Équipe Projet et Partenariat de la Présidence	Commanditaire et interlocuteur fonctionnel
Encadreur professionnel	Encadrement du stage
Encadreur académique	Suivi académique du stage

Source : informations communiquées pour le projet.

Ce tableau distingue l'équipe technique de l'encadrement du stage. Les tâches détaillées restent à compléter à partir de pièces qui attestent leur répartition.

3.4.2. Moyens matériels

Le développement a nécessité un poste informatique, une connexion Internet et les outils permettant d'exécuter les services de la plateforme. Le système d'exploitation utilisé était Windows 11.

Le frontend, le backend et PostgreSQL ont tourné dans des conteneurs.

Tableau 3.4 : Moyens matériels mobilisés

Désignation	**Rôle ou usage**	**Caractéristiques**
Poste informatique	Développement et vérification de la plateforme	Windows 11, environnement 64 bits ; configuration de référence indicative : processeur multicœur, 16 Go de mémoire vive et stockage SSD de 512 Go
Connexion Internet	Accès aux dépôts de code, aux dépendances et aux ressources nécessaires au développement	Connexion à haut débit permettant les échanges avec les dépôts Git et les registres de paquets

Docker et Docker Compose	Exécution des services conteneurisés	Versions : 29.7.2
Services conteneurisés	Exécution de la plateforme	Frontend, backend et PostgreSQL

Source : informations communiquées pour le projet et inventaire des moyens mobilisés pendant le stage.

Ce tableau présente les moyens matériels connus, sans prêter au poste de développement des caractéristiques qui ne sont pas attestées. Les versions logicielles et les conditions d'accès à Internet restent à compléter.

3.4.3. Moyens logiciels

La plateforme repose sur plusieurs technologies pour son backend, son frontend et sa base de données. Les versions indiquées ci-dessous proviennent des informations communiquées pour le projet.

Tableau 3.5 : Principaux moyens logiciels utilisés

Logiciel ou technologie	**Rôle dans le projet**	**Version ou référence communiquée**
Windows	Système d'exploitation	Windows 11
Node.js	Environnement du backend	Image node:20-alpine
Node.js	Environnement du frontend	Image node:22-alpine
NestJS	Développement du backend	^11.0.0 (déclarée)
TypeScript	Langage du backend et du frontend	Backend : ^5.7.0 ; frontend : ~6.0.2
TypeORM	Persistance des données	^0.3.31 (déclarée)
PostgreSQL	Système de gestion de base de données	16 (image postgres:16-alpine)
React	Développement du frontend	^19.2.8 (déclarée)
Vite	Développement et construction du frontend	^8.2.2 (déclarée)
Tailwind CSS	Mise en forme du frontend	^4.3.3 (déclarée)

React Hook Form et Zod	Gestion et validation des formulaires	React Hook Form ^7.87.0 ; Zod ^4.5.4
class-validator	Validation des données du backend	^0.14.1
JWT et Swagger	Authentification et documentation de l'API	@nestjs/jwt 11.0.2 et jsonwebtoken 9.0.3 ; @nestjs/swagger 11.4.7 et swagger-ui-express 5.0.1. La documentation est exposée à l'adresse /api/docs.
Docker et Docker Compose	Conteneurisation et orchestration des services	Docker Compose v2 ; images de développement basées sur Node.js 20 Alpine pour le backend, Node.js 22 Alpine pour le frontend et PostgreSQL 16 Alpine pour la base. La version de Docker Desktop ou du moteur Docker dépend du poste et n'est pas épinglée dans le dépôt.
Visual Studio Code	Édition et développement	Éditeur utilisé pour le développement
Git et GitHub	Versionnement et collaboration	Git sert au suivi des modifications et à la gestion des branches ; GitHub héberge le dépôt distant et permet le partage et la revue des changements.

Source : fichiers de configuration et manifestes de dépendances du projet, version examinée le 6 novembre 2026.

Ce tableau regroupe les principaux outils annoncés pour le développement de la plateforme. Les versions non confirmées sont signalées comme telles.

3.5. Chronogramme de réalisation du projet

Le stage s'est déroulé du 17 août au 31 octobre 2026, soit 76 jours calendaires en comptant le premier et le dernier jour. Le plan initial prévoyait cinq sprints sur soixante-dix jours au total.

Le tableau 3.6 distingue la planification initiale des éléments restant à préciser.

Tableau 3.6 : Planification initiale du projet

Période de travail	Durée ou dates	Tâches réalisées	Livrable
Sprint 1 — Cadrage et conception	14 jours — 17 au 30 août 2026	Cadrage fonctionnel, identification des acteurs et des rôles, analyse des besoins, premiers cas d'utilisation et préparation de l'environnement de développement.	Dossier de cadrage, matrice des rôles, cas d'utilisation initiaux et environnement de développement préparé.
Sprint 2 — Modélisation et socle technique	14 jours — 31 août au 13 septembre 2026	Modélisation des données, dictionnaire, règles de gestion, migrations de base de données, authentification JWT et contrôle des rôles.	Modèle de données, dictionnaire, migrations et socle d'authentification et d'autorisation.
Sprint 3 — Projets et partenariats	14 jours — 14 au 27 septembre 2026	Développement des modules liés aux établissements, partenaires, projets et partenariats ; soumission et publication des projets ; premiers tableaux de bord et accès public.	Modules métier principaux et parcours différenciés selon les rôles.
Sprint 4 — Conventions et documents	14 jours — 28 septembre au 11 octobre 2026	Gestion des conventions et des documents, statuts et archivage, téléversement, notifications et fonctions d'export.	Parcours de gestion des conventions et documents, notifications prévues et exports PDF/XLSX.
Sprint 5 — Intégration et vérifications	14 jours — 12 au 25 octobre 2026	Intégration des modules, corrections, vérification des parcours, préparation des captures et rédaction des éléments de présentation.	Version intégrée de la plateforme, liste de vérifications, captures disponibles et éléments du rapport.

Source : plan initial communiqué

Ce tableau présente une planification prévisionnelle du projet. Les étapes couvrent l'analyse des besoins, la conception, le développement, les vérifications fonctionnelles et la rédaction du mémoire.

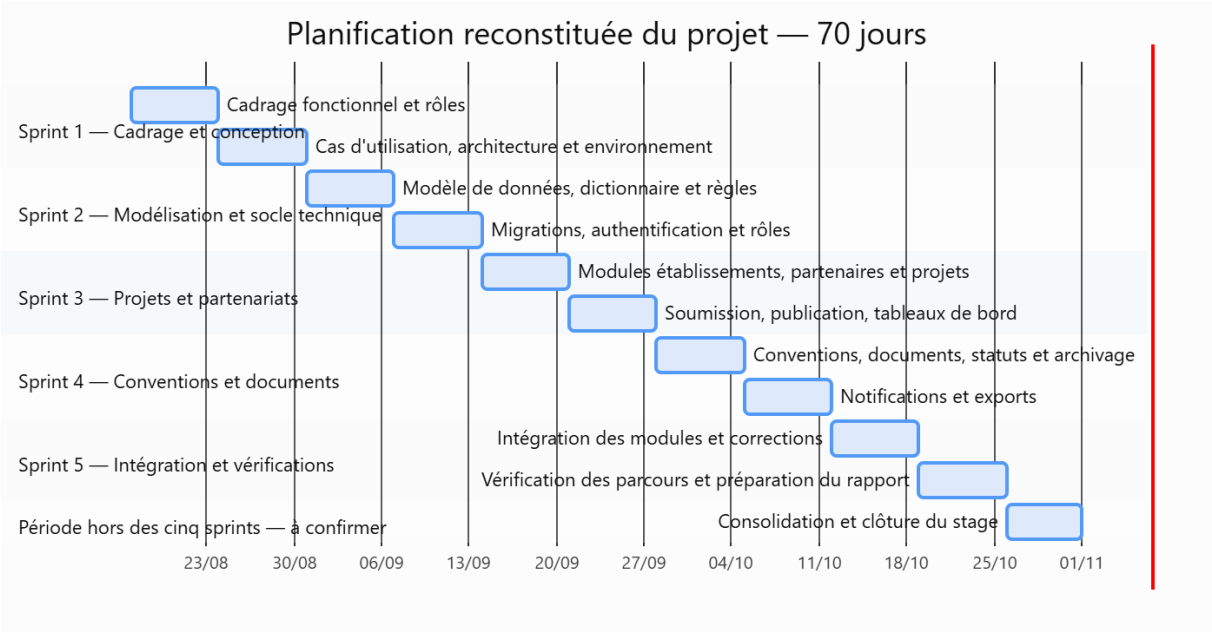


Figure 3.1 : Diagramme de Gantt de réalisation du projet

Source : réalisation personnelle avec Draw.io, à partir du calendrier validé.

Le diagramme de Gantt présentera les phases, leur durée et l'enchaînement des tâches, à partir du calendrier vérifié.

3.6. Résultat attendu

Le projet vise la réalisation d'une plateforme web centralisée pour organiser le suivi des partenaires, des projets, des partenariats, des cotutelles, des conventions et des documents associés. Le regroupement de ces informations doit faciliter leur recherche et leur consultation, tout en limitant leur dispersion entre plusieurs supports.

La plateforme distingue les profils Présidence, Établissement, Décideur et Visiteur. Les utilisateurs authentifiés accèdent aux fonctions correspondant à leur rôle et à leur périmètre. Le Visiteur consulte uniquement les informations rendues publiques. Les projets soumis par les établissements peuvent être examinés par la Présidence, qui peut les publier ou demander leur modification.

Le suivi des conventions porte sur leurs statuts, leurs échéances et les documents associés. Le système prévoit également l'archivage et des notifications liées aux événements et aux dates d'expiration. Les tableaux de bord doivent présenter des informations adaptées aux droits de chaque profil. Les fonctions d'export et de consultation des versions de documents complètent les outils de gestion prévus par le projet.

Le résultat attendu est ainsi une application qui facilite la centralisation, le suivi et le partage contrôlé des informations. Ces objectifs décrivent le périmètre visé ; ils ne constituent pas, à eux seuls, une preuve du fonctionnement de chaque fonctionnalité. Les fonctionnalités présentes et les vérifications effectivement documentées sont examinées dans la partie consacrée à la mise en œuvre et à l'évaluation.

PARTIE II : ANALYSE ET CONCEPTION

CHAPITRE IV : ANALYSE PRÉALABLE

Ce chapitre expose l'analyse de l'existant et les choix retenus pour la conception et la réalisation de la plateforme. La description de l'organisation antérieure est une reconstitution cohérente avec les informations disponibles sur le projet.

4.1. Analyse de l'existant

4.1.1. Organisation actuelle

Avant la mise en place de la plateforme, la gestion des projets et des partenariats pouvait s'organiser autour d'échanges entre les établissements et l'équipe Projet et Partenariat de la Présidence. L'établissement préparait les informations relatives au projet, aux partenaires et au partenariat, puis transmettait le dossier et les pièces associées à la Présidence au moyen de supports administratifs et bureautiques.

À réception, la Présidence vérifiait les informations et demandait des compléments lorsque le dossier était incomplet. Les données de suivi étaient ensuite mises à jour dans les supports disponibles, tandis que les conventions et les documents associés pouvaient être classés séparément. Le statut des dossiers et les échéances nécessitaient ainsi des consultations et des mises à jour régulières.

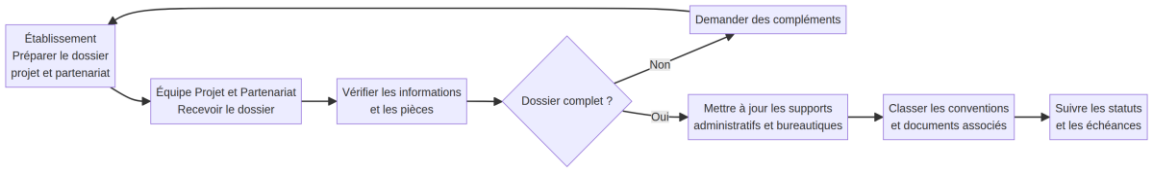


Figure 4.1 : Diagramme de flux de l'organisation actuelle

Source : reconstitution fondée sur la description de l'organisation antérieure présentée dans le rapport.

4.1.2. Critique de l'existant

Dans l'organisation antérieure reconstituée, les informations relatives aux établissements, aux partenaires, aux projets et aux conventions circulaient entre les services au moyen de supports distincts, tels que des fichiers bureautiques et des documents transmis séparément. L'équipe de la Présidence devait rassembler les informations, vérifier les dossiers reçus et mettre à jour leur état de suivi. Les pièces justificatives et les documents associés pouvaient être conservés séparément des données principales.

Cette organisation dispersée compliquait la recherche et le rapprochement des informations concernant un même projet. Les mises à jour effectuées sur plusieurs supports pouvaient également entraîner des doublons ou des écarts entre les données. Le suivi des statuts et des

échéances nécessitait des vérifications régulières, tandis que la consultation dépendait des supports disponibles et des personnes qui les géraient.

La plateforme répond à ces limites en regroupant les principales données dans une application et en différenciant les accès selon les profils. Cette critique constitue une reconstitution cohérente avec le périmètre du projet ; les difficultés précises et leur fréquence restent à valider avec les personnes concernées.

4.1.3. Moyens existants pour la réalisation du projet

Deux développeurs ont mené la réalisation du projet. J'ai surtout contribué au backend, l'autre développeur ayant travaillé principalement sur le frontend.

L'environnement de travail comprend Windows 11, Visual Studio Code, Git, GitHub, Docker et Docker Compose. Les services frontend, backend et PostgreSQL sont conteneurisés.

Les moyens connus sont présentés dans le tableau 4.1.

Tableau 4.1 : Moyens humains et matériels mobilisés

Catégorie	Moyen	Rôle ou contribution	Précisions
Humain	Étudiant stagiaire	Contribution principale au backend	Développement des services et routes de l'API, de la logique métier et de l'accès aux données avec TypeORM et PostgreSQL ; participation aux migrations, à l'authentification et au contrôle des rôles. Répartition indicative à valider avec le journal de travail.
Humain	Autre développeur	Contribution principale au frontend	Développement de l'interface avec React et Vite, des écrans de gestion et des tableaux de bord, puis intégration avec l'API. Répartition indicative à valider avec l'équipe.
Humain	Équipe Projet et Partenariat de la Présidence	Commanditaire et interlocuteur fonctionnel	Expression des besoins, clarification des profils et des parcours métier, retours sur les fonctions attendues. La fréquence des échanges et le détail des validations ne sont pas consignés.

Catégorie	Moyen	Rôle ou contribution	Précisions
Matériel et environnement	Poste informatique	Développement et vérification	Windows 11
Matériel et environnement	Visual Studio Code	Édition et développement	Éditeur utilisé pour le travail sur le code
Matériel et environnement	Git et GitHub	Versionnement et collaboration	Git pour le suivi des modifications ; GitHub pour l'hébergement du dépôt et le partage du code.
Matériel et environnement	Docker et Docker Compose	Exécution des services conteneurisés	Configuration du projet : PostgreSQL 16-alpine, image backend node:20-alpine, image frontend node:22-alpine et image de production frontend nginx:1.27-alpine.

Source : informations communiquées pour le projet et répartition des responsabilités retenue pendant le stage.

Ce tableau sépare les ressources humaines des outils de travail. Les tâches précises des développeurs et les caractéristiques du poste restent à compléter à partir d'informations attestées.

4.2. Choix de la solution

La solution retenue est une plateforme web de gestion et de suivi des projets et des partenariats. Elle répond au besoin de réunir les informations concernées et de différencier les accès selon les profils et les responsabilités.

La plateforme comporte des espaces pour la Présidence, les établissements et le Décideur, ainsi qu'un espace public sans compte. Les droits varient selon le profil et le domaine fonctionnel : l'Établissement n'a pas accès aux conventions, aux documents, aux publications ni aux synthèses, et le Décideur consulte en lecture seule les données métier qui lui sont ouvertes.

4.3. Choix de la méthode de conception et de gestion du projet

La gestion du projet suit une démarche itérative inspirée de Scrum. Le plan initial prévoyait cinq sprints sur soixante-dix jours.

Cette démarche permettait de découper le travail en étapes et de suivre l'avancement des fonctionnalités.

4.4. Choix de l'outil de conception

Draw.io a servi à produire les diagrammes du projet. Cet outil permet de représenter graphiquement les éléments de conception nécessaires à la description de la plateforme.

Les diagrammes comprennent ceux prévus dans les chapitres de conception : cas d'utilisation, séquences, modèle de domaine et architecture.

4.5. Choix du langage de programmation

Le choix du langage a été orienté vers quatre options : JavaScript, TypeScript, PHP et Python. JavaScript s'intègre directement à l'écosystème du navigateur et de Node.js, mais son typage dynamique offre moins de vérifications statiques. PHP et Python permettent de développer un backend, mais auraient introduit un langage et un environnement supplémentaires par rapport au frontend retenu.

TypeScript a donc été privilégié. Il conserve la compatibilité avec l'écosystème JavaScript tout en ajoutant un système de types qui facilite la structuration des modules et la détection précoce de certaines incohérences pendant le développement. Il est utilisé dans les deux parties de la plateforme : NestJS pour le backend et React avec Vite pour le frontend. Ce choix maintient une continuité technique entre l'interface et l'API et facilite l'évolution du code.

4.6. Choix des frameworks et des technologies d'interface

Le backend est développé avec NestJS 11 et TypeScript, le frontend avec React 19, TypeScript et Vite. React est une bibliothèque d'interface et Vite l'outil de développement et de construction du frontend ; ils n'ont donc pas le même rôle que NestJS [NestJS, s.d. ; React, s.d. ; Vite, s.d.].

Le tableau 4.2 présente les technologies principales et leur fonction dans le projet.

Tableau 4.2 : Technologies principales du backend et du frontend

Technologie	Rôle dans le projet	Version communiquée
NestJS	Développement du backend	^11.0.0 (déclarée)
TypeScript	Langage du backend et du frontend	Backend : ^5.7.0 ; frontend : ~6.0.2
React	Construction de l'interface utilisateur	^19.2.8 (déclarée)

Technologie	Rôle dans le projet	Version communiquée
Vite	Développement et construction du frontend	^8.2.2 (déclarée)
Tailwind CSS	Mise en forme du frontend	^4.3.3 (déclarée)
React Hook Form et Zod	Gestion et validation des formulaires	React Hook Form ^7.87.0 ; Zod ^4.5.4

Source : fichiers de configuration et manifestes de dépendances du projet, version examinée le 6 novembre 2026.

Ce tableau sépare les fonctions du serveur de celles de l'interface. Les versions non confirmées restent signalées comme telles.

NestJS organise le backend autour d'éléments comme les contrôleurs et les services. React construit le frontend à partir de composants, et Vite lance l'environnement de développement et prépare la construction de l'interface [NestJS, s.d. ; React, s.d. ; Vite, s.d.].

4.7. Choix de l'environnement de développement

L'environnement de travail est organisé autour de Windows 11. Visual Studio Code sert à éditer le code TypeScript, consulter les fichiers du projet et lancer les commandes de développement. Son choix se justifie par la prise en charge de TypeScript, les outils de débogage, le terminal intégré et l'intégration de Git. Pour ce projet, ces fonctions en font une solution adaptée aux besoins du développement frontend et backend. WebStorm et Sublime Text constituent des alternatives possibles ; Visual Studio Code offre ici un environnement intégré couvrant à la fois l'édition, le débogage et le versionnement.

Git assure le suivi des modifications et GitHub héberge le dépôt du projet. Docker Compose décrit et lance les services du frontend, du backend et de la base de données dans des conteneurs. Cette organisation sépare les services et facilite la mise en place de l'environnement de développement.

Les fichiers du projet indiquent les images et versions de configuration suivantes : Node.js 20-alpine pour le backend, Node.js 22-alpine pour le frontend, PostgreSQL 16-alpine pour la base de données et Nginx 1.27-alpine pour le frontend en production. TypeScript est déclaré en version ^5.7.0 côté backend et ~6.0.2 côté frontend ; Vite est déclaré en version ^8.2.2. Ces valeurs correspondent aux déclarations et aux images du dépôt. Les versions installées localement de Visual Studio Code, Git, Docker Engine et Docker Compose ne sont pas consignées dans les fichiers du projet.

4.8. Choix du système de gestion de base de données

Le système retenu pour la plateforme est PostgreSQL 16. Le choix a été orienté par la structure relationnelle des données : projets, établissements, partenaires, partenariats, conventions et documents sont liés par des relations qui nécessitent des contraintes d'intégrité et des transactions cohérentes. PostgreSQL prend également en charge les UUID et le type JSONB utilisés dans le modèle du projet [PostgreSQL, s.d.].

La comparaison de conception a porté sur PostgreSQL, MySQL et SQLite. MySQL constitue une solution relationnelle adaptée aux applications web, mais PostgreSQL correspond mieux aux types de données et aux fonctionnalités retenus pour cette plateforme. SQLite, léger et simple à configurer, convient davantage à des usages locaux ou de faible ampleur ; il a été écarté pour la base centrale d'une application destinée à plusieurs profils et à des données liées.

TypeORM 0.3.31 assure le mapping entre les objets du backend et les données stockées dans PostgreSQL. La structure de la base évolue au moyen de migrations, ce qui permet de versionner les changements de schéma avec le code de l'application [TypeORM, s.d.].

CHAPITRE V : CONCEPTION DU PROJET

Ce chapitre présente la méthode de conception, les données, les règles de gestion et les principaux modèles de la plateforme. Les diagrammes sont réalisés à l'aide de Draw.io.

5.1. Présentation de la méthode de conception

5.1.1. Méthode utilisée

La conception s'appuie sur le langage de modélisation unifié (Unified Modeling Language, UML), qui permet de représenter les acteurs, les fonctionnalités, les échanges entre un utilisateur et le système, ainsi que les relations entre les principales entités de l'application [OMG, 2017 ; Fowler, 2003].

La modélisation se limite aux éléments nécessaires pour comprendre la plateforme : les cas d'utilisation et leurs descriptions textuelles, les séquences système et le modèle de domaine

5.1.2. Outil de conception

Les diagrammes sont réalisés avec Draw.io, conformément au choix retenu pour le projet. Cet outil permet de créer et de mettre en forme les représentations nécessaires à la conception de la plateforme.

Dans les figures, Présidence, Établissement, Décideur et Visiteur désignent des acteurs ou des profils fonctionnels. Ils doivent être distingués des entités de données du modèle, telles que Utilisateur, Etablissement, Projet, Partenaire, Partenariat, Convention, CotutelleThese, Document et Publication.

Les principales associations relient les établissements à leurs utilisateurs et à leurs projets ; les partenariats aux établissements, aux partenaires, aux projets et aux conventions ; et les projets à leur établissement et, le cas échéant, à un partenariat. Une convention peut être rattachée à un projet ou à un partenariat. Un document est associé à un seul élément parmi un projet, une convention, une cotutelle de thèse ou un partenariat. Une publication est associée à un projet.

5.2. Dictionnaire des données

5.2.1. Organisation des données

Les données de la plateforme sont organisées autour des utilisateurs, des établissements, des partenaires, des partenariats, des projets, des conventions et des documents. Le modèle comprend également les cotutelles de thèse, les publications et leurs photos, ainsi que des données de référence et de suivi portant notamment sur les activités, les sites, les bailleurs, les

bénéficiaires, les notifications et les historiques. Des entités d'association relient ces différentes données.

Le registre TypeORM du backend recense **33 entités**, incluant les entités métier, les entités d'association et les entités de support. Dans les déclarations TypeORM actuelles, l'identifiant principal est de type uuid pour 31 de ces entités ; il est de type text pour PasswordResetToken et ExportHistorique

5.2.2. Dictionnaire des données

Le tableau 5.1 regroupe les rubriques principales des entités du modèle. Les longueurs, types, états et contraintes doivent être rapprochés du modèle de données définitif.

Tableau 5.1 : Dictionnaire des données de la plateforme

Entité	Rubrique	Définition	Longueur	Type	État	Remarque
Utilisateur	id	Identifiant de l'utilisateur	–	UUID	Obligatoire	Clé primaire
Utilisateur	email	Adresse électronique de connexion	–	VARCHAR	Obligatoire	Unique
Utilisateur	password_hash	Empreinte du mot de passe	–	VARCHAR	Obligatoire	Ne contient pas le mot de passe en clair
Utilisateur	role	Rôle d'accès	–	ENUM Role	Obligatoire	PRESIDENCE, ETABLISSEMENT ou DECIDEUR
Utilisateur	etablissement_id	Établissement de rattachement	–	UUID	Facultatif	Clé étrangère vers Etablissement
Utilisateur	actif	État actif du compte	–	BOOLEAN	Obligatoire	Valeur par défaut : true
Utilisateur	logo_url	Adresse du logo de l'utilisateur	–	VARCHAR	Facultatif	–
Utilisateur	derniere_connexion	Date de la dernière connexion	–	TIMESTAMP	Facultatif	–

Utilisateur	preferences	Préférences de l'utilisateur	–	JSONB	Obligatoire	Valeur par défaut : objet JSON vide
Utilisateur	created_at	Date de création	–	TIMESTAMP	Obligatoire	Gérée automatiquement
Utilisateur	updated_at	Date de modification	–	TIMESTAMP	Obligatoire	Gérée automatiquement
Etablissement	id	Identifiant de l'établissement	–	UUID	Obligatoire	Clé primaire
Etablissement	nom	Nom de l'établissement	–	VARCHAR	Obligatoire	Indexé
Etablissement	sigle	Sigle de l'établissement	–	VARCHAR	Facultatif	–
Etablissement	logo_url	Adresse du logo	–	VARCHAR	Facultatif	–
Etablissement	description	Présentation de l'établissement	–	TEXT	Facultatif	–
Etablissement	created_at	Date de création	–	TIMESTAMP	Obligatoire	Gérée automatiquement
Etablissement	updated_at	Date de modification	–	TIMESTAMP	Obligatoire	Gérée automatiquement
Partenaire	id	Identifiant du partenaire	–	UUID	Obligatoire	Clé primaire
Partenaire	nom	Nom du partenaire	–	VARCHAR	Obligatoire	Indexé
Partenaire	type	Type de partenaire	–	VARCHAR	Obligatoire	–
Partenaire	pays	Pays du partenaire	–	VARCHAR	Obligatoire	–
Partenaire	adresse	Adresse du partenaire	–	VARCHAR	Facultatif	–
Partenaire	telephone	Numéro de téléphone	–	VARCHAR	Facultatif	–
Partenaire	email	Adresse électronique	–	VARCHAR	Facultatif	–

Partenaire	site_web	Site web du partenaire	–	VARCHAR	Facultatif	–
Partenaire	description	Présentation du partenaire	–	TEXT	Facultatif	–
Partenaire	statut	État du partenaire	–	ENUM StatutPartenaire	Obligatoire	ACTIF ou SUSPENDU
Partenaire	visibilite	Visibilité de la donnée	–	ENUM Visibilite	Obligatoire	PUBLIC ou PRIVE
Partenaire	est_strategique	Indique si le partenaire est stratégique	–	BOOLEAN	Obligatoire	Valeur par défaut : false
Partenaire	soumis_le	Date de soumission	–	TIMESTAMP	Facultatif	–
Partenaire	publie_le	Date de publication	–	TIMESTAMP	Facultatif	–
Partenaire	archive	Indique si le partenaire est archivé	–	BOOLEAN	Obligatoire	Valeur par défaut : false
Partenaire	etablissement_id	Établissement de rattachement	–	UUID	Facultatif	Clé étrangère vers Etablissement
Partenaire	created_at	Date de création	–	TIMESTAMP	Obligatoire	Gérée automatiquement
Partenaire	updated_at	Date de modification	–	TIMESTAMP	Obligatoire	Gérée automatiquement
Projet	id	Identifiant du projet	–	UUID	Obligatoire	Clé primaire
Projet	nom	Nom du projet	–	VARCHAR	Obligatoire	–
Projet	intitule_officiel	Intitulé officiel	–	VARCHAR	Facultatif	–
Projet	type	Type du projet	–	VARCHAR	Obligatoire	–
Projet	domaine	Domaine du projet	–	VARCHAR	Facultatif	–
Projet	budget	Budget du projet	12,2	DECIMAL	Facultatif	Précision et échelle à vérifier

Projet	date_debut	Date de début	–	TIMESTAMP	Obligatoire	–
Projet	date_fin_prevue	Date de fin prévue	–	TIMESTAMP	Obligatoire	–
Projet	statut	État du projet	–	ENUM StatutProjet	Obligatoire	–
Projet	archive	Indique si le projet est archivé	–	BOOLEAN	Obligatoire	Valeur par défaut : false
Projet	soumis_le	Date de soumission	–	TIMESTAMP	Facultatif	–
Projet	publie_le	Date de publication	–	TIMESTAMP	Facultatif	–
Projet	etablissement_id	Établissement responsable	–	UUID	Facultatif	Clé étrangère vers Etablissement
Projet	partenariat_id	Partenariat associé	–	UUID	Facultatif	Clé étrangère vers Partenariat
Projet	porteur_id	Porteur associé	–	UUID	Facultatif	Entité et –
Projet	created_at	Date de création	–	TIMESTAMP	Obligatoire	Gérée automatiquement
Projet	updated_at	Date de modification	–	TIMESTAMP	Obligatoire	Gérée automatiquement
Partenariat	id	Identifiant du partenariat	–	UUID	Obligatoire	Clé primaire
Partenariat	intitule	Intitulé du partenariat	–	VARCHAR	Obligatoire	–
Partenariat	objet	Objet du partenariat	–	TEXT	Facultatif	–
Partenariat	domaine	Domaine du partenariat	–	ENUM DomaineParte nariat	Obligatoire	–
Partenariat	date_debut	Date de début	–	TIMESTAMP	Obligatoire	–
Partenariat	date_fin	Date de fin	–	TIMESTAMP	Facultatif	–

Partenariat	statut	État du partenariat	–	ENUM StatutPartenariat	Obligatoire	–
Partenariat	actions_realisees	Actions réalisées	–	TEXT	Facultatif	–
Partenariat	actions_en_cours	Actions en cours	–	TEXT	Facultatif	–
Partenariat	actions_prevues	Actions prévues	–	TEXT	Facultatif	–
Partenariat	resultats_obtenus	Résultats obtenus	–	TEXT	Facultatif	–
Partenariat	archive	Indique si le partenariat est archivé	–	BOOLEAN	Obligatoire	Valeur par défaut : false
Partenariat	soumis_le	Date de soumission	–	TIMESTAMP	Facultatif	–
Partenariat	publie_le	Date de publication	–	TIMESTAMP	Facultatif	–
Partenariat	created_at	Date de création	–	TIMESTAMP	Obligatoire	Gérée automatiquement
Partenariat	updated_at	Date de modification	–	TIMESTAMP	Obligatoire	Gérée automatiquement
Convention	id	Identifiant de la convention	–	UUID	Obligatoire	Clé primaire
Convention	reference	Référence de la convention	–	VARCHAR	Obligatoire	Unicité à confirmer
Convention	titre	Titre de la convention	–	VARCHAR	Obligatoire	–
Convention	date_signature	Date de signature	–	TIMESTAMP	Facultatif	–
Convention	date_debut	Date de début	–	TIMESTAMP	Facultatif	–
Convention	date_fin	Date de fin	–	TIMESTAMP	Facultatif	–
Convention	statut	Statut de suivi	–	ENUM StatutConvention	Obligatoire	–

Convention	description	Description de la convention	–	TEXT	Facultatif	–
Convention	archive	Indique si la convention est archivée	–	BOOLEAN	Obligatoire	Valeur par défaut : false
Convention	partenariat_id	Partenariat associé	–	UUID	Facultatif	–
Convention	projet_id	Projet associé	–	UUID	Facultatif	–
Convention	created_at	Date de création	–	TIMESTAMP	Obligatoire	Gérée automatiquement
Convention	updated_at	Date de modification	–	TIMESTAMP	Obligatoire	Gérée automatiquement
Document	id	Identifiant du document	–	UUID	Obligatoire	Clé primaire
Document	nom_fichier	Nom du fichier	–	VARCHAR	Obligatoire	–
Document	url_fichier	Adresse du fichier	–	VARCHAR	Obligatoire	Stockage et type à confirmer
Document	type	Type du document	–	ENUM TypeDocument	Obligatoire	–
Document	version	Numéro de version	–	INTEGER	Obligatoire	Valeur par défaut : 1
Document	est_archive	Indique si le document est archivé	–	BOOLEAN	Obligatoire	Valeur par défaut : false
Document	projet_id	Projet associé	–	UUID	Facultatif	–
Document	convention_id	Convention associée	–	UUID	Facultatif	–
Document	cotutelle_id	Cotutelle associée	–	UUID	Facultatif	–
Document	partenariat_id	Partenariat associé	–	UUID	Facultatif	–
Document	created_at	Date de création	–	TIMESTAMP	Obligatoire	Gérée automatiquement

Document	updated_at	Date de modification	–	TIMESTAMP	Obligatoire	Gérée automatiquement
Publication	id	Identifiant de la publication	–	UUID	Obligatoire	Clé primaire
Publication	titre	Titre public	–	VARCHAR	Obligatoire	–
Publication	description_publique	Description destinée au public	–	TEXT	Facultatif	–
Publication	date_publication	Date de publication	–	TIMESTAMP	Facultatif	–
Publication	statut	État de publication	–	ENUM StatutPublication	Obligatoire	–
Publication	resultats	Résultats présentés publiquement	–	TEXT	Facultatif	–
Publication	image_couverture_url	Adresse de l'image de couverture	–	VARCHAR	Facultatif	–
Publication	projet_id	Projet publié	–	UUID	Obligatoire	Clé étrangère et unicité à confirmer
Publication	created_at	Date de création	–	TIMESTAMP	Obligatoire	Gérée automatiquement
Publication	updated_at	Date de modification	–	TIMESTAMP	Obligatoire	Gérée automatiquement

Source : fichiers de configuration et manifestes de dépendances du projet.

Ce tableau reprend les rubriques des principales entités. Les types, longueurs, clés et valeurs d'énumération doivent être vérifiés avant validation définitive.

5.2.3. Entités d'association et entités de support

Le modèle comprend six entités d'association : PartenariatEtablissement, PartenariatPartenaire, ProjetPartenaire, ProjetBailleur, ProjetBeneficiaire et ProjetSite. Chacune relie les entités métier correspondantes et possède sa propre clé primaire. Pour chaque table, une contrainte d'unicité empêche de créer deux fois la même paire d'association.

L'entité Contact est rattachée à un partenaire. L'entité Coordonnee peut être rattachée à un partenaire ou à un contact, mais pas aux deux simultanément. Une contrainte d'unicité empêche également la répétition d'une même coordonnée normalisée pour un même type.

Les entités de support couvrent notamment les notifications (Notification), les actions consignées (HistoriqueAction) et les exports (ExportHistorique). Les synthèses sont structurées par établissement et par année : SyntheseItem distingue les éléments selon leur type, tandis que SynthesePartenaireStrategique associe un partenaire stratégique à un établissement. Des contraintes d'unicité préviennent les doublons correspondants.

5.3. Règles de gestion

Le tableau ci-dessous synthétise les règles observables dans le modèle TypeORM, ses migrations et le code applicatif. Les numéros RG01 à RG31 servent de repères dans ce chapitre

Tableau 5.2 : Règles de gestion proposées pour la plateforme

Numéro	**Énoncé**	**Entité(s) concernée(s)**	**Fondement dans l'application**
RG01	Un utilisateur possède un rôle PRESIDENCE, ETABLISSEMENT ou DECIDEUR.	Utilisateur	Enum Role
RG02	Le rattachement d'un utilisateur à un établissement est facultatif.	Utilisateur, Etablissement	Relation facultative dans Utilisateur
RG03	L'adresse électronique d'un utilisateur est unique.	Utilisateur	Contrainte d'unicité sur email
RG04	Un partenaire possède le statut ACTIF ou SUSPENDU.	Partenaire	Enum StatutPartenaire
RG05	La visibilité d'un partenaire est PUBLIC ou PRIVE.	Partenaire	Enum Visibilite
RG06	Un partenaire peut être rattaché à un établissement ; ce rattachement est facultatif.	Partenaire, Etablissement	Relation facultative dans Partenaire
RG07	Chaque contact est rattaché à un partenaire.	Contact, Partenaire	Relation obligatoire dans Contact
RG08	Une coordonnée est rattachée à un seul propriétaire : un partenaire ou un contact.	Coordonnee, Partenaire, Contact	Contrainte de migration ck_coordonnee_un_proprietaire

RG09	Une même coordonnée normalisée ne peut pas être enregistrée deux fois pour un même type.	Coordonnee	Unicité sur type et valeur_normalisee
RG10	Une même association partenariat–établissement ne peut pas être créée deux fois.	PartenariatEtablissement	Unicité sur partenariat_id et etablissement_id
RG11	Une même association partenariat–partenaire ne peut pas être créée deux fois.	PartenariatPartenaire	Unicité sur partenariat_id et partenaire_id
RG12	Une même association projet–partenaire ne peut pas être créée deux fois.	ProjetPartenaire	Unicité sur projet_id et partenaire_id
RG13	Une même association projet–bailleur ne peut pas être créée deux fois.	ProjetBailleur	Unicité sur projet_id et bailleur_id
RG14	Une même association projet–bénéficiaire ne peut pas être créée deux fois.	ProjetBeneficiaire	Unicité sur projet_id et beneficiaire_id
RG15	Une même association projet–site ne peut pas être créée deux fois.	ProjetSite	Unicité sur projet_id et site_id
RG16	Le statut d'un projet appartient à la liste BROUILLON, EN_ATTENTE, EN_COURS, SUSPENDU, EXPIRE ou TERMINE.	Projet	Enum StatutProjet
RG17	Le statut d'un partenariat appartient à la liste BROUILLON, EN_ATTENTE, EN_COURS, SUSPENDU ou TERMINE.	Partenariat	Enum StatutPartenariat
RG18	Le statut d'une convention appartient à la liste BROUILLON, EN_ATTENTE_SIGNATURE, SIGNEE, ACTIVE, SUSPENDUE ou EXPIREE.	Convention	Enum StatutConvention
RG19	La référence d'une convention est unique.	Convention	Contrainte d'unicité sur reference

RG20	Une convention peut avoir un lien facultatif vers un partenariat et un lien facultatif vers un projet. Le modèle ne déclare pas de contrainte imposant qu'un seul de ces liens soit renseigné.	Convention, Partenariat, Projet	Deux relations facultatives dans Convention
RG21	Chaque document est rattaché à exactement un parent : projet, convention, cotutelle de thèse ou partenariat.	Document, Projet, Convention, CotutelleThese, Partenariat	Contrainte ck_document_un_parent
RG22	Une publication est associée à un seul projet, et un projet ne peut avoir qu'une publication.	Publication, Projet	Relation un-à-un et unicité sur projet_id
RG23	Le statut d'une publication est BROUILLON ou PUBLIEE.	Publication	Enum StatutPublication
RG24	Le partenaire, le partenariat, le projet, la convention et la cotutelle disposent d'un indicateur d'archivage ; le document dispose de l'indicateur est_archive.	Partenaire, Partenariat, Projet, Convention, CotutelleThese, Document	Champs d'archivage des entités
RG25	Un compte de rôle ETABLISSEMENT peut créer et soumettre des données dans le périmètre de son établissement, selon les fonctions disponibles.	Partenaire, Partenariat, Projet	Contrôleurs et services métier
RG26	La publication des partenaires, partenariats et projets est réservée à la Présidence.	Partenaire, Partenariat, Projet	Contrôleurs et services métier
RG27	La Présidence peut approuver ou rejeter une demande de modification d'un projet. En cas d'approbation, le projet repasse en brouillon pour un nouveau cycle de modification et de soumission.	Projet, Notification, HistoriqueAction	Service de gestion des demandes de modification
RG28	Une convention non archivée dont la date de fin est dépassée passe au statut EXPIREE et déclenche des notifications selon les destinataires prévus par le traitement d'expiration.	Convention, Notification, Projet, Utilisateur	Règle ConventionExpireeRule

RG29	Les routes de l'espace public sont accessibles sans authentification et présentent les ressources conformément à leurs critères de visibilité et de publication.	Ressources publiques	Routes publiques et service de l'espace public
RG30	Les exports standards de projets et de conventions sont proposés en PDF ou XLSX aux rôles autorisés. Les exports avancés et leur historique sont réservés à la Présidence.	Exports, ExportHistorique	Contrôleurs d'export et règles de rôles
RG31	L'historique conserve des actions telles que la création, la modification, la soumission, la publication, l'archivage, la restauration et la suppression.	HistoriqueAction	Enum ActionHistorique et service d'historique

Source : règles proposées à partir des informations communiquées.

Ce tableau rassemble vingt-sept règles proposées, qu'il distingue de la numérotation officielle du cahier des charges. Leurs énoncés et leurs références exactes doivent être vérifiés avant d'être repris comme règles définitives.

5.4. Diagramme de cas d'utilisation

5.4.1. Acteurs du système

Le système compte quatre acteurs : la Présidence, l'Établissement, le Décideur et le Visiteur. Les trois premiers disposent d'un accès authentifié, le Visiteur accédant à l'espace public sans compte.

La Présidence gère les données qui relèvent de ses droits et publie les données soumises destinées à être rendues publiques. L'Établissement gère les données autorisées pour son périmètre et n'a pas accès aux conventions, aux documents, aux publications ni aux synthèses. Le Décideur consulte en lecture seule les données métier qui lui sont accessibles. Le Visiteur consulte les informations publiques.

5.4.2. Diagramme global des cas d'utilisation

La figure 5.1 regroupe les fonctions principales de la plateforme en onze cas d'utilisation, répartis entre les quatre acteurs : s'authentifier, gérer les partenaires et les partenariats dans son périmètre, gérer les établissements, créer un projet, soumettre un projet, publier les projets

soumis, gérer les conventions et les documents, consulter son tableau de bord, consulter les données en lecture seule, consulter les synthèses et consulter les contenus rendus publics.

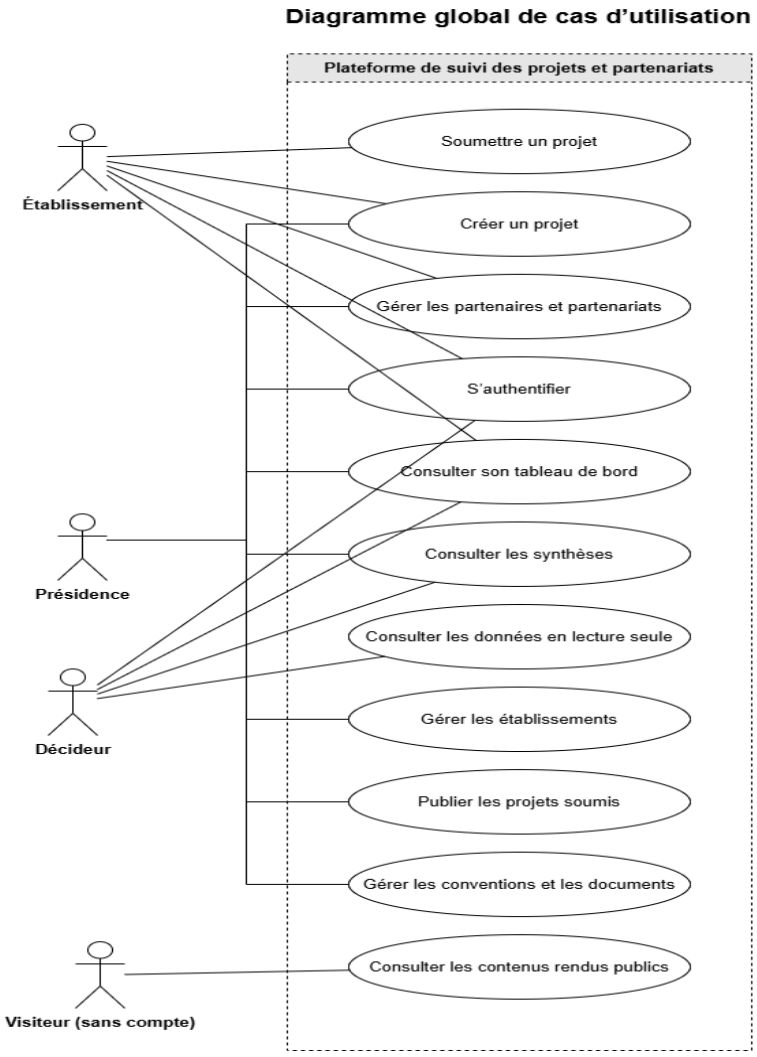


Figure 5.1 : Diagramme de cas d'utilisation global

Source : réalisation personnelle avec Draw.io, d'après les règles fonctionnelles de la plateforme.

La Présidence gère les données de son ressort, publie les projets soumis, gère les conventions et les documents et consulte les synthèses. L'Établissement agit dans son propre périmètre : il crée et soumet ses projets et gère ses partenaires et ses partenariats. Le Décideur consulte les données en lecture seule, ainsi que les synthèses et son tableau de bord. Le Visiteur, sans compte, ne consulte que les contenus rendus publics.

5.4.3. Cas d'utilisation

La demande de modification peut être envoyée par un compte Établissement pour un projet non archivé de son propre établissement, créé par un établissement et déjà soumis, en cours ou publié. La Présidence peut approuver ou rejeter la demande. En cas d'approbation, le projet

repasse en brouillon pour être modifié puis soumis de nouveau ; en cas de rejet, il reste verrouillé.

Les comptes Présidence, Établissement et Décideur peuvent consulter les archives de partenaires, partenariats, projets, conventions, cotutelles et documents. Pour un compte Établissement, la consultation est limitée aux données de son établissement. La restauration d'un élément archivé est réservée à la Présidence.

L'export standard des projets et des conventions est proposé en PDF ou XLSX aux rôles Présidence et Décideur. Les fonctions d'export avancé et la consultation de leur historique sont réservées à la Présidence.

5.5. Description textuelle des cas d'utilisation

Deux cas d'utilisation sont détaillés dans cette section : l'authentification, commune aux trois rôles authentifiés, et la création et la soumission d'un projet, scénario représenté dans les diagrammes de séquence. Les autres fonctionnalités sont décrites dans les sections correspondantes du mémoire.

5.5.1. S'authentifier

Le tableau 5.3 décrit le cas d'utilisation d'authentification.

Tableau 5.3 : Description textuelle du cas d'utilisation « S'authentifier »

Rubrique	**Contenu**
Nom	S'authentifier
Acteur(s)	Présidence, Établissement, Décideur
Objectif	Ouvrir une session et obtenir les éléments nécessaires à l'accès aux fonctions protégées
Préconditions	Le compte existe et l'utilisateur dispose de ses identifiants
Déclencheur	L'utilisateur envoie une requête à POST /api/auth/login
Scénario nominal	1. L'utilisateur saisit son adresse électronique et son mot de passe. 2. Le système vérifie les identifiants. 3. Le système retourne la réponse d'authentification prévue.
Scénarios alternatifs	Récupération du mot de passe par POST /api/auth/mot-de-passe-oublie et réinitialisation par POST /api/auth/reinitialiser-mot-de-passe.
Scénarios d'exception	Identifiants invalides ou compte non autorisé ;
Postconditions	L'utilisateur accède à l'espace correspondant à son rôle

Règles de gestion liées	RG1, RG2 et RG3.

Source : rôles et scénario d'authentification communiqués.

Ce cas concerne les trois profils authentifiés ; le Visiteur consulte les informations publiques sans y recourir.

5.5.2. Créer et soumettre un projet

Le tableau 5.4 décrit la création et la soumission d'un projet par un Établissement.

Tableau 5.4 : Description textuelle du cas d'utilisation « Créer et soumettre un projet »

Rubrique	**Contenu**
Nom	Créer et soumettre un projet
Acteur(s)	Acteur principal : Établissement ; acteur secondaire : Présidence pour les opérations qui lui sont attribuées
Objectif	Enregistrer un projet relevant du périmètre de l'établissement et transmettre les données autorisées
Préconditions	L'utilisateur possède un accès authentifié et le projet respecte les règles applicables
Déclencheur	Création par POST /api/projets et soumission par POST /api/projets/:id/soumettre
Scénario nominal	1. L'Établissement renseigne les informations du projet. 2. Le système vérifie les données reçues. 3. Le projet est enregistré. 4. L'Établissement réalise l'opération de soumission prévue. 5. Le système enregistre la soumission.
Scénarios alternatifs	Modification du projet avant sa soumission ; états intermédiaires
Scénarios d'exception	Données invalides, périmètre incompatible ou état ne permettant pas l'opération ; codes HTTP et messages
Postconditions	Le projet est enregistré
Règles de gestion liées	RG12, RG22 et RG23

Source : scénario proposé à partir des informations fonctionnelles communiquées.

Ce scénario distingue la création de la soumission. Les routes et le statut obtenu après soumission doivent être vérifiés avant validation du tableau.

5.6. Diagramme de séquence système

5.6.1. Séquence système : soumission d'un projet

Le diagramme de séquence système traite la plateforme comme une boîte noire : il ne montre que l'échange entre l'acteur et le système. Dans le scénario retenu, l'Établissement demande la soumission d'un projet en indiquant son identifiant, et le système lui renvoie le projet avec le statut « En attente ».

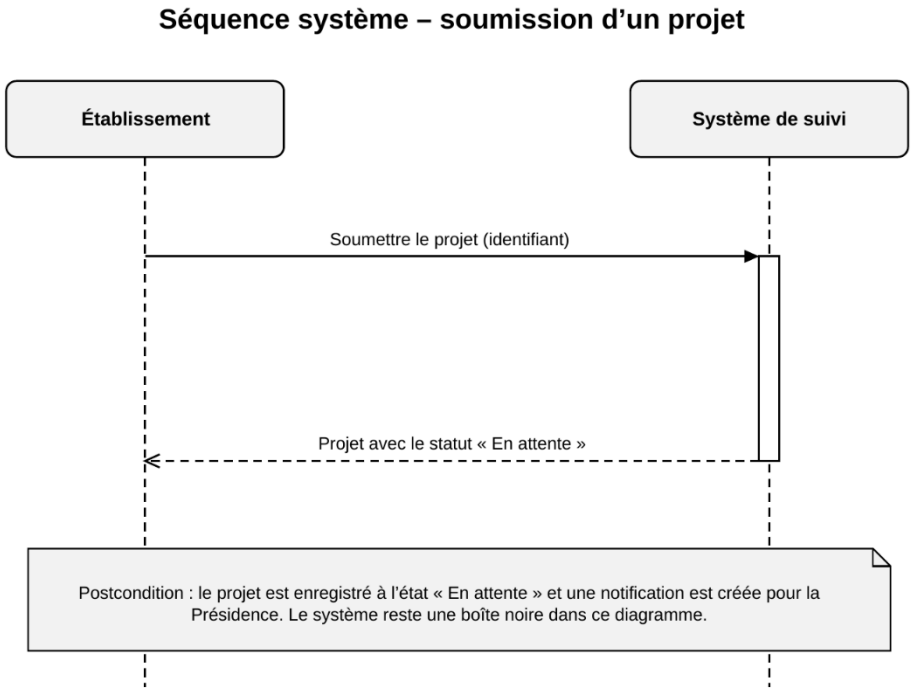


Figure 5.2 : Séquence système de la soumission d'un projet

Source : réalisation personnelle avec Draw.io, d'après le scénario de soumission d'un projet.

À l'issue de la soumission, le projet est enregistré à l'état « En attente » et une notification est créée pour la Présidence. Ces deux effets figurent en postcondition ; leur mécanisme est détaillé dans le chapitre VI.

5.7. Modèle de domaine

5.7.1. Entités principales

Le modèle de domaine retient huit concepts métier : Établissement, Utilisateur, Projet, Partenariat, Partenaire, Convention, Document et Notification. Il reste au niveau métier : les entités d'association, de support et d'historique (par exemple Contact, Coordonnée, Publication ou Cotutelle) figurent dans le dictionnaire des données.

5.7.2. Modèle de domaine global

La figure 5.3 présente ces concepts et leurs associations. Les établissements et les partenaires sont reliés par les partenariats, selon des associations plusieurs-à-plusieurs. Un établissement regroupe des comptes utilisateurs et des projets. Un partenariat peut être rattaché à des projets et à des conventions, et un projet à des conventions. Les documents se rattachent, de façon facultative, à un projet ou à une convention, et un utilisateur reçoit des notifications.

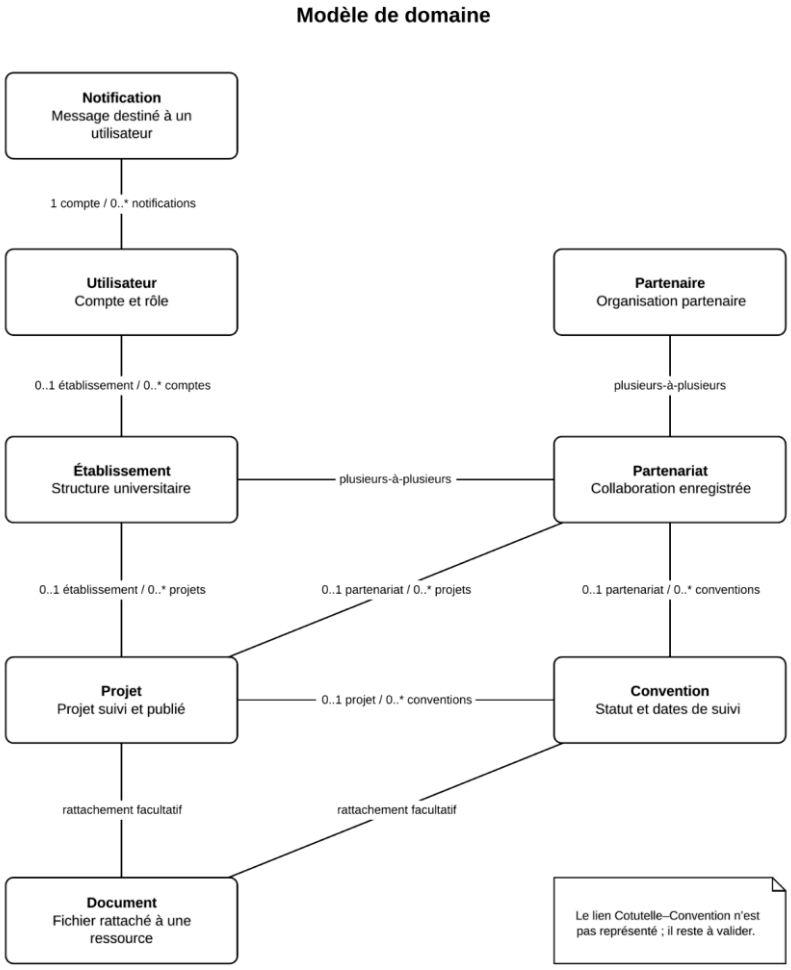


Figure 5.3 : Modèle de domaine des principaux concepts de la plateforme

Source : réalisation personnelle avec Draw.io, d'après le modèle de données de la plateforme.

CHAPITRE VI : CONCEPTION DÉTAILLÉE

Ce chapitre présente l'architecture de l'application, la séquence de conception, le diagramme de classes, et le diagramme de paquetage.

6.1. Architecture de l'application

La plateforme adopte une architecture modulaire organisée par domaine fonctionnel. Le backend est une application NestJS composée de modules intégrés dans une même application, et non un ensemble de microservices autonomes. Les principaux domaines sont l'authentification, les utilisateurs, les établissements, les partenaires, les partenariats, les projets, les cotutelles, les conventions, les documents, les publications, les notifications, les statistiques et les services de consultation.

Le navigateur accède au frontend développé avec React et Vite. Dans l'environnement Docker de développement, Vite relaie les requêtes destinées à /api vers le service backend. Celui-ci, une application NestJS, traite les requêtes HTTP, applique les contrôles communs et confie le traitement au service du domaine concerné. TypeORM assure l'accès aux données stockées dans PostgreSQL 16.

Les guards globaux s'exécutent dans cet ordre : ThrottlerGuard, JwtAuthGuard, puis RolesGuard. Les routes publiques contournent l'authentification JWT lorsque leur configuration le prévoit. Le ValidationPipe global contrôle ensuite les données reçues, avec des options de filtrage, de refus des propriétés non autorisées et de transformation. Des contrôles de périmètre supplémentaires s'appliquent notamment aux opérations sur les projets et les partenariats. Le backend comprend en outre un intercepteur global de réponse et un filtre global d'exceptions.

Les fichiers sont gérés par un stockage local. Les logos et les médias de publication prévus sont servis comme ressources publiques, alors que le téléchargement des documents passe par une route soumise aux contrôles d'accès.

Les trois services et leur exécution sont représentés par le diagramme de déploiement de la section 6.5 (figure 6.4).

6.1.1. Modules et responsabilités

Le module principal rassemble les modules de l'application, répartis par domaine fonctionnel. Ils partagent des mécanismes communs : base de données, authentification, validation, gestion des notifications et historique.

Les modules métier traitent notamment les utilisateurs, les établissements, les partenaires, les partenariats, les projets, les cotutelles, les conventions, les documents et les publications. Les modules transversaux prennent en charge les notifications, les archives, la recherche, les exports, les statistiques, les tableaux de bord, les synthèses, l'espace public, les messages de contact et l'historique.

Un mécanisme d'événements internes prend en charge les événements métier : la soumission d'un projet, d'un partenariat ou d'un partenaire peut ainsi déclencher des listeners de notification.

6.1.2. Contrôles et traitements communs

L'application applique d'abord une limitation globale du débit, puis l'authentification JWT et le contrôle des rôles. Les routes explicitement publiques peuvent être consultées sans authentification. Des guards supplémentaires, placés sur les contrôleurs concernés, vérifient le périmètre de l'Établissement.

Une fois les guards passés, le ValidationPipe global traite les DTO, puis le contrôleur transmet la requête au service métier. Ce dernier accède à PostgreSQL par TypeORM pour lire ou écrire des données. Le gestionnaire global d'exceptions traite les erreurs et l'intercepteur global structure les réponses réussies.

6.2. Diagramme de séquence de conception

Les diagrammes de séquence de conception décrivent les échanges entre les composants internes de l'application. Un seul est présenté ici, celui du scénario de soumission d'un projet, déjà traité en séquence système. L'authentification est décrite en texte.

6.2.1. Authentification

La route de connexion est publique du point de vue de l'authentification JWT, mais elle reste soumise à la limitation du débit et à la validation des données. Le service d'authentification recherche l'utilisateur, vérifie le mot de passe et renvoie les jetons ainsi que les informations de profil prévues dans la réponse.

6.2.2. Création et soumission d'un projet

Le scénario retenu est celui de la soumission, déjà vu au chapitre V. Le projet existe à ce stade. Sa création passe par les guards globaux, le guard de périmètre du contrôleur des projets et la validation du DTO, puis par le service métier, qui détermine l'établissement associé à partir du rôle de l'utilisateur et regroupe les écritures dans une transaction.

La figure 6.1 détaille la soumission. L'utilisateur de l'Établissement déclenche l'action dans l'interface, qui envoie une requête POST /api/projets/:id/soumettre. L'API applique les contrôles communs, puis ProjetsService vérifie le rôle, le périmètre et le statut, lit le projet dans PostgreSQL et, s'il se trouve bien dans le périmètre de l'utilisateur, enregistre l'état EN_ATTENTE ainsi que la date de soumission. Le service émet ensuite l'événement projet.soumis, dont le listener crée la notification destinée aux comptes de la Présidence. La réponse remonte jusqu'à l'interface, qui affiche le statut « En attente ».

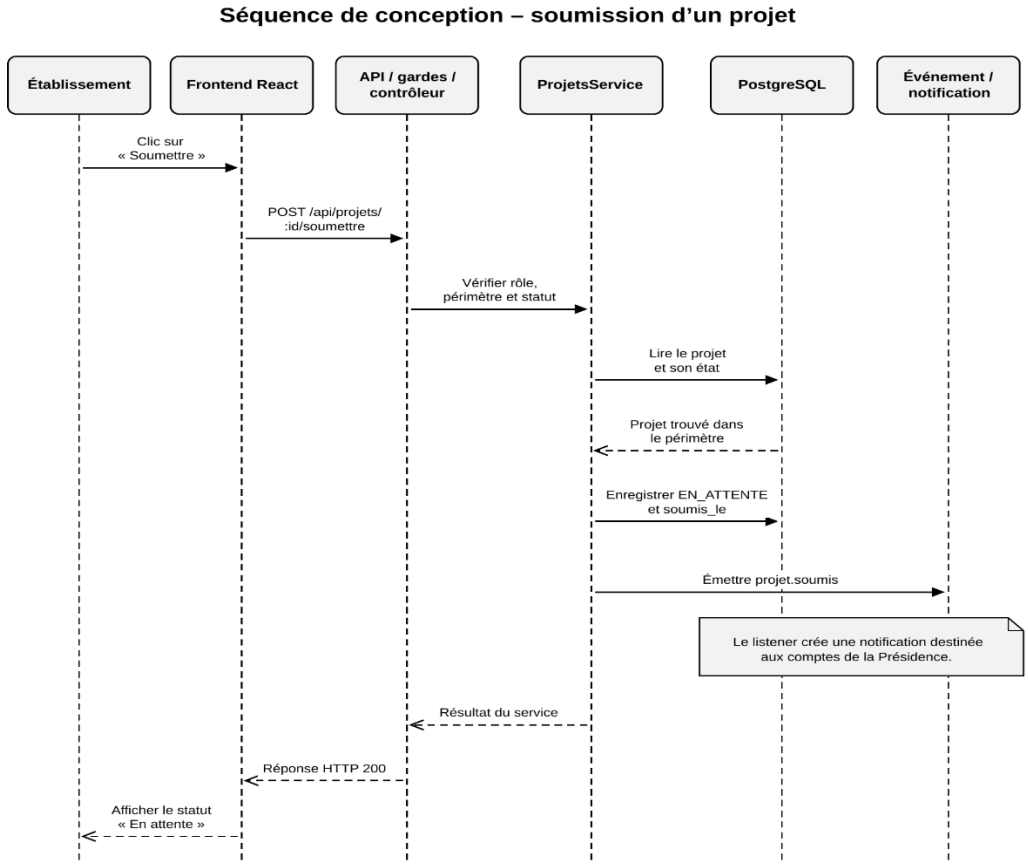


Figure 6.1 : Séquence de conception de la soumission d'un projet

Source : réalisation personnelle avec Draw.io, d'après le fonctionnement de l'application.

Pour la lisibilité, le diagramme ne représente pas l'inscription de l'action SOUMISSION dans l'historique, effectuée par HistoriqueService.enregistrer(). La soumission est déclenchée par la route POST /api/projets/:id/soumettre, traitée par ProjetsController.soumettre() puis par ProjetsService.soumettre() ; l'événement projet.soumis est pris en charge par SoumissionListener.onProjetSoumis().

6.3. Diagramme de classe

Le diagramme de classes se limite aux classes qui interviennent dans la soumission d'un projet. ProjetsController reçoit la requête, CreateProjetDto porte les données de création et ProjetsService effectue les traitements. Projet et Etablissement sont les entités concernées. SoumissionListener réagit à l'événement projet.soumis et NotificationsService crée les notifications destinées à la Présidence.

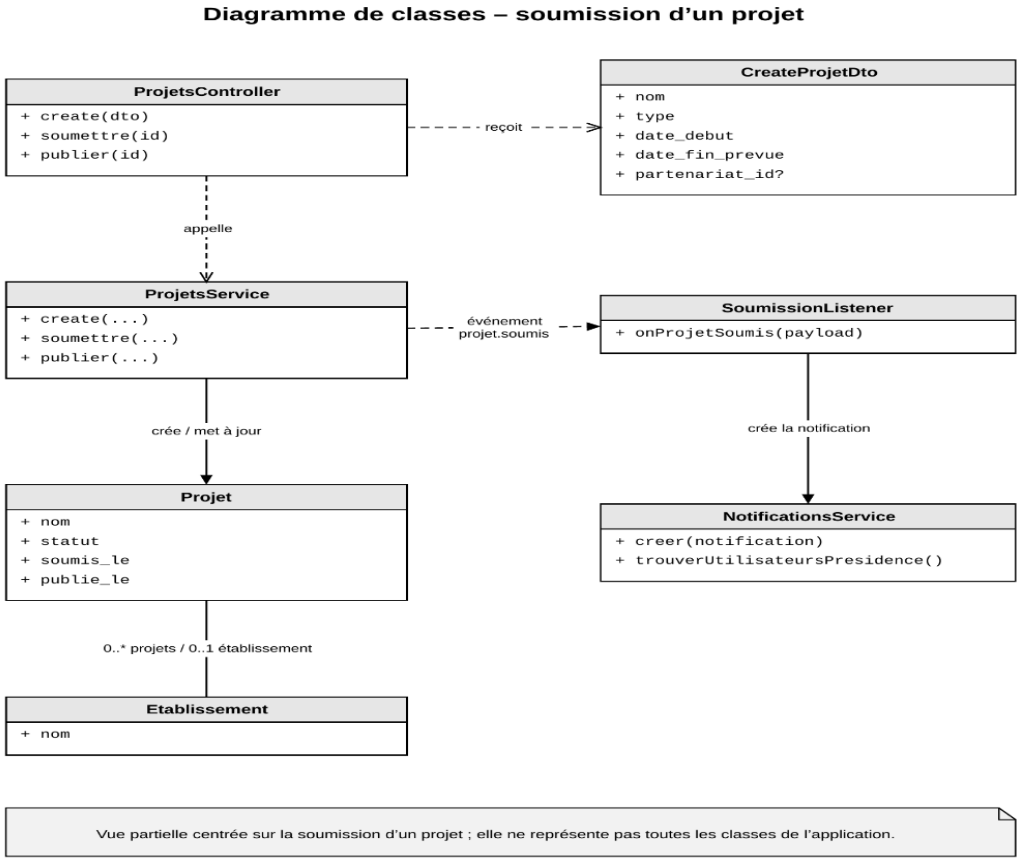


Figure 6.2 : Diagramme de classes du traitement de la soumission d'un projet

Source : réalisation personnelle avec Draw.io, d'après les composants techniques de la plateforme.

soumission d'un projet, sans reproduire toutes les classes de l'application. Le traitement fait intervenir ProjetsController.soumettre(), qui appelle ProjetsService.soumettre(). Ce service vérifie que le projet a été créé par un établissement et qu'il est au statut BROUILLON, puis renseigne son statut à EN_ATTENTE et sa date soumis_le.

Le service émet ensuite l'événement projet.soumis. Celui-ci est traité par SoumissionListener.onProjetSoumis(), qui s'appuie sur NotificationsService pour notifier les utilisateurs actifs de la Présidence. Le service consigne également l'action SOUMISSION au

moyen de HistoriqueService.enregistrer(), puis retourne le projet sous la forme d'un ProjetResponseDto.

Les principaux éléments à représenter sont donc les classes ProjetsController, ProjetsService, Projet, SoumissionListener, NotificationsService et HistoriqueService, ainsi que l'événement projet.soumis. Les attributs du projet concernés par ce traitement comprennent notamment id, nom, statut, cree_par_role et soumis_le.

6.4. Diagramme de paquetage

Le diagramme de paquetage donne une vue synthétique de l'organisation de l'application en quatre ensembles : le frontend React et Vite, l'API NestJS (préfixe /api, contrôleurs REST, validation et guards), les modules métier regroupés par domaine fonctionnel, et la couche de persistance (TypeORM, migrations, PostgreSQL). Le frontend appelle l'API par HTTP, l'API s'appuie sur les modules métier, et ceux-ci accèdent aux données par TypeORM. Tous les modules appartiennent à une seule application NestJS : leur regroupement en paquetages ne fait pas d'eux des microservices indépendants.

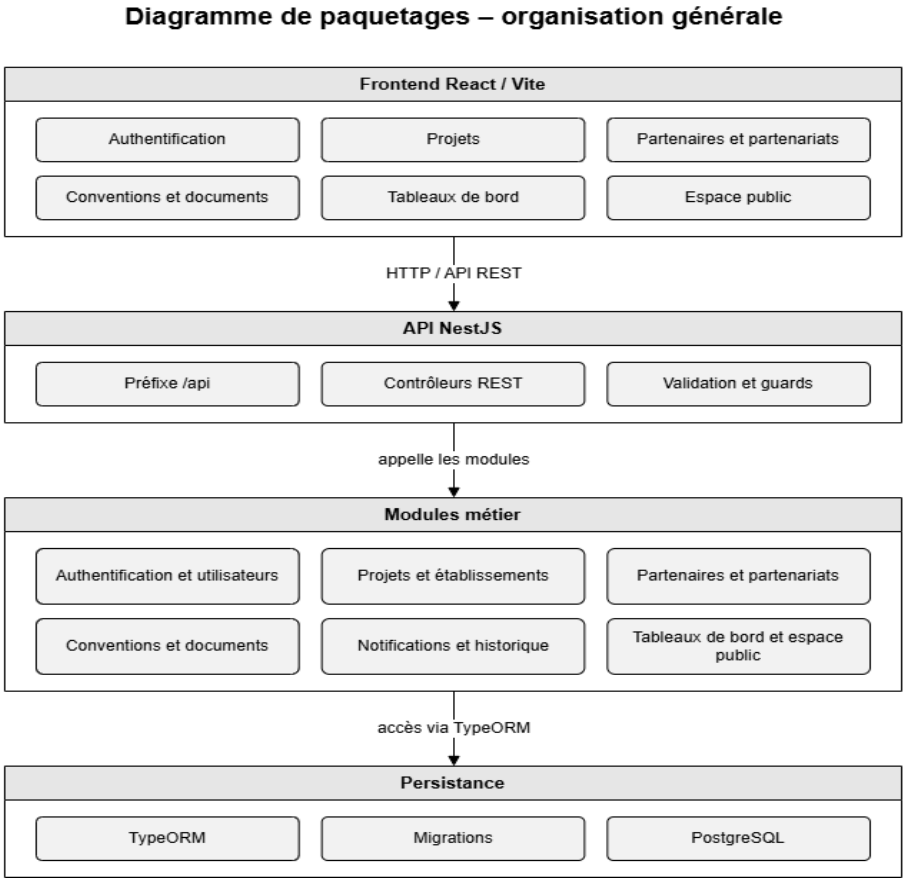


Figure 6.3 : Diagramme de paquetage de l'application

Source : réalisation personnelle avec Draw.io, d'après l'organisation modulaire de l'application

PARTIE III : MISE EN ŒUVRE ET RÉSULTATS

CHAPITRE VII : MISE EN ŒUVRE

Ce chapitre présente l'architecture du système mise en œuvre, la configuration des outils, la génération de la base de données et les principales fonctions de la plateforme.

7.1. Architecture du système

La plateforme s'exécute à l'aide de trois services conteneurisés : le frontend React et Vite, le backend NestJS et PostgreSQL 16. Dans l'environnement Docker Compose de développement, les ports publiés sont 5173 pour le frontend, 3000 pour l'API et 5432 pour PostgreSQL.

Le navigateur accède à l'interface React servie par Vite. En développement, Vite transmet les requêtes /api au service backend sur le réseau Docker. Le backend applique les contrôles d'accès et la validation, puis confie les opérations métier aux services concernés, et TypeORM assure l'accès aux données de PostgreSQL.

PostgreSQL conserve ses données dans le volume nommé db_data, et le répertoire de téléversement du backend est monté à part. Les documents se téléchargent par une route protégée, tandis que les logos et les images de publication sont servis comme médias publics, selon la configuration de l'application.

L'architecture d'ensemble des services est décrite dans la section 6.1.

7.2. Configuration des outils de conception

Les diagrammes du mémoire ont été réalisés avec Draw.io, qui a servi à préparer les représentations de l'architecture, des paquetages, des classes et des séquences de conception.

Les figures du mémoire sont en noir et blanc et accompagnées d'un titre, d'une légende et d'une source. La mise en page doit rester lisible dans le document final.

La figure 7.1 est une capture de l'espace de travail de Draw.io, avec un diagramme du mémoire ouvert.

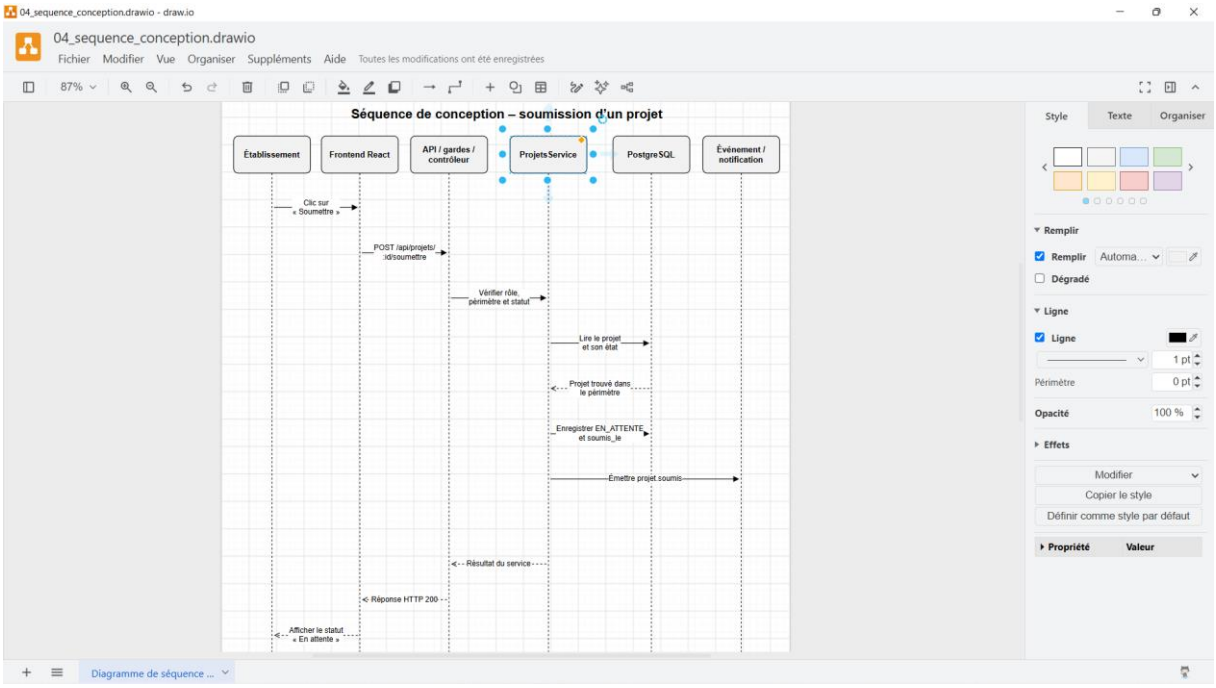


Figure 7.1 : Espace de travail de Draw.io

Source : capture d'écran personnelle.

7.3. Génération de la base de données

La base de données utilise PostgreSQL 16 et TypeORM 0.3.31. La configuration désactive la synchronisation automatique des entités, et les évolutions de structure passent par les migrations TypeORM. La version examinée compte vingt-deux migrations.

La commande de migration prévue dans l'environnement du backend est la suivante :

npm run migration:run

Dans l'environnement Docker Compose de développement, le service backend exécute les migrations avant de démarrer NestJS en mode surveillance. La base de données démarre avec un contrôle de santé, et le backend attend cet état pour se lancer.

Le volume db_data préserve les données PostgreSQL lors des redémarrages des conteneurs. Les variables nécessaires à la connexion sont transmises au service backend ; leurs valeurs sensibles ne sont pas reproduites dans ce mémoire.

La figure 7.2 montre le résultat de l'exécution des migrations.

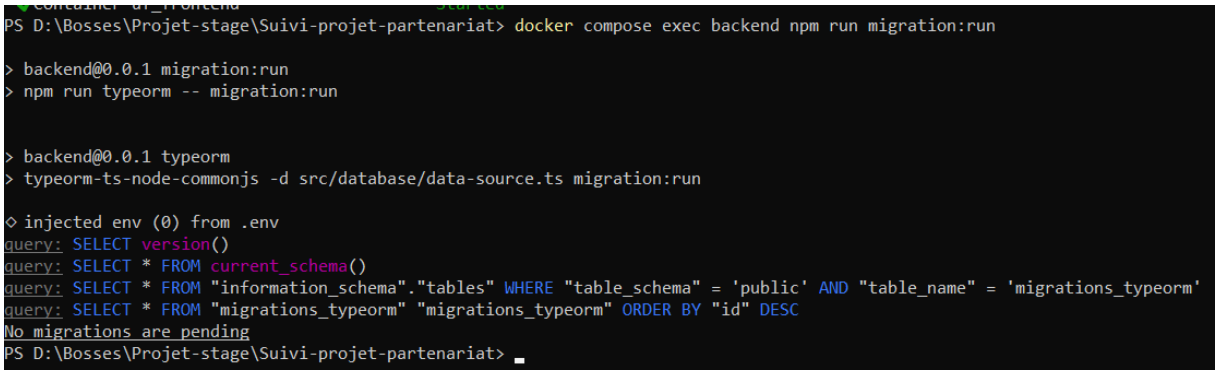


Figure 7.2 : Exécution des migrations de la base de données

Source : capture d'écran personnelle.

7.4. Configuration de l'environnement de développement

L'environnement de développement comprend Windows 11, Visual Studio Code, Git, GitHub, Docker et Docker Compose. Le backend utilise Node.js 20 Alpine, NestJS 11 et TypeScript ; le frontend, Node.js 22 Alpine, React 19, Vite et TypeScript.

Docker Compose organise les services frontend, backend et PostgreSQL. Les sources du frontend et du backend sont montées dans leurs conteneurs de développement, où Vite et NestJS surveillent les modifications. Le répertoire des fichiers téléversés est lui aussi monté dans le service backend [Docker, s.d.].

Le frontend utilise l'URL relative /api, que le serveur Vite relaie vers le service backend. Cette configuration évite que le navigateur ne communique directement avec une adresse locale propre à l'environnement du serveur.

La configuration globale du backend comprend Helmet, CORS, le préfixe /api, le ValidationPipe, un intercepteur global de réponse et un filtre global d'exceptions. Les guards globaux s'exécutent dans l'ordre ThrottlerGuard, JwtAuthGuard puis RolesGuard.



Figure 7.3 : Fichier docker-compose.yml

7.5. Extraits de code des fonctionnalités principales

Les extraits de code de cette section doivent provenir de la version de l'application retenue pour le mémoire. Chacun doit rester fidèle à la fonction correspondante et être accompagné de son emplacement et du nom de la fonction.

7.5.1. Authentification JWT

L'authentification vérifie les identifiants, l'état du compte et le mot de passe avant de renvoyer une réponse contenant les jetons prévus. Le service met aussi à jour les informations de connexion de l'utilisateur.

```
async login(dto: LoginDto): Promise<AuthResponseDto> {
  const { email, mot_de_passe } = dto;

  const utilisateur = await this.utilisateurRepo.findOne({
    where: { email },
    relations: { etablissement: true },
  });

  if (!utilisateur) throw new UnauthorizedException('Identifiants invalides');
  if (!utilisateur.actif) {
    throw new UnauthorizedException(
      'Votre compte a été désactivé. Veuillez contacter la Présidence.',
    );
  }

  const valid = await bcrypt.compare(mot_de_passe, utilisateur.password_hash);
  if (!valid) throw new UnauthorizedException('Identifiants invalides');

  await this.utilisateurRepo.update(utilisateur.id, {
    derniere_connexion: new Date(),
  });

  const { accessToken, refreshToken } = await this.genererTokens(
    utilisateur.id,
    utilisateur.email,
    utilisateur.role,
  );

  return {
    access_token: accessToken,
    refresh_token: refreshToken,
    expires_in: 3600,
    utilisateur: {
      id: utilisateur.id,
      email: utilisateur.email,
      role: utilisateur.role,
      etablissement_id: utilisateur.etablissement_id || undefined,
      etablissement_nom: utilisateur.etablissement?.nom || undefined,
      preferences: utilisateur.preferences ?? {},
    },
  };
}
```

Figure 7.4 : Authentification JWT — login

7.5.2. Création et soumission d'un projet

Lorsqu'un compte d'établissement crée un projet, le rattachement à l'établissement est déduit du compte authentifié. La création du projet et de ses associations se fait dans une transaction TypeORM.

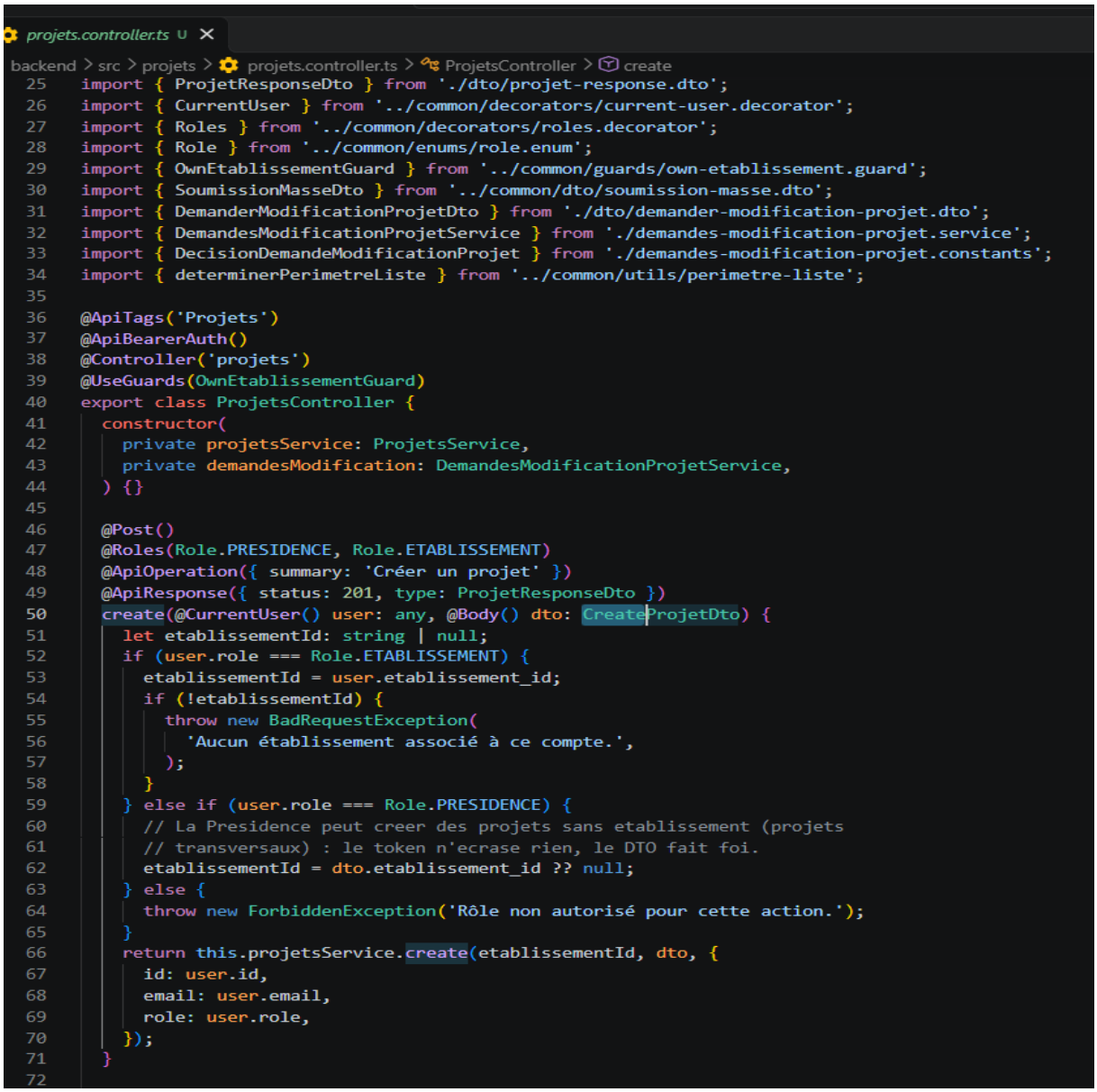


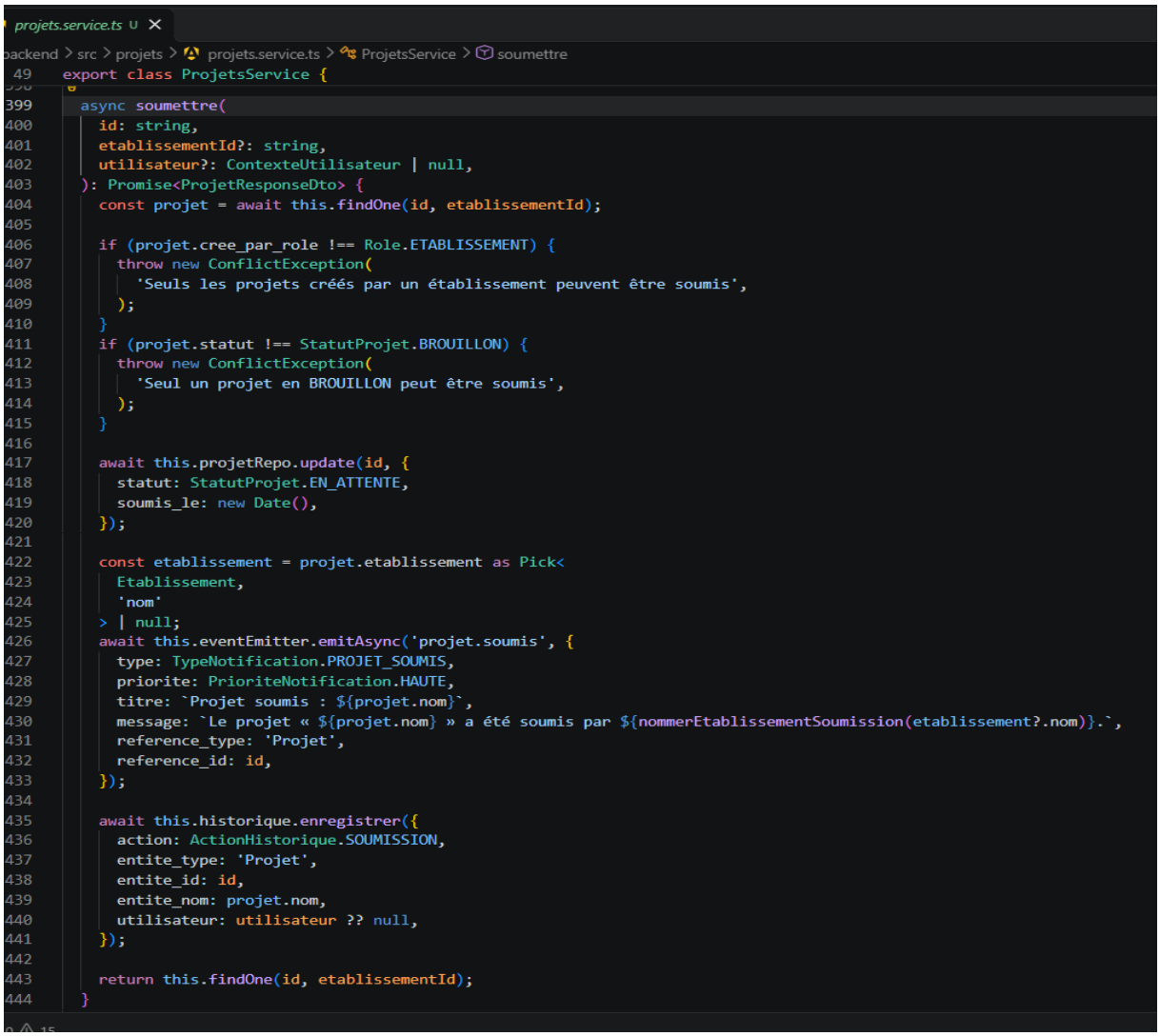
Figure 7.5 : Contrôleur des projets — création



```
49  export class ProjetsService {
72    async create(
77      const projetId = await this.dataSource.transaction(async (manager) => {
109       const projet = projetRepo.create({
110         nom: dto.nom,
111         intitule_officiel: dto.intitule_officiel ?? null,
112         type: dto.type,
113         domaine: dto.domaine ?? null,
114         budget: dto.budget ?? null,
115         date_debut: new Date(dto.date_debut),
116         date_fin_prevue: new Date(dto.date_fin_prevue),
117         statut:
118           utilisateur?.role === Role.ETABLISSEMENT
119             ? StatutProjet.BROUILLON
120             : dto.statut || StatutProjet.BROUILLON,
121         archive: false,
122         etablissement_id: etablissementId,
123         partenariat_id: dto.partenariat_id ?? null,
124         porteur_id: porteurId,
125         cree_par_utilisateur_id: utilisateur?.id ?? null,
126         cree_par_role: utilisateur?.role ?? null,
127       });
128
129       const savedProjet = await projetRepo.save(projet);
130       const projetPartenaireRepo = manager.getRepository(ProjetPartenaire);
131       const projetBeneficiaireRepo = manager.getRepository(ProjetBeneficiaire);
132       const projetBailleurRepo = manager.getRepository(ProjetBailleur);
133       const projetSiteRepo = manager.getRepository(ProjetSite);
134
135       if (dto.partenaires_ids) {
136         await projetPartenaireRepo.save(
137           [...new Set(dto.partenaires_ids)].map((partenaire_id) =>
138             projetPartenaireRepo.create({
139               projet_id: savedProjet.id,
140               partenaire_id,
141             }),
142           ),
143         );
144       }
145       if (dto.beneficiaires_ids) {
146         await projetBeneficiaireRepo.save(
147           [...new Set(dto.beneficiaires_ids)].map((beneficiaire_id) =>
148             projetBeneficiaireRepo.create({
149               projet_id: savedProjet.id,
150               beneficiaire_id,
151             }),
152           ),
```

Figure 7.6 : Service des projets — création

Seul l'Établissement peut soumettre un projet. Le service vérifie notamment que le projet peut être soumis, met à jour son statut, émet l'événement de soumission et consigne l'action dans l'historique.

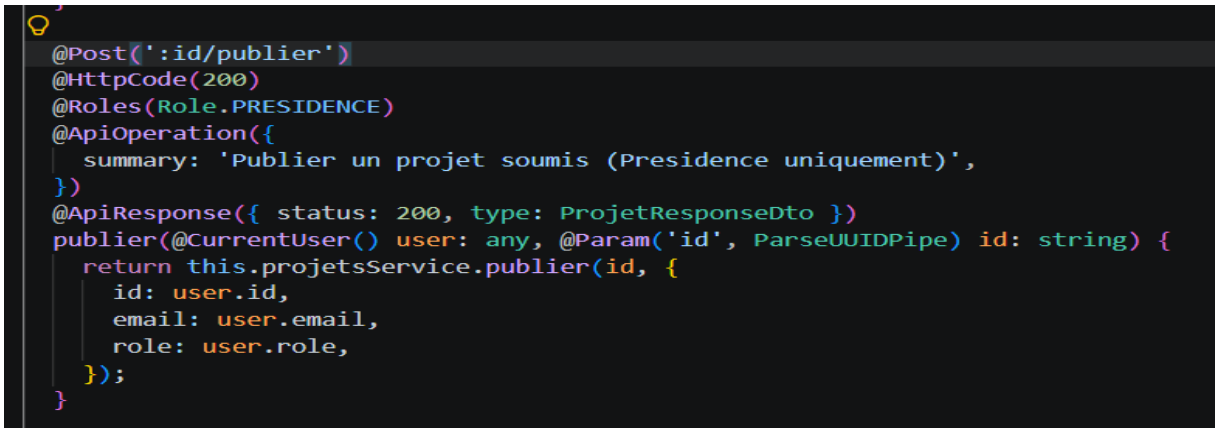


```
projets.service.ts U
backend > src > projets > projets.service.ts > ProjetsService > soumettre
 49  export class ProjetsService {
399    async soumettre(
400      id: string,
401      etablissementId?: string,
402      utilisateur?: ContexteUtilisateur | null,
403    ): Promise<ProjetResponseDto> {
404      const projet = await this.findOne(id, etablissementId);
405
406      if (projet.cree_par_role !== Role.ETABLISSEMENT) {
407        throw new ConflictException(
408          'Seuls les projets créés par un établissement peuvent être soumis',
409        );
410      }
411      if (projet.statut !== StatutProjet.BROUILLON) {
412        throw new ConflictException(
413          'Seul un projet en BROUILLON peut être soumis',
414        );
415      }
416
417      await this.projetRepo.update(id, {
418        statut: StatutProjet.EN_ATTENTE,
419        soumis_le: new Date(),
420      });
421
422      const etablissement = projet.etablissement as Pick<
423        Etablissement,
424        'nom'
425      > | null;
426      await this.eventEmitter.emitAsync('projet.soumis', {
427        type: TypeNotification.PROJET_SOUMIS,
428        priorite: PrioriteNotification.HAUTE,
429        titre: `Projet soumis : ${projet.nom}`,
430        message: `Le projet « ${projet.nom} » a été soumis par ${nommerEtablissementSoumission(etablissement?.nom)}.`,
431        reference_type: 'Projet',
432        reference_id: id,
433      });
434
435      await this.historique.enregistrer({
436        action: ActionHistorique.SOUMISSION,
437        entite_type: 'Projet',
438        entite_id: id,
439        entite_nom: projet.nom,
440        utilisateur: utilisateur ?? null,
441      });
442
443      return this.findOne(id, etablissementId);
444    }
```

Figure 7.7 : Service des projets — soumission

7.5.3. Publication d'un projet

La Présidence publie une donnée soumise destinée à être rendue publique. La publication met à jour le statut et la date de publication, et enregistre l'action correspondante ; elle ne constitue pas une étape de validation séparée.



```
@Post(':id/publier')
@HttpCode(200)
@Roles(Role.PRESIDENCE)
@ApiOperation({
  summary: 'Publier un projet soumis (Presidence uniquement)',
})
@ApiResponse({ status: 200, type: ProjetResponseDto })
publier(@CurrentUser() user: any, @Param('id', ParseUUIDPipe) id: string) {
  return this.projetsService.publier(id, {
    id: user.id,
    email: user.email,
    role: user.role,
  });
}
```

Figure 7.8 : Contrôleur des projets — publication

```
export class ProjetsService {
  async publier(
    id: string,
    utilisateur?: ContexteUtilisateur | null,
  ): Promise<ProjetResponseDto> {
    const projet = await this.findOne(id);

    const estDonneePresidence = projet.cree_par_role === Role.PRESIDENCE;
    if (!projet.soumis_le && !estDonneePresidence) {
      throw new ConflictException(
        'Seul un projet soumis ou créé par la Présidence peut être publié',
      );
    }
    if (projet.publie_le) {
      throw new ConflictException('Ce projet est déjà publié');
    }

    await this.projetRepo.update(id, {
      publie_le: new Date(),
      statut:
        projet.statut === StatutProjet.BROUILLON ||
        projet.statut === StatutProjet.EN_ATTENTE
          ? StatutProjet.EN_COURS
          : projet.statut,
    });

    this.eventEmitter.emit('projet.publie', {
      type: TypeNotification.PROJET_PUBLIE,
      priorite: PrioriteNotification.MOYENNE,
      titre: `Projet publie : ${projet.nom}`,
      message: `Le projet "${projet.nom}" a ete publie par la presidence.`,
      reference_type: 'Projet',
      reference_id: id,
      etablissement_id: projet.etablissement_id,
    });

    await this.historique.enregistrer({
      action: ActionHistorique.PUBLICATION,
      entite_type: 'Projet',
      entite_id: id,
      entite_nom: projet.nom,
      utilisateur: utilisateur ?? null,
    });

    return this.findOne(id);
  }
```

Figure 7.9 : Service des projets — publication

7.5.4. Gestion des conventions

La gestion des conventions repose sur le statut, les dates et les associations disponibles. À la création, le service vérifie les données reçues et utilise une transaction pour enregistrer la convention et ses relations. La Présidence peut créer, modifier et archiver une convention ; le Décideur peut aussi la consulter.

```
end > src > conventions > conventions.service.ts > ConventionsService > update
export class ConventionsService {
  async create(dto: CreateConventionDto): Promise<ConventionResponseDto> {
    const convention = await this.dataSource.transaction(async (manager) => {
      const cotutelle = await cotutelleRepo.findOne({
        where: { id: dto.cotutelle_id, archive: false },
      });
      if (!cotutelle) {
        throw new NotFoundException('Cotutelle non trouvee');
      }
    }

    const existante = await conventionRepo.findOne({
      where: { reference },
    });
    if (existante) {
      throw new ConflictException(
        `Une convention avec la reference "${reference}" existe deja`,
      );
    }

    const creee = await conventionRepo.save(
      conventionRepo.create({
        reference,
        titre: dto.titre,
        date_signature: dto.date_signature
          ? new Date(dto.date_signature)
          : null,
        date_debut: dto.date_debut ? new Date(dto.date_debut) : null,
        date_fin: dto.date_fin ? new Date(dto.date_fin) : null,
        statut: dto.statut || StatutConvention.BROUILLON,
        description: dto.description ?? null,
        archive: false,
        partenariat_id: dto.partenariat_id || null,
        projet_id: dto.projet_id || null,
      }),
    );

    if (dto.cotutelle_id) {
      await cotutelleRepo.update(dto.cotutelle_id, {
        convention_id: creee.id,
      });
    }

    return creee;
  });

  return this.findOne(convention.id);
}
```

Figure 7.10 : Service des conventions — création



```
export class ConventionsService {
  async update(
    await this.dataSource.transaction(async (manager) => {
      }
      if (dto.date_debut !== undefined) {
        data.date_debut = dto.date_debut ? new Date(dto.date_debut) : null;
      }
      if (dto.date_fin !== undefined) {
        data.date_fin = dto.date_fin ? new Date(dto.date_fin) : null;
      }
      if (dto.statut) data.statut = dto.statut;
      if (dto.description !== undefined)
        data.description = dto.description ?? null;
      if (dto.partenariat_id !== undefined) {
        data.partenariat_id = dto.partenariat_id || null;
      }
      if (dto.projet_id !== undefined) {
        data.projet_id = dto.projet_id || null;
      }

      if (Object.keys(data).length > 0) {
        await conventionRepo.update(id, data);
      }

      if (dto.cotutelle_id !== undefined) {
        await cotutelleRepo.update(
          { convention_id: id },
          { convention_id: null },
        );

        if (dto.cotutelle_id) {
          const cotutelle = await cotutelleRepo.findOne({
            where: { id: dto.cotutelle_id, archive: false },
          });
          if (!cotutelle) {
            throw new NotFoundException('Cotutelle non trouvee');
          }

          await cotutelleRepo.update(dto.cotutelle_id, {
            convention_id: id,
          });
        }
      }
    });

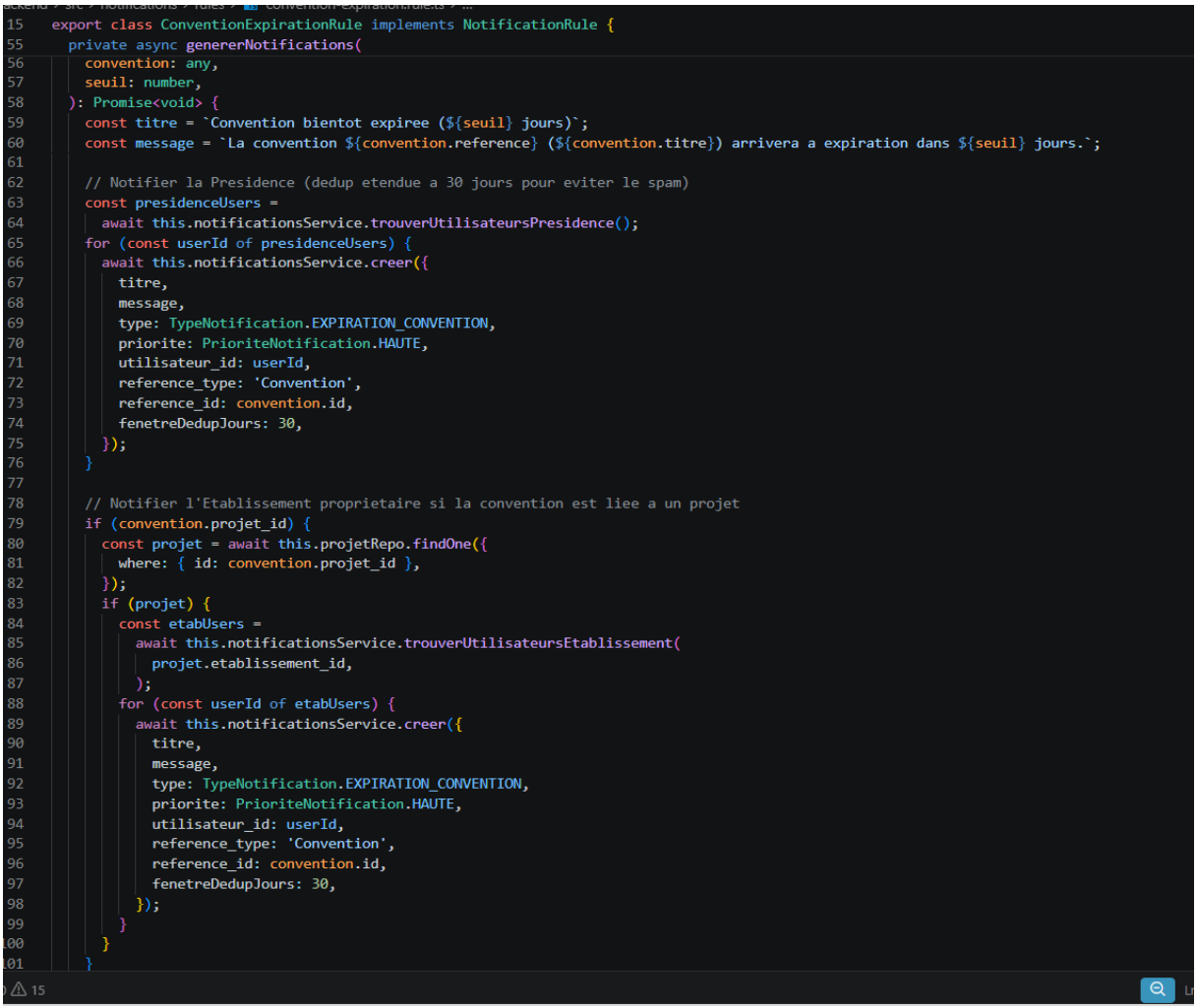
    return this.findOne(id);
  }
```

Figure 7.11 : Service des conventions — mise à jour

7.5.5. Notifications liées aux conventions

Les règles de notification couvrent l'approche de l'expiration et l'expiration constatée d'une convention. Les seuils d'alerte configurés sont de trente, quinze et sept jours. Les notifications sont adressées à la Présidence et, lorsqu'une convention est liée à un projet, aux utilisateurs de l'établissement concerné.

Un service de planification définit une exécution quotidienne de ces règles. Toutefois, la configuration de l'application examinée ne déclare pas l'initialisation de ScheduleModule.forRoot(), si bien que l'exécution automatique des tâches planifiées n'est pas attestée et doit être vérifiée avant d'être présentée comme effective.



```
export class ConventionExpirationRule implements NotificationRule {
  private async genererNotifications(
    convention: any,
    seuil: number,
  ): Promise<void> {
    const titre = `Convention bientot expiree (${seuil} jours)`;
    const message = `La convention ${convention.reference} (${convention.titre}) arrivera a expiration dans ${seuil} jours.`;

    // Notifier la Presidence (dedup etendue a 30 jours pour eviter le spam)
    const presidenceUsers =
      await this.notificationsService.trouverUtilisateursPresidence();
    for (const userId of presidenceUsers) {
      await this.notificationsService.creer({
        titre,
        message,
        type: TypeNotification.EXPIRATION_CONVENTION,
        priorite: PrioriteNotification.HAUTE,
        utilisateur_id: userId,
        reference_type: 'Convention',
        reference_id: convention.id,
        fenetreDedupJours: 30,
      });
    }

    // Notifier l'Etablissement proprietaire si la convention est liee a un projet
    if (convention.projet_id) {
      const projet = await this.projetRepo.findOne({
        where: { id: convention.projet_id },
      });
      if (projet) {
        const etabUsers =
          await this.notificationsService.trouverUtilisateursEtablissement(
            projet.etablissement_id,
          );
        for (const userId of etabUsers) {
          await this.notificationsService.creer({
            titre,
            message,
            type: TypeNotification.EXPIRATION_CONVENTION,
            priorite: PrioriteNotification.HAUTE,
            utilisateur_id: userId,
            reference_type: 'Convention',
            reference_id: convention.id,
            fenetreDedupJours: 30,
          });
        }
      }
    }
```

Figure 7.12 : Notifications d'expiration des conventions

7.5.6. Tableaux de bord

Les tableaux de bord fournissent des compteurs et des données de consultation selon le rôle et le périmètre de l'utilisateur. Les services correspondants construisent les résultats à partir des données disponibles.

```
export class TableauDeBordService {
  private async getCompteurs(
      where: {
        archive: false,
        statut: StatutProjet.EN_COURS,
        ...filtreEtab,
      },
    }),
    this.conventionRepo.count({
      where: { archive: false, ...filtreEtabProjet },
    }),
    this.conventionRepo.count({
      where: {
        archive: false,
        statut: In([StatutConvention.SIGNEE, StatutConvention.ACTIVE]),
        ...filtreEtabProjet,
      },
    }),
    this.conventionRepo.count({
      where: {
        archive: false,
        statut: StatutConvention.ACTIVE,
        ...filtreEtabProjet,
      },
    }),
    this.compterConventionsAEcheance(etablissementId),
    this.publicationRepo.count(),
  ]);

  const repartition_conventions =
    await this.getRepartitionConventions(etablissementId);

  return {
    partenaires,
    partenaires_actifs,
    partenariats,
    cotutelles,
    projets,
    projets_en_cours,
    conventions,
    conventions_signees,
    conventions_actives,
    conventions_a_echeance,
    publications,
    repartition_conventions,
  };
}
```

Figure 7.13 : Service des tableaux de bord

7.6. Tests réalisés

Les scénarios représentés dans les tests backend portent notamment sur la création et la soumission de projets, le respect du périmètre d'un établissement, les transitions de statut, les demandes de modification, la validation des dates et l'archivage. Le tableau ci-dessous reconstitue des résultats attendus à partir de ces scénarios et des règles implémentées.

Scénarios de vérification des règles de gestion des projets

Scénario	Résultat attendu reconstitué
Création d'un projet par un compte Établissement	Le projet est associé à l'établissement du compte.

Soumission d'un projet admissible au statut BROUILLON	Le projet passe à EN_ATTENTE, la date de soumission est renseignée, l'action est historisée et une notification est prévue pour la Présidence.
Nouvelle soumission d'un projet qui n'est plus en brouillon	La demande est refusée.
Consultation par un établissement d'un projet appartenant à un autre établissement	Les données hors périmètre ne sont pas exposées.
Envoi d'une demande invalide, notamment avec des dates non conformes	La requête est rejetée avec une erreur de validation.
Archivage d'un projet comportant des activités inachevées, lorsque cette condition s'applique	L'archivage est refusé ; il est accepté lorsque les conditions prévues sont satisfaites.

7.6.1. Difficulté liée au formulaire de création de convention

Le suivi du projet signale qu'une charge utile vide {} a été observée lors de la création d'une convention, le 30 octobre 2026. Les éléments disponibles ne permettent pas d'identifier avec certitude à quelle étape les valeurs ont été perdues. La cause la plus plausible est une anomalie dans la préparation ou la transmission des données du formulaire ; elle ne peut toutefois pas être attribuée à une règle imposant un lien avec un projet, un partenariat ou une cotutelle.

Dans la version actuelle, les trois identifiants de rattachement sont facultatifs et le schéma ne contient pas de règle root.liens. Le formulaire transmet les valeurs validées au moyen de handleSubmit. Les choix « Aucun » sont convertis en chaînes vides, puis le client de requête normalise ces valeurs en null. La règle historique évoquée sous la forme « au moins un lien » ne correspond donc pas au comportement actuel.

La version actuelle prévoit aussi une sélection automatique du statut « Signée » à la création lorsque la date de signature est atteinte, sauf si l'utilisateur choisit manuellement un autre statut. Le backend vérifie qu'une convention au statut SIGNEE possède une date de signature et refuse le statut EN_ATTENTE_SIGNATURE lorsque cette date est atteinte ou passée.

CHAPITRE VIII : PRÉSENTATION DE L'APPLICATION

Ce chapitre présente les principaux écrans de la plateforme, selon un parcours de démonstration qui couvre les espaces authentifiés de la Présidence, de l'Établissement et du Décideur, puis l'espace public ouvert au Visiteur.

8.1. Authentification

L'utilisateur accède à la plateforme par la page de connexion, où il saisit son adresse électronique et son mot de passe. Une fois authentifié, il est orienté vers l'espace correspondant à son rôle. Le Visiteur, lui, accède à l'espace public sans compte.

La figure 8.1 présente la page de connexion.

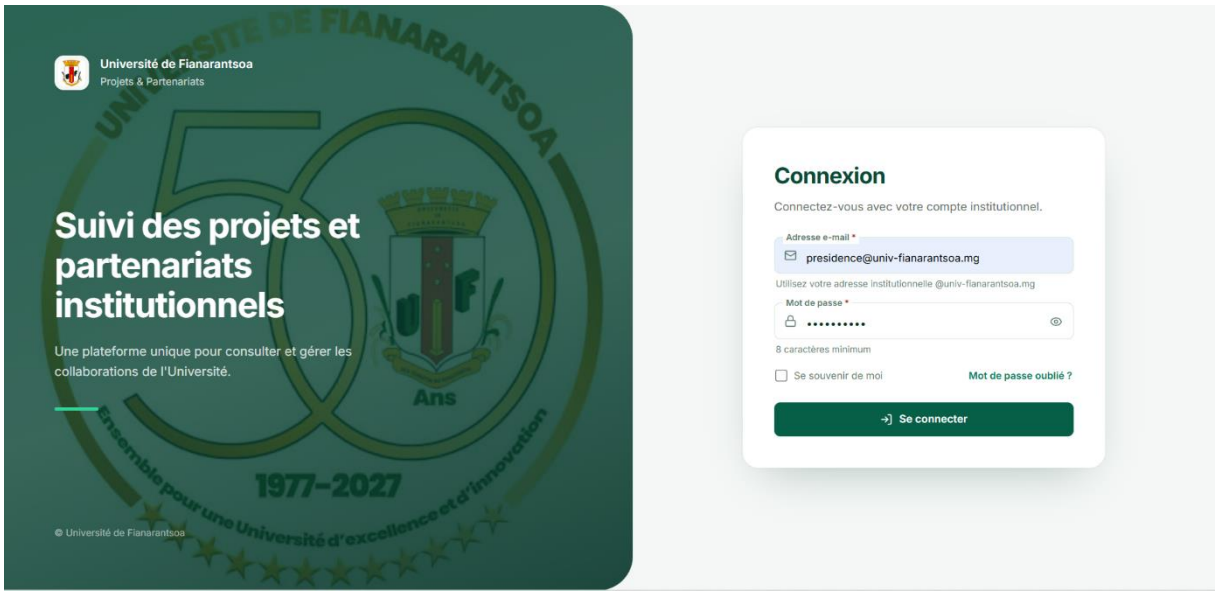


Figure 8.1 : Page de connexion de la plateforme

Source : réalisation personnelle.

Cette page est le point d'entrée des trois rôles authentifiés : elle permet de saisir ses identifiants et d'accéder à l'espace associé au rôle du compte.

8.2. Tableau de bord de la Présidence

La Présidence dispose d'une vue d'ensemble des projets, des partenariats et des conventions. Le tableau de bord indique notamment les partenaires actifs, les projets en cours, les conventions signées, les échéances proches, la répartition des projets par statut, les conventions à traiter et l'activité mensuelle.

La figure 8.2 présente le tableau de bord de la Présidence.

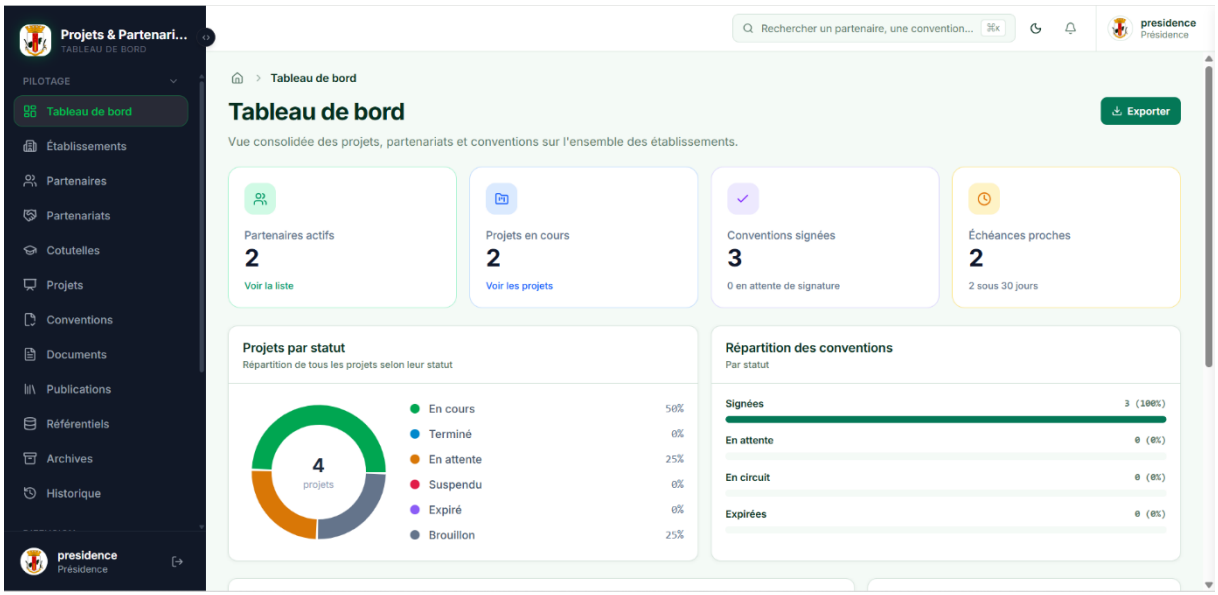


Figure 8.2 : Tableau de bord de la Présidence

Source : réalisation personnelle.

Cet écran regroupe des compteurs et des indicateurs de pilotage, et donne à la Présidence une vue consolidée des données couvertes par le tableau de bord.

8.3. Gestion des établissements

L'espace de gestion des établissements permet à la Présidence de consulter les établissements et leurs informations principales, et d'identifier les structures concernées par les projets et les partenariats.

La figure 8.3 présente la gestion des établissements.

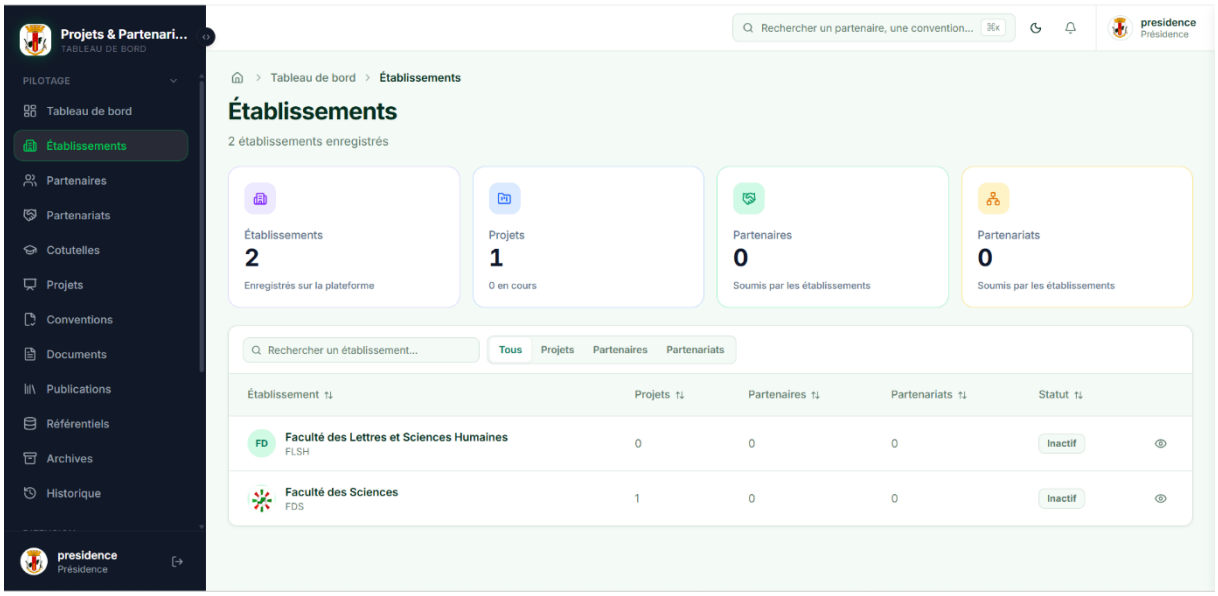


Figure 8.3 : Gestion des établissements

Source : réalisation personnelle.

Cet écran présente les établissements enregistrés et leurs informations associées, que la Présidence peut consulter.

8.4. Gestion des partenaires

La Présidence consulte la liste des partenaires, leurs informations principales et leur statut. Elle peut ouvrir la fiche détaillée d'un partenaire, avec les contacts et coordonnées qui lui sont associés.

La figure 8.4 présente la gestion des partenaires.

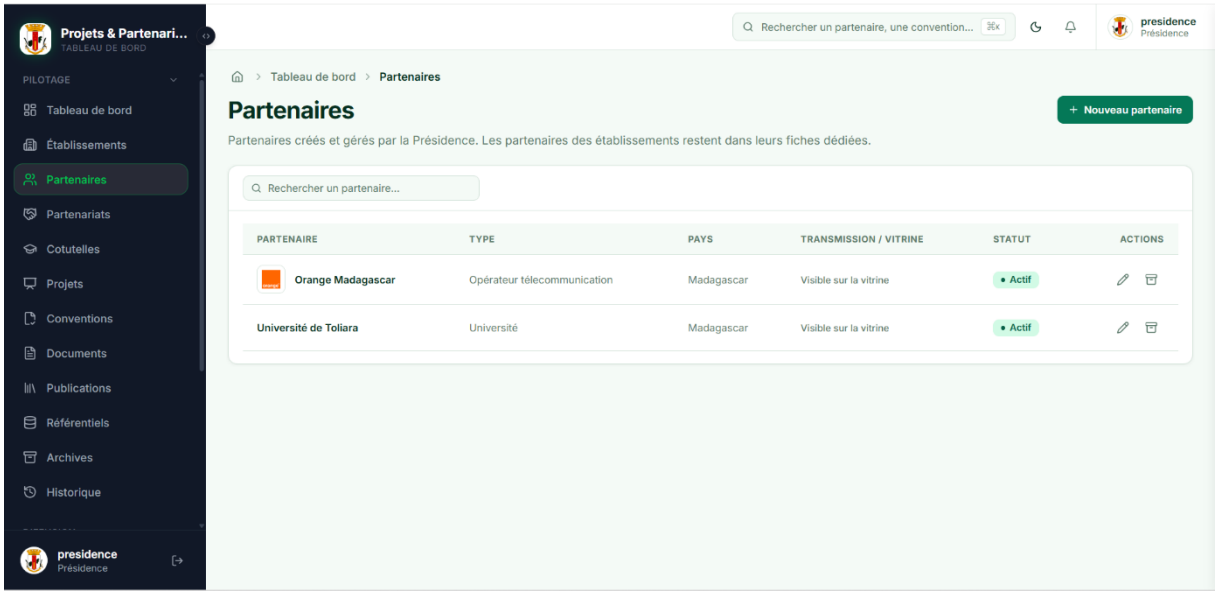


Figure 8.4 : Gestion des partenaires

Source : réalisation personnelle.

Cet écran présente les partenaires enregistrés et leurs principales informations. La fiche détaillée donne accès aux renseignements associés, dans la limite des droits de l'utilisateur.

8.5. Gestion des partenariats

La page des partenariats présente les collaborations enregistrées, leurs domaines et les informations associées. Les établissements gèrent les partenariats de leur périmètre, et la Présidence dispose des fonctions correspondant à ses droits.

La figure 8.5 présente la gestion des partenariats.

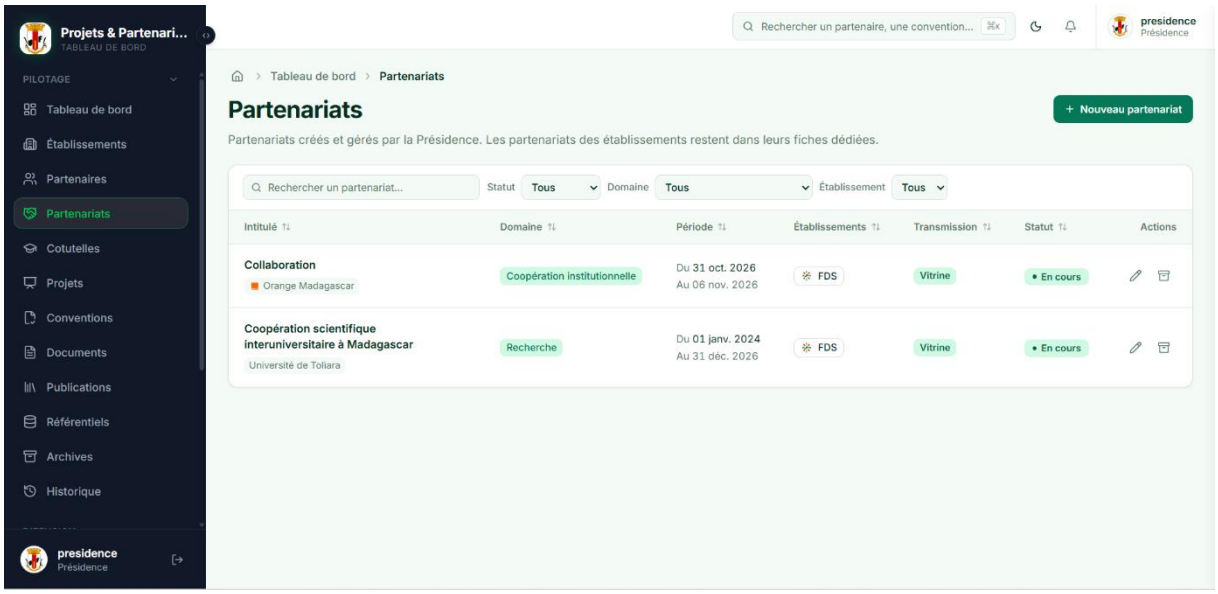


Figure 8.5 : Gestion des partenariats

Source : réalisation personnelle.

Cet écran permet de consulter les partenariats et leurs informations principales.

8.6. Création et suivi d'un projet

L'Établissement renseigne les informations d'un projet dans le formulaire prévu. Selon les données disponibles et les champs du formulaire, il peut y associer des partenaires, des bailleurs, des bénéficiaires, des sites et d'autres éléments.

La figure 8.6 présente le formulaire de création d'un projet.

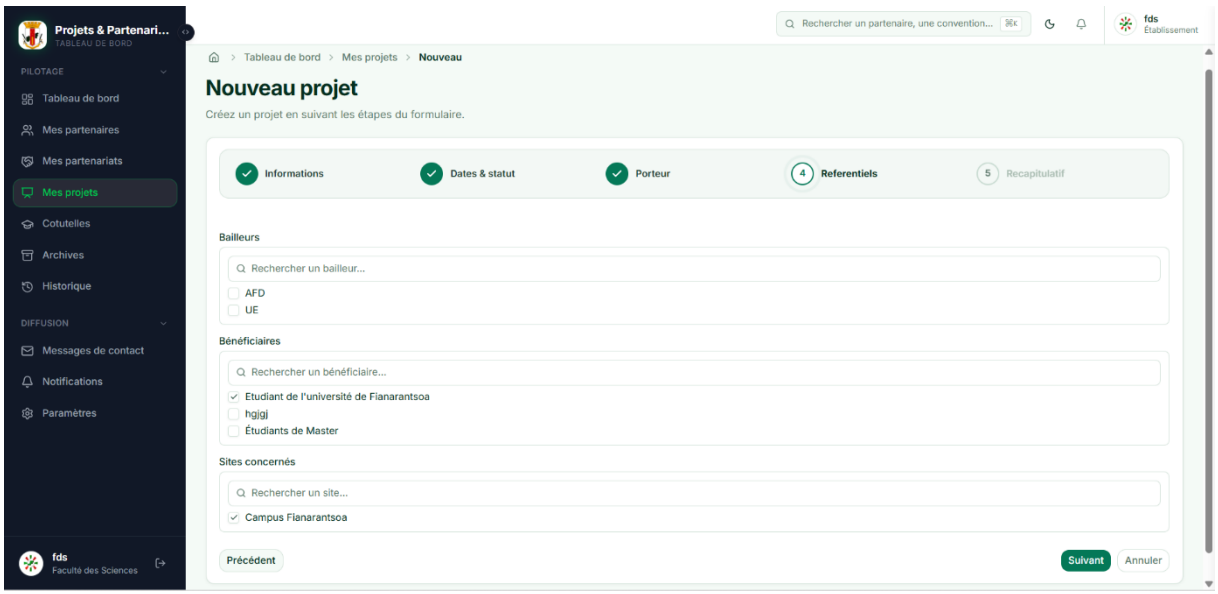


Figure 8.6 : Création d'un projet par un établissement

Source : réalisation personnelle.

Ce formulaire réunit les informations nécessaires à la création du projet par un établissement.

8.7. Soumission d'un projet

Une fois les informations nécessaires saisies, l'Établissement soumet le projet à la Présidence. Le projet prend alors l'état correspondant à la soumission, et une notification est envoyée à la Présidence.

La figure 8.7 présente la soumission d'un projet.

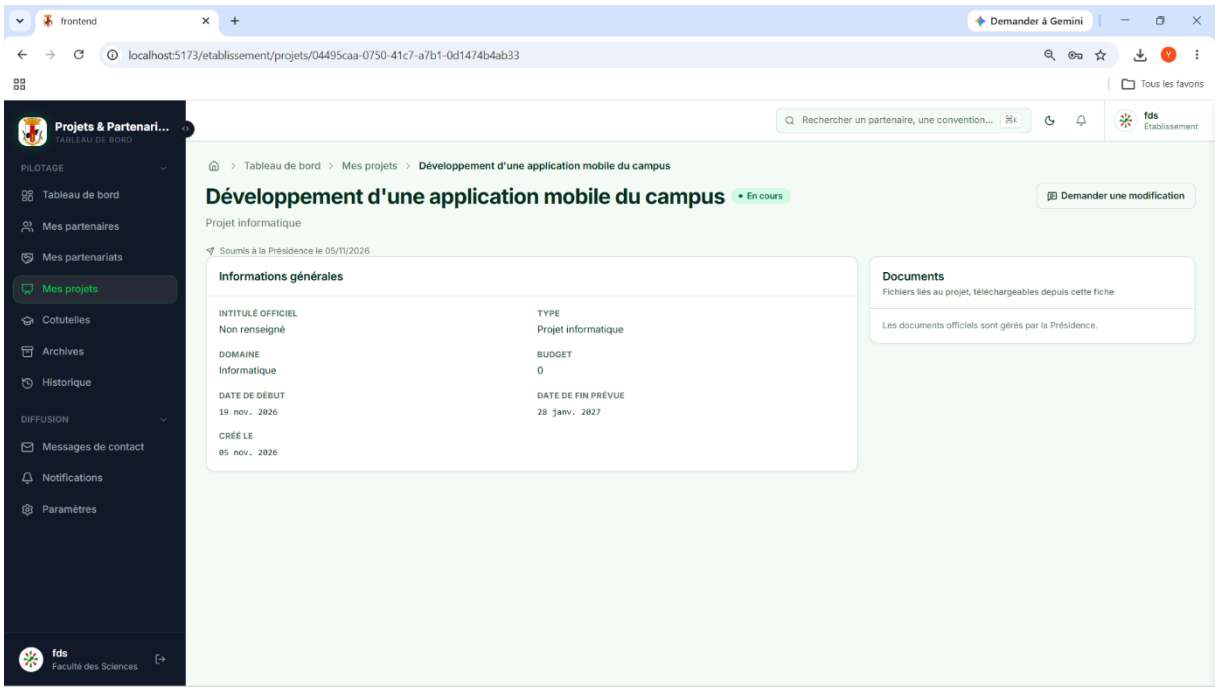


Figure 8.7 : Soumission d'un projet par un établissement

Source : réalisation personnelle.

Cet écran illustre la transmission d'un projet enregistré par l'Établissement. La Présidence peut ensuite consulter le projet soumis et publier les données destinées à être rendues publiques.

8.8. Publication et demande de modification d'un projet

La publication et la demande de modification sont deux opérations distinctes. La Présidence publie les données soumises destinées à être rendues publiques. L'Établissement peut demander la réouverture d'un projet de son périmètre, et la Présidence traite ensuite la demande.

La figure 8.8a présente la publication d'un projet par la Présidence.

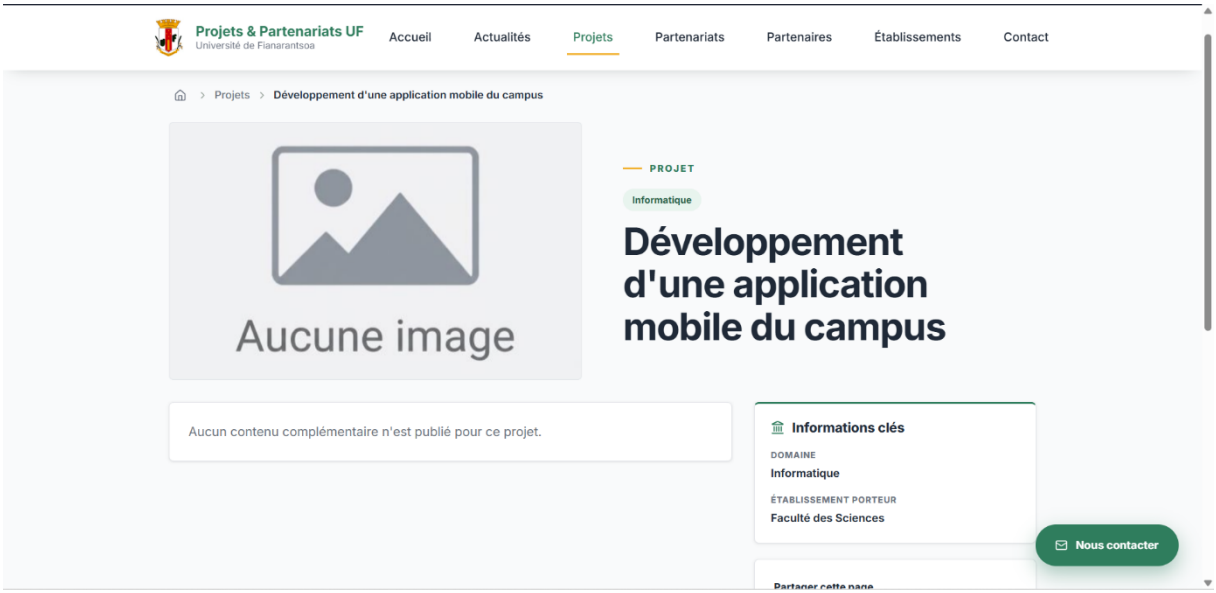


Figure 8.8a : Publication d'un projet par la Présidence

Source : réalisation personnelle.

Cet écran illustre la publication d'un projet soumis : la Présidence publie la donnée, sans que la capture laisse entendre qu'il existe une étape distincte de validation de la publication.

La figure 8.8b présente la demande de modification d'un projet par l'Établissement et son traitement par la Présidence.

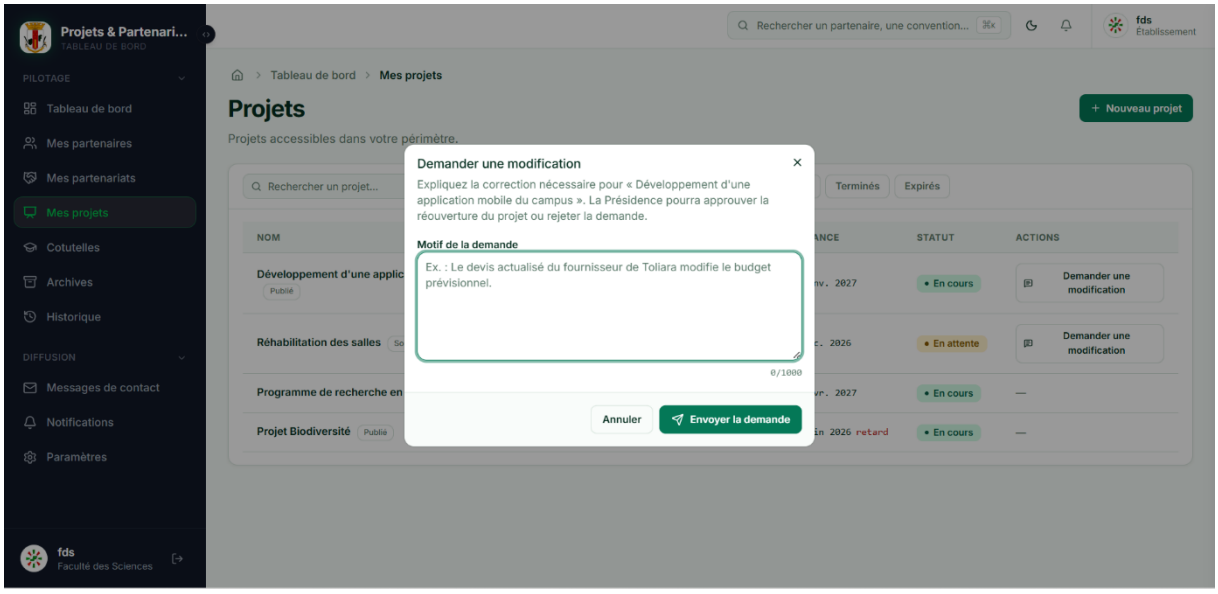


Figure 8.8b : Demande de modification d'un projet

Source : réalisation personnelle.

Cette figure illustre une opération distincte de la publication. La demande est adressée à la Présidence, qui peut l'approuver ou la rejeter.

8.9. Gestion des conventions

La Présidence consulte et gère les conventions. Les statuts prévus sont Brouillon, En attente de signature, Signée, Active, Suspendue et Expirée. Elle gère aussi les informations et les dates associées aux conventions, auxquelles l'Établissement n'a pas accès.

La figure 8.9 présente la gestion des conventions.

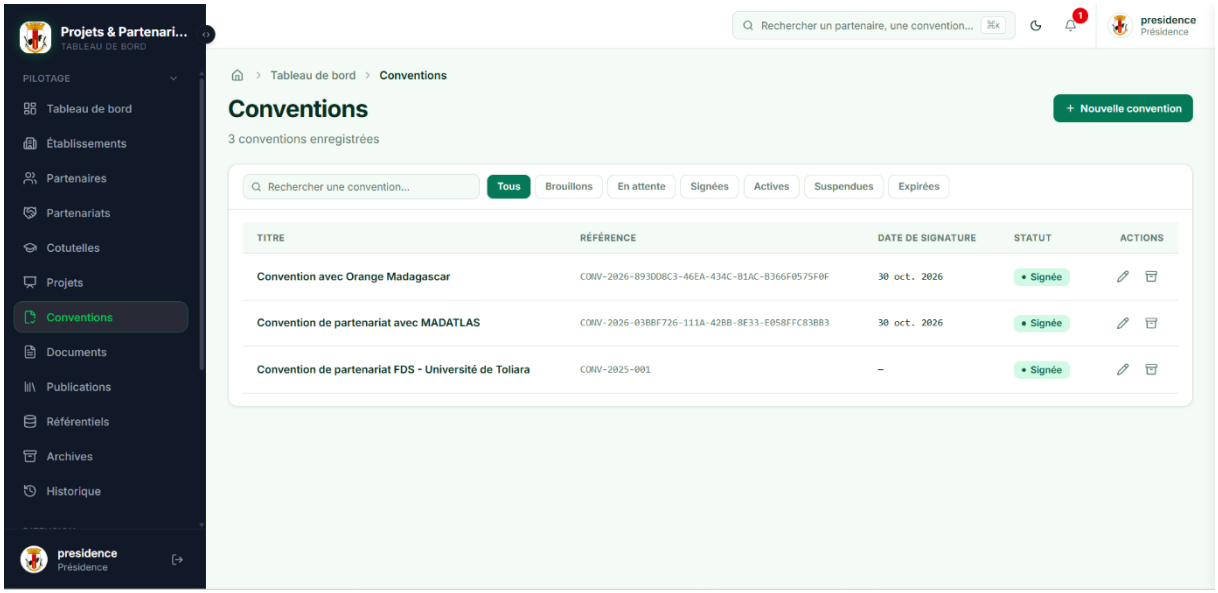


Figure 8.9 : Gestion des conventions par statut

Source : réalisation personnelle.

Cet écran présente les conventions et leur statut de suivi. La Présidence y consulte les informations et les dates associées, ainsi que les opérations autorisées.

8.10. Téléversement et consultation des documents

La Présidence téléverse les documents depuis le formulaire prévu. Les formats acceptés sont PDF, JPEG, PNG, DOC et DOCX, avec une taille maximale de dix mégaoctets par fichier. Chaque document est associé à une ressource.

La figure 8.10 présente le téléversement ou la consultation des documents.

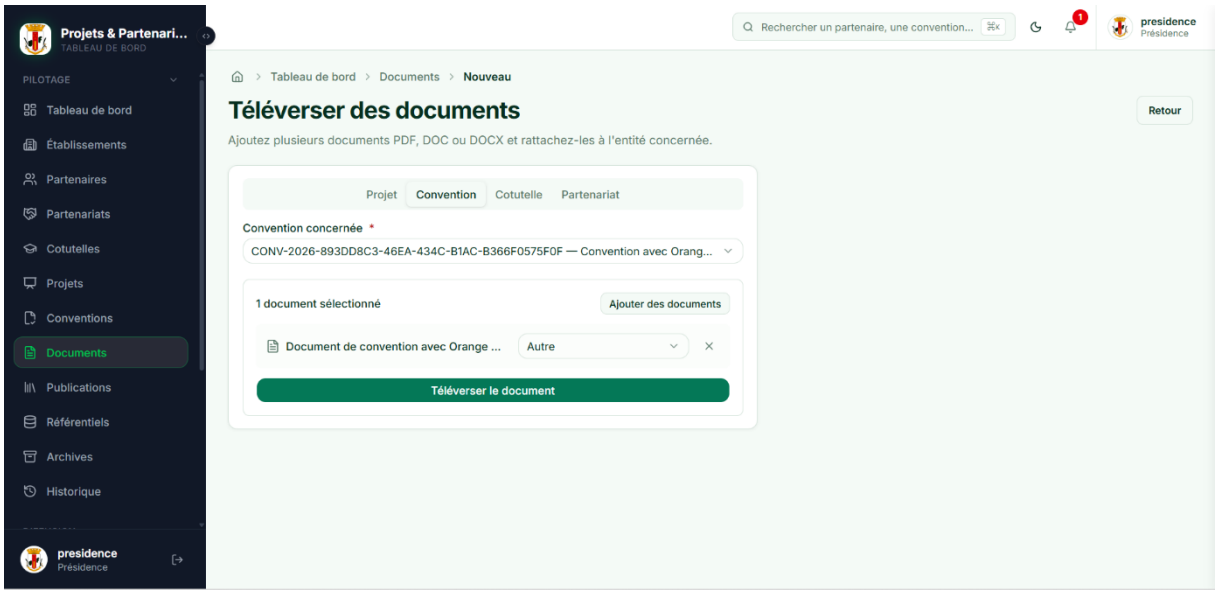


Figure 8.10 : Téléversement et consultation des documents

Source : réalisation personnelle.

Cet écran illustre l'ajout d'un document. Les conventions et les documents ne sont pas accessibles à l'Établissement.

8.11. Notifications

Le code prévoit des notifications événementielles liées aux soumissions ainsi que des règles de notification relatives aux échéances des conventions. L'activation effective de l'exécution périodique des alertes d'échéance n'est pas confirmée par la configuration examinée.

La figure 8.11 présente une notification de soumission d'un nouveau projet d'un établissement.

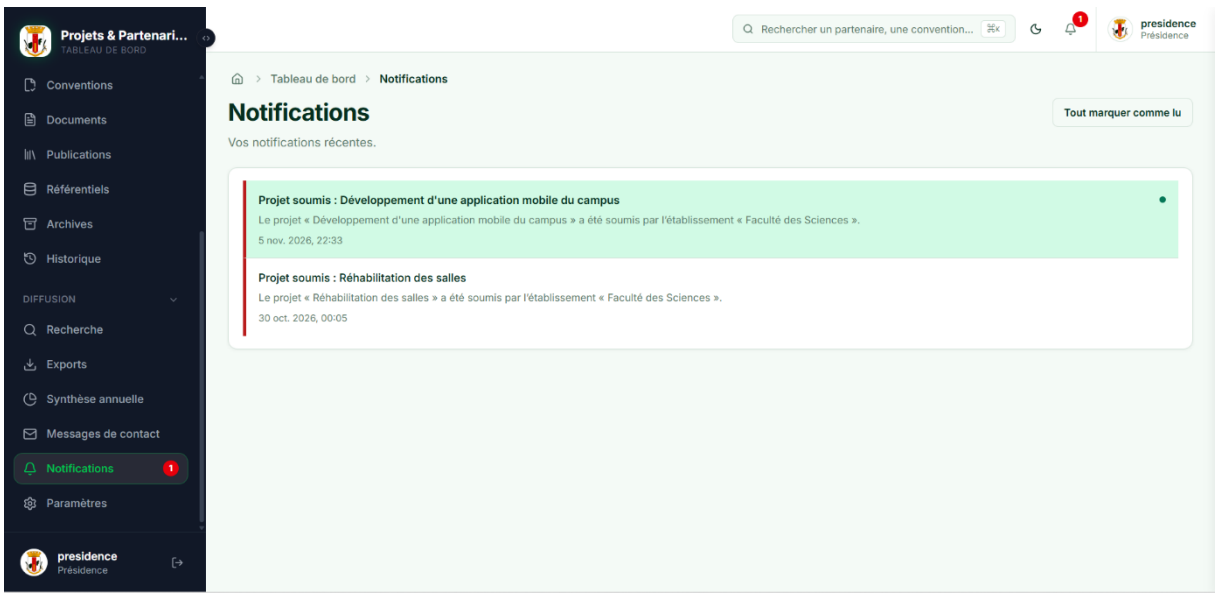


Figure 8.11 : Notification liée à une soumission d'un nouveau projet

Source : réalisation personnelle.

Cet écran illustre l'affichage des notifications destinées à la présidence.

8.12. Consultation des tableaux de bord

Les tableaux de bord varient selon le rôle. La Présidence consulte les indicateurs globaux. L'Établissement accède à un tableau de bord limité à ses propres données, notamment ses partenaires, ses projets et ses partenariats. Le Décideur consulte des indicateurs en lecture seule, parmi lesquels des comparaisons entre établissements et des informations sur les conventions.

La figure 8.12 présente un tableau de bord d'un établissement.

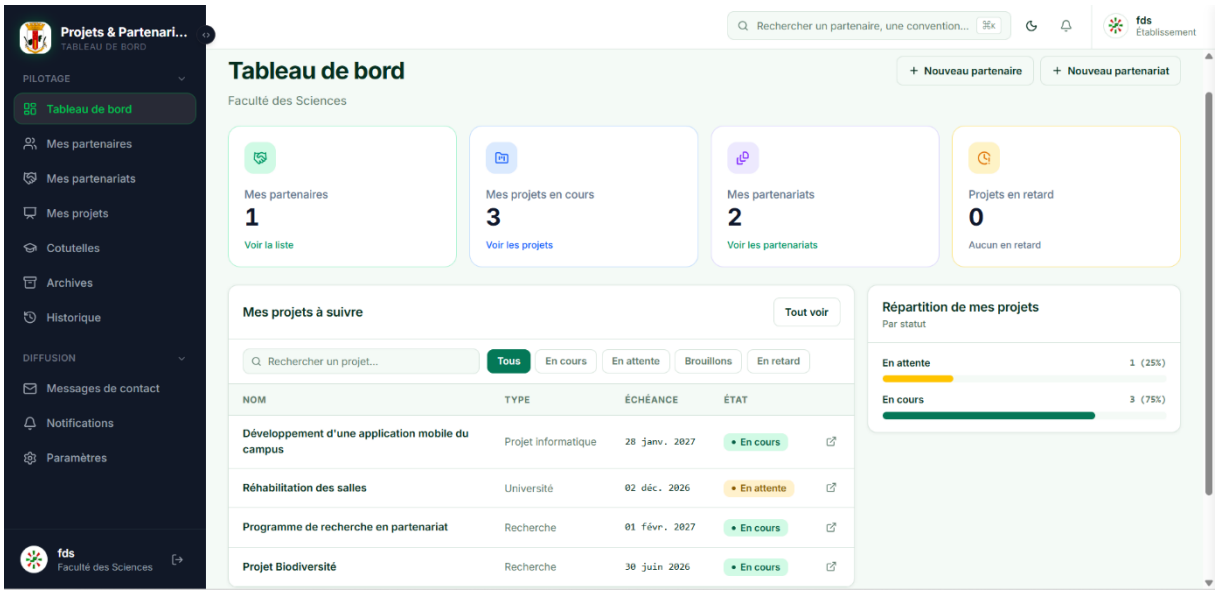


Figure 8.12 : Consultation des tableaux de bord

Source : réalisation personnelle.

Cet écran illustre la consultation, par un établissement, de son tableau de bord.

8.13. Espace public

L'espace public est ouvert au Visiteur sans compte. Il présente les informations rendues publiques : établissements, partenaires, projets, partenariats et publications prévues pour la consultation publique.

La figure 8.13 présente l'espace public.

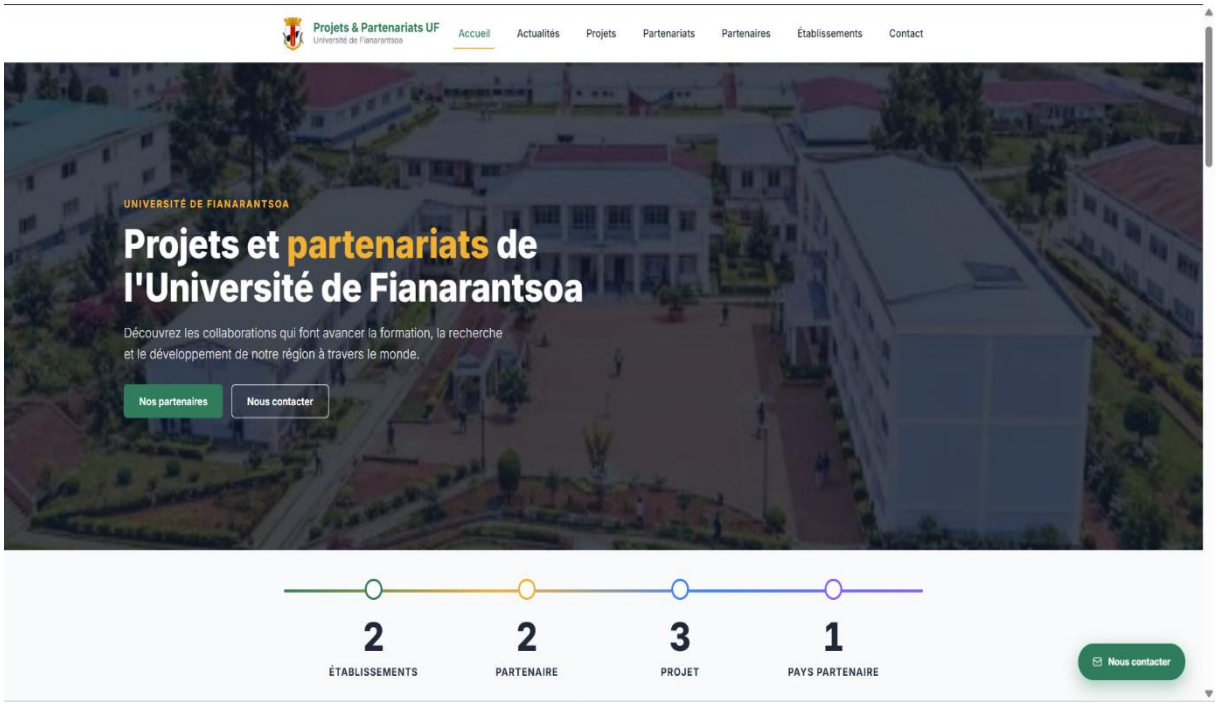


Figure 8.13 : Espace public de la plateforme

Source : réalisation personnelle.

Cet écran illustre l'accès du Visiteur aux contenus publics. Les documents et les conventions n'y sont pas exposés : seules les informations rendues publiques sont consultables.

CHAPITRE IX : ÉVALUATION ET SUGGESTIONS

Ce chapitre présente les fonctionnalités réalisées, les compare aux pratiques antérieures décrites pour le projet, puis expose les limites de la version développée et les pistes d'évolution.

9.1. Évaluation de l'application

Avant la plateforme, le suivi des projets et des partenariats reposait sur des systèmes internes de l'Université, des fichiers bureautiques ou l'absence de solution centralisée.

Le tableau 9.1 compare ces pratiques, sous réserve de cette vérification, aux fonctions de la plateforme.

Tableau 9.1 : Comparaison entre les pratiques antérieures et les fonctionnalités de la plateforme

Critère	**Pratiques antérieures, à confirmer**	**Plateforme réalisée**
Centralisation des données	Données réparties entre plusieurs supports	Données gérées dans une application commune
Gestion des projets	Suivi dépendant des outils utilisés par les services	Création, soumission, consultation et publication des projets
Gestion des partenaires et partenariats	Informations dispersées ou suivies séparément	Gestion structurée des partenaires, des contacts et des partenariats
Gestion des conventions	Suivi manuel des informations et des échéances	Gestion des statuts, des dates et de l'archivage
Gestion documentaire	Fichiers conservés dans des emplacements distincts	Téléversement et consultation de documents selon les droits d'accès
Échéances des conventions	Repérage manuel des échéance	Dates et règles de notification présentes ; déclenchement automatique périodique
Contrôle des accès	Accès dépendant du support utilisé	Accès différencié selon le rôle et, pour l'Établissement, son périmètre
Tableaux de bord	Consolidation manuelle des informations	Indicateurs accessibles à la Présidence, à l'Établissement dans son périmètre et au Décideur en lecture seule

Source : informations recueillies auprès des acteurs du projet et fonctionnalités observables dans la version étudiée.

Le tableau fait ressortir les fonctions de centralisation, de gestion structurée et de contrôle d'accès de l'application.

La plateforme regroupe les données relatives aux projets, aux partenaires, aux partenariats, aux conventions et aux documents associés. La Présidence dispose d'indicateurs globaux, l'Établissement consulte les informations de son périmètre et le Décideur accède aux données en lecture seule. Ce sont là des apports de l'application, qui ne suffisent pas à établir leur effet sur les pratiques réelles des utilisateurs. Les conventions disposent de statuts, de dates, d'un archivage et de règles liées aux échéances.

9.2. Contributions académiques et professionnelles

Sur le plan académique, le projet a mobilisé des notions de conception de systèmes d'information et de modélisation UML. Le mémoire présente le diagramme global des cas d'utilisation, des diagrammes de séquence, un modèle de domaine, un diagramme de classes et un diagramme de paquetage. Ces modèles décrivent les acteurs, les échanges et l'organisation des principaux éléments du système.

La réalisation repose sur une architecture organisée par domaines et sur NestJS, TypeScript, TypeORM et PostgreSQL pour le backend. Le recours aux migrations, aux objets de transfert, aux mécanismes de validation, à l'authentification et au contrôle des rôles a permis de mettre en pratique plusieurs notions de génie logiciel.

Sur le plan professionnel, la plateforme offre un espace commun pour gérer les informations liées aux projets et aux partenariats, ainsi que des tableaux de bord dont les données varient selon le rôle et le périmètre d'accès. Ces apports décrivent les fonctions livrées ; leur appropriation par les services et leur incidence sur le pilotage restent à évaluer.

9.3. Limitations de l'étude et pistes de recherche futures

La version évaluée ne couvre pas toutes les fonctions envisagées pour le suivi des projets. La gestion des activités et du chronogramme n'est pas opérationnelle et reste une limite du périmètre réalisé ; l'existence éventuelle de structures de données correspondantes n'en fait pas une fonctionnalité utilisable.

La gestion des conventions comporte des statuts de suivi, des dates d'échéance et des fonctions d'archivage. La plateforme prévoit des notifications liées aux échéances ; leur déclenchement dépend de la configuration et de l'exécution du service concerné. La version étudiée ne met pas en place de circuit de validation des conventions par étapes.

La couverture des tests et la vérification des parcours fonctionnels forment une autre limite : les éléments disponibles ne permettent d'établir ni la couverture réelle ni les résultats d'exécution de tous les tests.

Une première piste d'évolution consiste à étudier le déploiement de l'application sur un serveur afin de la rendre accessible dans un environnement partagé. Cette évolution nécessite une infrastructure adaptée, des sauvegardes et une supervision.

Une deuxième piste concerne le renforcement des tests et de la sécurité : élargir les scénarios testés, vérifier régulièrement les contrôles d'accès et améliorer le suivi des erreurs.

Une troisième piste porte sur l'ergonomie, avec la simplification des formulaires, l'amélioration de la navigation et l'adaptation des écrans aux besoins des différents profils.

Une quatrième piste porte sur l'étude des besoins liés à la gestion des activités et du chronogramme, ainsi que sur l'éventualité d'un circuit de validation des conventions par étapes.

CONCLUSION

Au terme de ce stage effectué au sein d'ITDC MADA, j'ai contribué principalement au développement backend d'une plateforme de gestion et de suivi des projets et partenariats de l'Université de Fianarantsoa. Le projet a été commandité par l'équipe Projet et Partenariat de la Présidence. La plateforme organise les données par domaines fonctionnels et distingue les droits des profils Présidence, Établissement et Décideur, ainsi que l'accès public du Visiteur.

La réalisation s'appuie sur NestJS, TypeScript, TypeORM et PostgreSQL pour le backend, ainsi que React et Vite pour le frontend. Docker Compose décrit les services de développement. Le mémoire présente les choix de conception, les fonctionnalités, les règles d'accès et les limites constatées. Les résultats des tests et l'incidence de la plateforme sur les pratiques des utilisateurs ne sont pas affirmés sans éléments d'évaluation correspondants.

La gestion des activités et du chronogramme ne constitue pas une fonctionnalité opérationnelle de la version décrite. Les notifications liées aux échéances sont prévues par l'application ; leur fonctionnement périodique dépend de la configuration du service de tâches. Les perspectives portent sur la validation des besoins par le commanditaire, le renforcement des tests fonctionnels et l'évaluation de l'usage de la plateforme.

BIBLIOGRAPHIE

Ouvrages et normes de référence

FOWLER Martin, UML Distilled: A Brief Guide to the Standard Object Modeling Language, 3e éd., Boston, Addison-Wesley, 2003.

OBJECT MANAGEMENT GROUP, Unified Modeling Language, Version 2.5.1, spécification formelle, 2017.

SOMMERVILLE Ian, Software Engineering, 10e éd., Harlow, Pearson, 2021.

WEBOGRAPHIE

DOCKER, Docker Compose [en ligne], s.d. Disponible à l'adresse : https://docs.docker.com/compose/ [consulté le 20.08.2026].

NESTJS, Documentation officielle de NestJS [en ligne], s.d. Disponible à l'adresse : https://docs.nestjs.com/ [consulté le 25.08.2026].

OBJECT MANAGEMENT GROUP, About the Unified Modeling Language Specification Version 2.5.1 [en ligne], 2017. Disponible à l'adresse : https://www.omg.org/spec/UML/2.5.1/About-UML [consulté le 17.08.2026].

POSTGRESQL GLOBAL DEVELOPMENT GROUP, PostgreSQL 16 Documentation [en ligne], s.d. Disponible à l'adresse : https://www.postgresql.org/docs/16/ [consulté le 02.09.2026].

REACT, Learn React [en ligne], s.d. Disponible à l'adresse : https://react.dev/learn [consulté le 10.09.2026].

TYPEORM, Getting Started [en ligne], s.d. Disponible à l'adresse : https://typeorm.io/docs/getting-started/ [consulté le 20.09.2026].

VITE, Getting Started [en ligne], s.d. Disponible à l'adresse : https://vite.dev/guide/ [consulté le 06.10.2026].

GLOSSAIRE

API — Interface HTTP utilisée par le frontend pour accéder aux fonctions du backend.

Authentification — Vérification de l'identité d'un utilisateur avant l'accès aux fonctions protégées.

Backend — Partie serveur qui applique les règles métier, contrôle les accès et échange avec la base de données.

Base de données relationnelle — Dépôt de données organisé en tables liées; le projet utilise PostgreSQL.

Conteneur — Environnement isolé utilisé pour exécuter un service; les services de développement sont décrits avec Docker Compose.

DTO — Objet qui transporte les données d'une requête et dont les champs sont validés par le backend.

Endpoint — Point d'accès HTTP associé à une opération de l'API.

Frontend — Interface web développée avec React et Vite dans le projet.

JWT — Jeton signé utilisé dans le mécanisme d'authentification décrit par le projet.

ORM — Outil qui relie les objets du code aux données relationnelles; TypeORM assure cette fonction dans le projet.

Persistance — Conservation des données au-delà de l'exécution d'une requête ou d'un processus.

Rôle — Profil d'accès qui détermine les actions disponibles pour un utilisateur.

TypeORM — Outil de persistance utilisé par le backend pour accéder à PostgreSQL.

UML — Langage de modélisation utilisé pour représenter notamment les cas d'utilisation, séquences et classes.

UUID — Identifiant universellement unique utilisé dans le modèle de données présenté.

ANNEXES

Annexe A — Sources techniques de la version examinée

Les fichiers ci-dessous ont été consultés pour décrire la configuration technique de la version étudiée. Ils permettent de vérifier les dépendances et les services déclarés, mais ne constituent pas à eux seuls une preuve d'exécution des tests ni une validation fonctionnelle.

README.md — présentation générale du dépôt.

backend/README.md — documentation du backend.

frontend/README.md — documentation du frontend.

backend/package.json — dépendances déclarées du backend.

frontend/package.json — dépendances déclarées du frontend.

docker-compose.yml — définition des services de développement.

TABLE DES MATIÈRES

AVANT-PROPOS 6

REMERCIEMENTS I

Résumé III

Abstract III

LISTE DES FIGURES IV

LISTE DES TABLEAUX VI

LISTE DES ABREVIATIONS VII

GLOSSAIRE VIII

SOMMAIRE (3 NIVEAUX) X

INTRODUCTION 1

PARTIE I : PRESENTATION GENERALE 3

CHAPITRE I : PRÉSENTATION DE L'ÉCOLE de Management et d'Innovation Technologique 4

1.1 Historique : 4

1.2 Missions 5

1.3 Formations existantes 5

1.4 Organigramme de l'EMIT 10

CHAPITRE II : PRÉSENTATION D'ITDC MADA 12

2.1 Historique 12

2.2 Services 12

2.3 Formations 13

2.4 Mission 13

2.6 Méthode du travail 14

CHAPITRE III : PRÉSENTATION DU PROJET 16

3.1. Formulation du projet 16

3.2. Justification de l'importance du sujet 17

3.3. Objectifs et besoins des utilisateurs 17
3.4. Moyens nécessaires à la réalisation du projet 20
3.5. Chronogramme de réalisation du projet 23
3.6. Résultat attendu 25
PARTIE II : ANALYSE ET CONCEPTION 27
CHAPITRE IV : ANALYSE PRÉALABLE 28
4.1. Analyse de l'existant 28
4.2. Choix de la solution 30
4.3. Choix de la méthode de conception et de gestion du projet 30
4.4. Choix de l'outil de conception 31
4.5. Choix du langage de programmation 31
4.6. Choix des frameworks et des technologies d'interface 31
4.7. Choix de l'environnement de développement 32
4.8. Choix du système de gestion de base de données 33
CHAPITRE V : CONCEPTION DU PROJET 34
5.1. Présentation de la méthode de conception 34
5.2. Dictionnaire des données 34
5.3. Règles de gestion 42
5.4. Diagramme de cas d'utilisation 45
5.5. Description textuelle des cas d'utilisation 47
5.6. Diagramme de séquence système 49
5.7. Modèle de domaine 49
CHAPITRE VI : CONCEPTION DÉTAILLÉE 51
6.1. Architecture de l'application 51
6.2. Diagramme de séquence de conception 52
6.3. Diagramme de classe 54
6.4. Diagramme de paquetage 55

PARTIE III : MISE EN ŒUVRE ET RÉSULTATS 56
CHAPITRE VII : MISE EN ŒUVRE 57
7.1. Architecture du système 57
7.2. Configuration des outils de conception 57
7.3. Génération de la base de données 58
7.4. Configuration de l'environnement de développement 59
7.5. Extraits de code des fonctionnalités principales 61
7.6. Tests réalisés 69
CHAPITRE VIII : PRÉSENTATION DE L'APPLICATION 71
8.1. Authentification 71
8.2. Tableau de bord de la Présidence 71
8.3. Gestion des établissements 72
8.4. Gestion des partenaires 73
8.5. Gestion des partenariats 73
8.6. Création et suivi d'un projet 74
8.7. Soumission d'un projet 75
8.8. Publication et demande de modification d'un projet 75
8.9. Gestion des conventions 77
8.10. Téléversement et consultation des documents 77
8.11. Notifications 78
8.12. Consultation des tableaux de bord 79
8.13. Espace public 79
CHAPITRE IX : ÉVALUATION ET SUGGESTIONS 81
9.1. Évaluation de l'application 81
9.2. Contributions académiques et professionnelles 82
9.3. Limitations de l'étude et pistes de recherche futures 82
CONCLUSION 84

BIBLIOGRAPHIE XII
Sources primaires du projet ..... **Erreur ! Signet non défini.**
Ouvrages et normes de référence XII
WEBOGRAPHIE XII
Dépôt GitHub du projet ..... **Erreur ! Signet non défini.**
GLOSSAIRE XIII
ANNEXES XIV
Annexe A — Sources techniques de la version examinée XIV
TABLE DES MATIÈRES XV